Deep Bayesian Inference for Control: Normalizing Flow – Based Approaches to System Identification, State Estimation, and Optimal Control

genehmigte Dissertation
zur Erlangung des akademischen Grades der Doktor-Ingenieurin (Dr.-Ing.).
an der
Fakultät für Angewandte Informatik
der
Universität Augsburg

vorgelegt von
Yi Zhang
aus
Tangshan, China

2026

Dekan: Prof. Dr. Jörg Hähner
Doktorvater: Prof. Dr.-Ing. Lars Mikelsons

I dedicate this thesis to my family.

Contents

Acronyms

List of Figures

List of Tables

Chapter I

Introduction

I.1 Motivation

In this seemingly certain world, uncertainty lingers like a shadow, yet from the uncertain, we often find certainty.

The paradoxical relationship between certainty and uncertainty appears across disciplines, demonstrating how truth often arises from navigating ambiguity. Quantum mechanics redefines predictability under randomness in the subatomic world, where Newtonian deterministic physics collapses. This uncertainty led to groundbreaking laws (e.g. Heisenberg's Uncertainty Principle), which now underpin modern technologies like quantum computing and all forms of spectroscopy. In medical diagnosis, for example, a patient may present vague symptoms, such as fatigue or pain, that could indicate anything from stress to cancer. Doctors converge on the cause through iterative hypothesis-testing approaches. Climate science also epitomizes how we manage uncertainty. Climate models incorporate a wide range of sources of variability and uncertainty. A common way to address this uncertainty is to combine the results of different simulation models. By running multi-model ensembles, scientists can identify consistent patterns across the uncertainty.

From quantum randomness to medical diagnosis to climate models, uncertainty does not destroy deterministic systems, but instead provides access to identifying them. This perspective supports the use of probabilistic inference methods, which offer a rigorous framework for representing uncertainty and reasoning from uncertainty. In statistics, modeling the uncertainty of variables from given data samples using a probability distribution rather than a point estimate is known as inference *. The estimated distribution facilitates uncertainty quantification and provides a measure of the reliability of the results, making it particularly well suited for real-world applications.

Probabilistic inference grounded in Bayes' rule is known as ***Bayesian inference***. Bayes'

*We note that "inference" is an overloaded term in this context. We distinguish between: (i) Bayesian Inference, the statistical goal of recovering the posterior distribution; (ii) Variational Inference, the mechanism to achieve Bayesian inference; (iii) Inference: the process that maps data to latent space in generative models, which is in contrast to generation; and (iv) Computational Inference, the deep learning engineering practice of executing the generation pass (for generative models meaning from latent to data) of a trained network on test data.

rule represents unknown random variables through beliefs (the prior). The beliefs are updated using observed data (likelihood), yielding the posterior distribution. A simplified formulation is:

$$\text{posterior} = \text{prior} \times \text{likelihood} \,/\, \text{total evidence (or marginal likelihood)}.$$

For example, we can illustrate Bayes' rule using the medical diagnosis scenario: the objective is to obtain the probability of a disease given a positive test result:

$$P(\text{Disease}|\text{Positive}) = \frac{P(\text{Positive}|\text{Disease}) \times P(\text{Disease})}{P(\text{Positive})},$$
$$P(\text{Positive}) = P(\text{Positive}|\text{Disease}) \times P(\text{Disease}) + P(\text{Positive}|\overline{\text{Disease}}) \times P(\overline{\text{Disease}}),$$

where $\overline{\text{Disease}}$ denotes "No Disease".

In a more general Bayesian context, inference is the process of obtaining the posterior distribution of target variables from observed data. This perspective extends naturally to modern machine learning, where unknown values, structures, and policies can all be understood as random variables and probability distributions (**?**).

Especially in **DGM!**s (**DGM!**s), Bayesian inference plays an essential role (**?**). By introducing random variables to represent complex data in latent space, **DGM!**s enable data synthesis from latent variables (generation) based on recognition of the underlying latent representation from data (inference). However, directly inferring the posterior distribution over the latent variables using Bayes' rule is generally intractable, since all possible values of the latent variables are rarely obtainable in complex and high-dimensional models. Addressing this challenge is essential to fully realize the potential of **DGM!**s. **VAE!**s (**VAE!**s) (**?**) tackle this problem by introducing an inference network as a tractable approximation to the posterior. **NF!**s (**NF!**s) (**?**) construct bijective transformations between observed data and latent variables, achieving exact computation of posteriors. Diffusion models (**?**) sidestep direct calculation of the posterior distribution over latent variables by designing it as a simple Gaussian, and frame both inference and generation as an iterative denoising process from noise through learned transformations.

In summary, **DGM!**s are inherently probabilistic; through probabilistic modeling and inference, they are powerful tools for extracting latent information and representing uncertainty in complex data. These capabilities have been demonstrated in numerous applications involving images, text, and speech. The same principles extend naturally to scientific and engineering domains. One prominent question is: how can **DGM!**s support the identification and modeling of complex, stochastic, nonlinear dynamical systems? In control theory, the process of building mathematical models of dynamical systems from observed input-output data is called ***system identification***. Interpreting system

identification in probabilistic terms, ***dynamics modeling*** means inferring the posterior of the next states based on the previous states and system inputs, while ***parameter identification*** means estimating the posterior over system parameters given observations. Identifying systems probabilistically enables uncertainty quantification, which is crucial for reliable and robust decision-making in real-world applications.

Classical methods for system identification rely on linearized models or prespecified nonlinear structures. Model parameters are then estimated from observed data using estimation techniques. Subsequently, a validation process is employed to assess and select the most appropriate model structure. Although these methods offer strong physical interpretability and are often data-efficient, they exhibit several limitations: parameter estimation can be computationally inefficient with large amounts of data; results are sensitive to the choice of model structures and depend on expert knowledge; uncertainty is difficult to handle and quantify; and they are generally inapplicable to partially observable systems and high-dimensional raw data.

Alongside the demand for uncertainty quantification, deep learning methods are well suited for system identification because of their strength in representing information from complex data, their flexibility in function approximation, and their scalability to high-dimensional data. Furthermore, their compatibility with gradient-based optimization enables efficient navigation of high-dimensional landscapes, allowing optimizers to find solutions nearly equivalent to the global optimum. This provides a distinct advantage over classical methods, where low-dimensional parameter spaces are more prone to being trapped in poor local minima that fail to capture the underlying dynamics. Taking uncertainty quantification into account, a simple yet effective technique is Gaussian Process Regression (**?**), which involves assigning a Gaussian process prior to the latent regressor and performing statistical conditioning based on the available observations. This method has gained widespread adoption due to its nonparametric character and adaptability. Nonetheless, its significant computational and memory requirements limit its feasibility when dealing with large datasets. Another straightforward approach to combining the advantages of deep learning methods and uncertainty modeling is the **BNN!** (**BNN!**) (**?**). Unlike standard neural networks that produce point estimates, **BNN!**s assign probability distributions over the network parameters, thereby inducing distributions for outputs. However, **BNN!**s show limitations in expressiveness because the approximate inference may not reliably capture true posterior behavior (**?**). Therefore, the adaptation and application of **DGM!**s are essential for achieving rich representational capacity, flexible modeling frameworks, and the ability to effectively handle large-scale data.

Compared with classical Bayesian inference methods, **DGM!**s provide a powerful route to amortized Bayesian inference. Instead of solving a separate inverse problem for each new observation, a single conditional generative model is optimized over many simulated

or measured samples and then reused to infer posterior distributions for unseen cases. This amortization is particularly attractive for practical engineering datasets, where repeated parameter estimation must be performed under noise, model uncertainty, and large numbers of operating conditions, for example, in end-of-line testing of large numbers of products.

Different **DGM!**s have been investigated in system identification applications. Here, we briefly evaluate three prominent frameworks: **VAE!**s, **NF!**s, and diffusion models. **VAE!**s work by compressing complex data into a lower-dimensional latent space and then learning to reconstruct original data from compressed latent variables. This makes **VAE!**s flexible enough to work with time-series modeling architectures such as **RNN!** (**RNN!**) (**?**) and **NARX!** (**NARX!**) (**?**), which is beneficial for uncovering time-dependent features in data. However, **VAE!**s suffer from a well-known problem of variance shrinkage (**?**) and tend to converge to a degenerate local optimum, which is also known as posterior collapse (**?**). Unlike **VAE!**s, **NF!**s take a different approach by using a series of invertible transformations to turn a simple probability distribution into a complex one that matches the data. The requirement of invertibility restricts their choices of neural network architectures, which limits their direct applicability to time series modeling. On the other hand, **NF!**s provide exact and expressive density estimation capabilities. This means they can precisely calculate the mathematical probability of a specific outcome, making them exceptionally strong at quantifying uncertainty and enhancing the accuracy of probabilistic forecasts. Diffusion models have recently gained popularity by learning to generate high-quality and coherent data through a process of gradually reversing noise. They relax architectural constraints and show strong predictive generation capabilities. Nevertheless, their iterative data generation procedure results in substantial computational overhead. Balancing expressiveness, computational efficiency, and sample quality, ***normalizing flows*** are chosen in this work as the generative backbone of probabilistic system identification.

While parameter identification focuses on determining the time-invariant system parameters, ***state estimation*** is the process of inferring hidden, time-varying state variables from a history of noisy and potentially partial observations and known control inputs. Because of the noise in the sensors (measurement noise) and unmodeled disturbances in the system itself (process noise), state estimation is naturally probabilistic. Thus, estimating the target states means computing their posterior distribution. The traditional method for state estimation is the Luenberger State Observer, which assumes the system is deterministic and both the system dynamics and measurement models are linear and time-invariant. Taking stochasticity into account, the Kalman filter is the most widely applied method. Beyond system linearity, a Kalman filter assumes the process and measurement noises are additive zero-mean Gaussian noise. To handle nonlinear systems, the Extended Kalman filter linearizes the nonlinear functions around the state estimate. However, the linearization introduces errors, leading to incorrect estimates or even divergence. An improved

modification is the Unscented Kalman filter, which carefully selects the means and variances and propagates them through the nonlinear functions. However, it is still only compatible with Gaussian noise.

By relaxing the requirement for rigid physical modeling and avoiding restrictive assumptions about uncertainty, while maintaining high expressiveness, modeling flexibility, and scalability to large datasets, the selected **NF!** architecture establishes a foundation for both probabilistic system identification and state estimation. From a data-centric perspective, these tasks represent different mapping challenges: parameter identification maps time series to a set of static data points; state estimation involves inferring data points or reconstructing temporal trajectories from noisy observations. To address these challenges, it is necessary to extend the standard normalizing flows, given the limited research currently available in this area. Bridging this gap not only broadens the applicability of **NF!**s but also contributes to advancing Bayesian inference techniques for complex, time-varying systems.

Having explored system identification and state estimation through Bayesian inference, we can advance toward ***optimal control***. By leveraging data as the primary source of insight into system behavior, we enable control strategies that are more adaptive, scalable, and robust. This is especially advantageous in situations where analytical models are incomplete, uncertain, or impractical to derive. One direct application is model-free control, which obtains control inputs directly from observations and reference trajectories without requiring an explicit system model. This architecture is especially effective for real-time path planning and tracking. By bypassing complex analytical derivations, it allows for the rapid evaluation of numerous candidate trajectories, enabling the system to select the best path under time-sensitive constraints. In recent literature, diffusion models have attracted widespread attention for representing control policies due to their ability to generate high-quality trajectories under complex observations. However, while diffusion models excel in sample quality, normalizing flows provide more efficient sampling and a more rigorous probabilistic formulation. Despite these advantages, the use of **NF!**s as a generative backbone for control remains largely underexplored. Building on model-free principles, **RL!** (**RL!**) extends this paradigm by optimizing these stochastic policies through continuous interaction with the environment. **RL!** agents learn to maximize long-term rewards while adapting to system uncertainties, effectively enhancing the capabilities of traditional **MPC!** (**MPC!**). Integrating **RL!** with **NF!**s offers a powerful framework for achieving uncertainty-aware, robust, and self-improving control in dynamic environments.

In summary, this work is motivated by four key considerations:

- Bayesian inference provides a principled framework for modeling uncertainty and extracting knowledge under uncertainty, enabling system identification and state estimation based solely on observations. This is particularly valuable in scenarios

where first-principles models are unavailable, incomplete, or prohibitively complex.

- **DGM!**s enable amortized Bayesian inference by learning reusable posterior estimators from many data samples, making them well suited to large-scale parameter identification tasks. The differences among **DGM!**s in these tasks have not been fully assessed, particularly in terms of posterior accuracy, uncertainty calibration, identifiability, and expressiveness for multimodality.

- While **NF!**s (**NF!**s) offer the expressiveness, exact density estimation, and scalability required for complex dynamical systems, their application to system identification and state estimation remains significantly underexplored. Moreover, current **NF!** architectures are often not natively designed for temporal dependencies; therefore, specific adaptations to handle sequential data and state trajectories for these tasks need to be explored.

- Integrating **NF!**s (**NF!**s) in control and reinforcement learning frameworks enables adaptive, uncertainty-aware decision-making in complex, unpredictable environments. This approach has not been fully investigated and offers significant potential for advancing control strategies.

I.2 Objectives and Contributions

The central aim of this dissertation is to leverage the expressiveness, computational efficiency, and mathematical rigor of normalizing flows to develop a unified, data-driven framework for probabilistic system identification, state estimation, and optimal control. The primary objectives of this dissertation are as follows:

- To investigate the efficacy and limitations of normalizing flows in identifying parameters from system observations.

- To adapt and extend normalizing flow architectures for sequential data processing, enabling the estimation of hidden, time-varying states in stochastic dynamical systems.

- To integrate and evaluate normalizing flows into control frameworks, exploring their potential to provide uncertainty-aware decision-making within optimal control and reinforcement learning frameworks.

Building upon these objectives, the key contributions of this dissertation are threefold:

- Parameter Identification

  - Unification of a Deep-Generative-Model-Based Bayesian Inference Framework: A comprehensive framework that leverages deep generative models for parameter identification, comparing their performance and characteristics across different scenarios.
  - Investigating the Relationship Between Sensitivity and Identifiability: Integration of sensitivity analysis with normalizing flows to enable reliable parameter identification in systems where parameters exhibit varying levels of identifiability.
  - Combination of MCMC with Normalizing Flows: The integration of Markov Chain Monte Carlo methods with normalizing flows to achieve robust parameter identification under observational and system uncertainty.
  - Real-World Application of Normalizing Flows for Parameter Identification: The application of normalizing flows to real-world systems, demonstrating their practical utility and effectiveness in parameter identification tasks.

- State Estimation: Development of novel state estimation methods, including amortized sequence flow, latent state-space model flow, and conditional stochastic interpolant, to capture complex, non-Gaussian uncertain state trajectories from observations in stochastic dynamical systems.

- Optimal Control: Integration of flow-based approaches into stochastic control frameworks to enable uncertainty-aware decision-making in complex, unpredictable environments.

I.3 Outline

The remainder of this dissertation is organized as follows:

- Chapter **??** (Background and Preliminaries) establishes the foundations required for this work. It introduces the principles of Bayesian inference, the mechanics of deep generative models, particularly normalizing flows, and the connection from Bayesian inference to control theory.

- Chapter **??** (Literature Review) provides a comprehensive survey of the current state of the art in applying deep generative models in parameter identification, state estimation, and optimal control or reinforcement learning. Through this review, the specific research gaps addressed by this dissertation are identified.

- Chapter **??** (Methodology and Numerical Validation) details the core technical frameworks developed in this work. These methodologies are categorized into three

primary research pillars: parameter identification, state estimation, and optimal control. Each method is accompanied by numerical experiments and synthetic benchmarks to provide theoretical proof of concept and demonstrate the efficacy of the proposed algorithms.

- Chapter **??** (Experiments and Results) presents a series of benchmarks and real-world case studies designed to evaluate the proposed frameworks. This chapter validates the performance of the developed methods in terms of accuracy, computational efficiency, and robustness to noise, following the same three-pillar structure as the methodology.
- Chapter **??** (Conclusion and Future Work) synthesizes the key findings of this research, reflects on the contributions made to the fields of system identification, state estimation, and optimal control, and outlines potential directions for future research.

Chapter II

Background and Preliminaries

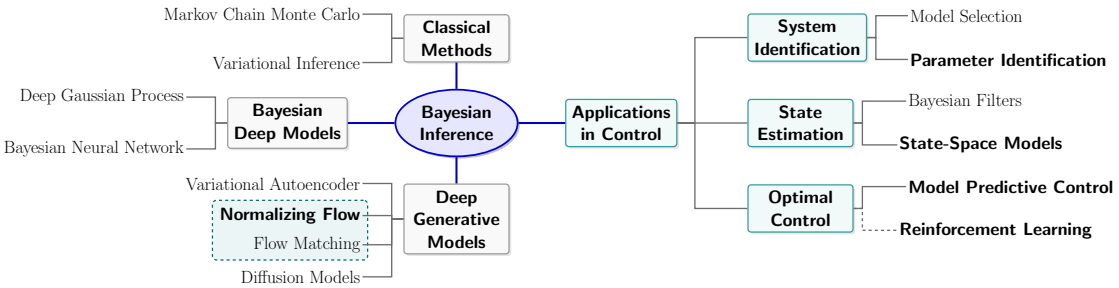


Figure II.1: Conceptual map illustrating the integration of Bayesian inference with deep generative modeling and its applications in control engineering. The left side categorizes the modeling paradigms employed, moving from classical statistical methods to modern deep generative architectures. Notably, Normalizing Flows and Flow Matching are grouped together due to their shared theoretical foundation in continuous-time differential transformations and exact density estimation. On the right, the framework is mapped to core control tasks: system identification, state estimation, and optimal control. A dashed line connects Reinforcement Learning and Optimal Control, signifying the modern methodological convergence of these two disciplines where many data-driven learning methods can be viewed as an adaptive realization of classical optimal control theory.

This section introduces the necessary foundational concepts and methodologies for understanding the rest of this work. Before the main content, we explain the unified notation used throughout this work. We first review classical Bayesian inference methods. Then we introduce deep probabilistic methods, including **DGP!** (**DGP!**), **BNN!** (**BNN!**), and **VAE!** (**VAE!**). Next, we present the core methods of this work, **NF!**s (**NF!**s), a powerful class of generative models for flexible density estimation. Finally, we bridge Bayesian inference to control methods, covering system identification, state estimation, and model predictive control. The overview for this chapter is shown in Figure **??**.

II.1 Notations

To maintain clarity throughout this document, we adopt the following notation:

Category	Description
Basic Symbols	Scalars: x; Vectors: $\boldsymbol{x}$; Matrices: $\boldsymbol{X}$; Sets: $\mathcal{D}$
Random Variables	Multidimensional random variables: $\boldsymbol{x} = [x_1, \ldots, x_D]^T$; Sample index: $\boldsymbol{x}^{(i)}$; Stochastic processes: $X(t)$
Time Series	Time series of length T: $\boldsymbol{x}_{(1:T)}$; Time-dependent variables: $\boldsymbol{x}(t)$ or $x(t)$; Dimension index: $\boldsymbol{x}_i$
Variable Modifications	Estimated: $\hat{\boldsymbol{x}}$; Transformed: $\tilde{\boldsymbol{x}}$; Time derivative: $\dot{\boldsymbol{x}}$
Probability and Statistics	PDF: $p(\cdot)$; Joint PDF: $p(\cdot, \cdot)$; Conditional PDF: $p(\cdot\|\cdot)$; Expectation: $\mathbb{E}_{p(x)}[f(x)]$; Covariance: $\mathrm{Cov}[\boldsymbol{x}, \boldsymbol{y}]$; Variance: $\mathrm{Var}[\boldsymbol{x}]$
Matrix Operations	Identity: $\boldsymbol{I}$; Jacobian: $\boldsymbol{J}$; Determinant: $\det \lvert\boldsymbol{J}\rvert$; Trace: $\mathrm{Tr}(\boldsymbol{X})$; Transpose: $\boldsymbol{X}^\top$

Table II.1: Notation Summary

II.2 Bayesian Inference

Bayesian inference gets its name from Bayes' rule, which is used to perform statistical inference over probability distributions. This chapter draws on *Bayesian Data Analysis* (**?**), *Handbook of MCMC* (**?**), and *Variational Inference* (**?**).

II.2.1 Bayes' Rule

Bayes' rule is derived from the chain rule of conditional probability. For target variables $\boldsymbol{\theta}$ and the corresponding observed data $\mathcal{D}$, the chain rule states that:

$$p(\boldsymbol{\theta}, \mathcal{D}) = p(\boldsymbol{\theta}|\mathcal{D})p(\mathcal{D}) = p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{II.1}$$

Rearranging the terms gives Bayes' rule or Bayes' theorem:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}. \tag{II.2}$$

In Bayes' theorem, $p(\boldsymbol{\theta})$ is the initial distribution of the variables before any data samples are considered, which is called the *prior distribution*. $p(\boldsymbol{\theta}|\mathcal{D})$ is the posterior distribution of the target variables after being updated by the observed data. $p(\mathcal{D}|\boldsymbol{\theta})$ is the likelihood of expected data given the setting of parameters $\boldsymbol{\theta}$. $p(\mathcal{D})$ is called the *marginal likelihood* or *evidence* of expected data across all possible variable settings:

$$p(\mathcal{D}) = \Sigma_{\boldsymbol{\theta}'} p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}', \tag{II.3a}$$

$$p(\mathcal{D}) = \int p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')d\boldsymbol{\theta}'. \tag{II.3b}$$

In practice, it is often treated as a normalizing constant and ignored. The unnormalized posterior is then given by:

$$p_u(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{II.4}$$

Normally, the posterior distribution does not have a closed-form expression. Instead, we approximate it using numerical methods. Computational algorithms that rely on repeated random sampling to obtain numerical results are referred to as "Monte Carlo" methods. Monte Carlo methods provide ways to approximate the posterior distribution by expectations:

$$E = \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[f(\boldsymbol{\theta})] = \int f(\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}, \tag{II.5}$$

where $f(\boldsymbol{\theta})$ is any function of interest, for example, $f(\boldsymbol{\theta}) = \boldsymbol{\theta}$. In the simplest case, we can approximate it using standard numerical techniques such as a Riemann sum over a discrete grid of points:

$$E \approx \Sigma_{\boldsymbol{\theta}'} f(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}'. \tag{II.6}$$

However, this approach is only feasible for low-dimensional $\boldsymbol{\theta}$ due to the curse of dimensionality. Moreover, for complex distributions, the discretization is highly inefficient. Therefore, for high-dimensional parameters and complex distributions, we often resort to sampling-based or optimization-based methods to approximate the posterior distribution, which will be discussed in the following sections.

II.2.2 Markov Chain Monte Carlo

The "Markov Chain" aspect indicates that these algorithms generate a sequence of samples in which each sample depends only on the previous one. In contrast to the basic Monte Carlo methods, **MCMC!** (**MCMC!**) methods produce auto-correlated samples, so that each new sample adds less new information than an independent draw would. Different sequence-update algorithms lead to different **MCMC!** methods. The goal of **MCMC!** is to construct a Markov chain that has the desired posterior distribution as its *equilibrium*

distribution. After running the chain for a sufficient number of iterations, the samples generated from the chain can be treated as samples from the target posterior distribution.

This concept is analogous to the steady-state behavior of a dynamical system. Typically, a dynamical system undergoes a transient phase before settling into an equilibrium where the states remain constant. In **MCMC!**, while the variables continue to move, their probability distribution reaches a steady state. If we interpret the probability density as the energy landscape of a physical system, equilibrium is reached when the system has settled into a stable statistical configuration. At this point, although individual samples continue to transition between different regions of the parameter space, the net movement between these regions is zero. This balance ensures that the density of samples remains constant over time, allowing the chain to accurately represent the invariant target posterior distribution.

II.2.2.1 Metropolis-Hastings Algorithm

The **MH!** (**MH!**) algorithm, which is named after Metropolis (**?**) and Hastings (**?**), is a widely used **MCMC!** method. To understand its logic, consider the "Autonomous Rover Mapping" analogy: Imagine an autonomous rover equipped with a noisy sensor is deployed into a dark and unmapped crater. Its task is to create a high-resolution density map of water-ice concentrations across the crater floor. It doesn't have a satellite map; it only knows the signal strength of ice at its current coordinates using a neutron spectrometer. It cannot teleport and thus has to move from point to point in order to take readings. The density map is essentially a confidence map of where the ice is located ($\boldsymbol{\theta}$) given the sensor observations ($\mathcal{D}$). An alignment between the MH algorithm and its analogy is shown in Table **??**.

Although this procedure allows flexible approximation of complex posterior distributions, it suffers from a slow convergence rate because of its random exploration strategy. Thus, gradient-based **MCMC!** methods are proposed to incorporate the structure of the target density, leading to faster sampling.

II.2.2.2 Metropolis-Adjusted Langevin Algorithm

The **MALA!** (**MALA!**) (**?**) combines the **MH!** algorithm with Langevin diffusion, whose stationary distribution is the target density. In this framework, the rover no longer jumps blindly; instead, it uses its onboard sensors to detect the local slope of the resource distribution. Its movement is then governed by a first-order control law consisting of two parts: a deterministic drift that pushes the rover uphill toward higher density, and a stochastic diffusion (Gaussian noise) that ensures the rover continues to explore rather than getting stuck at a single peak.

MH Algorithm (Random Walk)	**Autonomous Rover Mapping**
1. Initialization Set starting point $\boldsymbol{\theta}_{(0)}$. Define proposal $q(\boldsymbol{\theta}'\|\boldsymbol{\theta})$, which is usually a Gaussian $\mathcal{N}(\boldsymbol{\theta}, \sigma^2)$.	**1. Power-On & Config** The rover starts at GPS coordinate $\boldsymbol{\theta}_{(0)}$. Set the Jogging Logic (q): a random-walk jump distance based on motor constraints.
2. Iteration Loop (for each i):	**2. Navigation Duty Cycle**:
(a) Propose a new state: Generate a candidate $\boldsymbol{\theta}'_{(i)}$ using the proposal distribution $q(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)})$.	*(a) Candidate Path Planning:* A phantom coordinate $\boldsymbol{\theta}'_{(i)}$ nearby is calculated. This is a potential move, but the actuators are not yet engaged.
(b) Calculate Transition Probability $T(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)})$*:* $T(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)}) = \min\left(1, \frac{p(\boldsymbol{\theta}'_{(i)} \mid \mathcal{D})\, q(\boldsymbol{\theta}_{(i-1)} \mid \boldsymbol{\theta}'_{(i)})}{p(\boldsymbol{\theta}_{(i-1)} \mid \mathcal{D})\, q(\boldsymbol{\theta}'_{(i)} \mid \boldsymbol{\theta}_{(i-1)})}\right)$ (II.7)	*(b) Sensor Evaluation based on Data* $\mathcal{D}$*:* The rover asks: "Based on our sensor history ($\mathcal{D}$), is the ice likely denser at the phantom spot $\boldsymbol{\theta}'$ than my current spot $\boldsymbol{\theta}$?" It also checks for mechanical bias in q.
(c) Update via Accept/Reject: $\boldsymbol{\theta}_{(i)} = \begin{cases} \boldsymbol{\theta}'_{(i)}, & \text{if } u < T(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)}) \\ \boldsymbol{\theta}_{(i-1)}, & \text{otherwise} \end{cases}$ (II.8)	*(c) Actuator Actuation:* If the spot is "better," the motors drive there. If "worse," it rolls a die: it might still move there to explore, or stay put and record the current spot again.
3. Accumulation Run the chain for more than N_{burn} iterations. The first N_{burn} samples are treated as burn-in, after which the chain is assumed to have reached its stationary distribution. The retained samples are $\{\boldsymbol{\theta}_{(N_{\text{burn}}+1)}, \ldots, \boldsymbol{\theta}_{(N_{\text{burn}}+B)}\}$.	**3. Mission Duration** The rover first drives long enough to leave its arbitrary starting region and reach a representative exploration pattern. Only after this warm-up period does it write the subsequent (x, y) coordinates into the mission logbook.
4. Estimator $p(\boldsymbol{\theta}\|\mathcal{D}) \approx \frac{1}{B}\sum_{b=1}^{B} \delta_{(N_{\text{burn}}+b)}(\boldsymbol{\theta})$ $\delta_{(N_{\text{burn}}+b)}(\boldsymbol{\theta})$ denotes a Dirac delta centered at the retained sample $\boldsymbol{\theta}_{(N_{\text{burn}}+b)}$.	**4. Resource Density Map** Download the log and plot the coordinates. The resulting point cloud density shows the ice distribution $p(\boldsymbol{\theta}\|\mathcal{D})$ across the crater.

Table II.2: Alignment between the Metropolis-Hastings (MH) algorithm and an autonomous rover mapping.

In general, for a stochastic process $\Theta(t)$ for variables $\boldsymbol{\theta}$, the Langevin diffusion is described by the following **SDE!** (**SDE!**):

$$d\Theta(t) = \nabla \log p(\Theta(t))dt + \sqrt{2}dW(t), \tag{II.9}$$

where $W(t)$ denotes a standard Wiener process (**?**). The Wiener process is a continuous-time stochastic process with independent, normally distributed increments, which can be interpreted as Brownian motion. It has the following properties:

(1) $W(0) = \mathbf{0}$;

(2) for $t > s \geq 0$, $W(t) - W(s) \sim \sqrt{t-s}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I})$;

(3) for $t_1 > s_1 \geq 0$, and $t_2 > s_2 \geq 0$, $W(t_1) - W(s_1)$ and $W(t_2) - W(s_2)$ are independent.

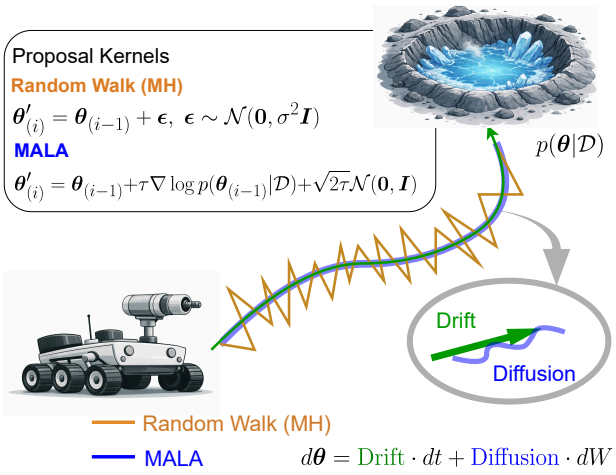


Figure II.2: An illustration of **MH!** algorithm and **MALA!** using the autonomous rover as a conceptual sampler. The crater representing $p(\boldsymbol{\theta}|\boldsymbol{\mathcal{D}})$ serves as the objective landscape where the sampler aims to concentrate its states. The Random Walk (MH) is shown as the jagged orange path, while the **MALA!** is represented by a smoother green path (drift) along with a blue band (diffusion).

A comparison between the **MH!** algorithm and **MALA!** is illustrated in Figure **??**. The **MALA!** uses the first-order Euler-Maruyama discretization of the **SDE!**, resulting in the following proposal distribution:

$$\Theta(t) = \Theta(i-1) + \tau \nabla \log p(\Theta(i-1)) + \sqrt{2\tau}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{II.10}$$

where τ is a step size parameter. Because of the discretization, convergence to the target density is no longer guaranteed. Therefore, the **MH!** algorithm is applied to correct the bias introduced by the discretization. For each iteration i, the **MALA!** updates the state as follows (corresponding to the iteration part in Table **??**):

(a) **Guidance with Gradient:** Unlike the uninformed **MH!**, which only considers the current position, **MALA!** first computes the gradient of the log-posterior: $\nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})$. This determines the drift direction toward regions of higher probability.

(b) **Informed Proposal:** Propose a new state $\boldsymbol{\theta}'_{(i)}$ using the Langevin dynamics (**??**): $\boldsymbol{\theta}'_{(i)} = \boldsymbol{\theta}_{(i-1)} + \tau \nabla \log p(\boldsymbol{\theta}_{(i-1)}) + \sqrt{2\tau}\, \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

(c) **Transition Probability:** Calculate the transition probability according to Equation (**??**). Specifically, the proposal distribution is derived from the Langevin dynamics:

$$q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) \propto \exp(-\frac{1}{4\tau} \left\| \boldsymbol{\theta}'_{(i)} - \boldsymbol{\theta}_{(i-1)} - \tau \nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D}) \right\|_2^2). \tag{II.11}$$

(d) **Correction (Accept/Reject):** Using the same mechanism as MH, a uniform random number $u \sim U(0,1)$ is generated. Then, the state $\boldsymbol{\theta}_{(i)}$ is updated by the accepting ratio according to Equation (**??**).

Compared to the **MH!**, **MALA!** converges faster and scales better in higher dimensions, at the cost of requiring gradients and tuning. As a more advanced **MCMC!** method, **HMC!** (**HMC!**) (**?**) further improves sampling efficiency by simulating the whole sampling trajectory as Hamiltonian dynamics.

II.2.3 Variational Inference

Unlike **MCMC!**, **VI!** (**VI!**) (**??**) is an optimization-based method that approximates the target posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$ with a simpler, parameterized distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$, where $\boldsymbol{\phi}$ denotes the variational factors. The goal of **VI!** is to find the optimal variational factors $\boldsymbol{\phi}^*$ that minimize the distance between the approximate distribution and the true posterior. The distance is normally measured by the **KL!** (**KL!**)-divergence (**?**), which is defined as:

$$D_{\mathrm{KL}}(q(z)||p(z)) = \mathbb{E}_{q(z)}[\log \frac{q(z)}{p(z)}] = \int q(z) \log \frac{q(z)}{p(z)} dz. \tag{II.12}$$

Thus, the optimization problem of **VI!** can be formulated as:

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})). \tag{II.13}$$

Substituting Bayes' rule (**??**) into **KL!**-divergence, we have:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) \,||\, \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}). \tag{II.14}$$

The marginal likelihood $p(\mathcal{D})$ is usually intractable, as it involves integrating over all possible parameter settings. Therefore, the divergence cannot be computed directly.

Expanding the above equation with the definition of **KL!**-divergence, we have:

$$\begin{aligned} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})}[\log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \log p(\mathcal{D}|\boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) + \log p(\mathcal{D})] \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}). \end{aligned} \tag{II.15}$$

Since $p(\mathcal{D})$ is independent of the variational factors $\boldsymbol{\phi}$, the last term is reduced to $\log p(\mathcal{D})$. Combining the terms $p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) = p(\boldsymbol{\theta}, \mathcal{D})$ gives

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) + \log p(\mathcal{D}) \tag{II.16}$$

Or combining the terms $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$ and $p(\boldsymbol{\theta})$ via **KL!**-divergence gives:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\mathcal{D}) \tag{II.17}$$

Rearranging the terms in Equation (**??**) gives:

$$\log p(\mathcal{D}) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{II.18}$$

Since the **KL!**-divergence is always non-negative, we have:

$$\log p(\mathcal{D}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{II.19}$$

Because of this inequality, the right-hand side is called the **ELBO!** (**ELBO!**), also known as the variational lower bound. Another way to formulate the **ELBO!** using Equation (**??**) is:

$$\mathrm{ELBO} = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})). \tag{II.20}$$

Then, minimizing the **KL!**-divergence is equivalent to maximizing the **ELBO!** (**??**), leading to the equivalent optimization problem:

$$\boldsymbol{\phi}^* = \arg\max_{\boldsymbol{\phi}} \mathrm{ELBO} = \arg\min_{\boldsymbol{\phi}} [\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D})]. \tag{II.21}$$

To complete the optimization, we need to choose a suitable family of variational distributions $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$. A common choice is the mean-field family (**?**), which assumes that the variables $\boldsymbol{\theta}$ are mutually independent and each governed by a distinct factor under q:

$$q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \Pi_i q_{\phi_i}(\theta_i), \tag{II.22}$$

where each factor ϕ_i governs a variable θ_i. Then, the **ELBO!** (**??**) is iteratively optimized by updating each factor while holding the others fixed. The solution of the optimal distribution $q_{\phi_i}(\theta_i)$ is proportional to the exponential expected log of the complete condition:

$$\log q^*_{\phi_i}(\theta_i) \propto \mathbb{E}_{\Pi_{j\neq i} q_{\phi_i}(\theta_i)}[\log p(\theta_i|\boldsymbol{\theta}_{j\neq i}, \mathcal{D})], \tag{II.23}$$

where the expectation is taken with respect to the distributions of all other groups $j \neq i$. This algorithm is called **CAVI!** (**CAVI!**) (**?**).

The assumption of the mean-field **VI!** simplifies the optimization problem, and thus makes it scalable for high-dimensional variables and also enables parallelizable computation. However, it may also limit the expressiveness of the variational distribution, especially when the posterior correlations are crucial, leading to suboptimal approximations of the true posterior. In summary, **VI!** is generally faster than **MCMC!** and scales better to large datasets, but it may provide less accurate approximations of the posterior distribution (**?**).

II.3 Deep Probabilistic Methods

Although classical methods such as **MCMC!** and **VI!** provide general solutions for Bayesian inference, they are limited by their "approximation-per-point" behavior. Deep probabilistic methods, in contrast, enable amortized Bayesian inference and unlock both high expressiveness and scalability to large datasets. In the following section, we briefly review three prominent deep probabilistic methods: the Deep Gaussian Process, the Bayesian Neural Network, and the Variational Autoencoder. The former two methods are direct extensions of classical Bayesian inference with deep architectures, while the last one is a fundamental deep generative model that leverages variational inference.

II.3.1 Deep Gaussian Process

A Gaussian process can be interpreted as an infinite-dimensional Gaussian distribution. It is fully specified by a mean function $m(\cdot)$ and a covariance (kernel) function $k(\cdot,\cdot)$. For simplicity, we consider a single-variable case where the variable is x and the corresponding Gaussian process is $X(t)$. We collect M observations at discrete input points $\boldsymbol{x} = [x_1, \ldots, x_M]^\top$, the associated random variables follow a multivariate normal distribution:

$$\begin{bmatrix} X(t_1) \\ \vdots \\ X(t_M) \end{bmatrix} \sim \mathcal{N}\left(\boldsymbol{\mu} = \begin{bmatrix} \mu(t_1) \\ \vdots \\ \mu(t_M) \end{bmatrix}, \boldsymbol{K} = \begin{bmatrix} k(t_1,t_1) & \ldots & k(t_1,t_M) \\ \vdots & \ddots & \vdots \\ k(t_M,t_1) & \ldots & k(t_M,t_M) \end{bmatrix}\right). \tag{II.24}$$

Note that the time here is not really temporal but indicates the coordinates of variables in the parameter domain. For example, the t_i-th observation in the dataset is x_i. Thus, in the following, we call t_i an index, instead of a time point. Using a Gaussian process to infer variables from observed data based on prior distributions leads to **GPR!** (**GPR!**) (**?**). In comparison to **MCMC!**, **GPR!** aims to directly approximate the target distribution function with a Gaussian-based surrogate model instead of sampling them iteratively.

II.3.1.1 Gaussian Process Regression

We assume that we observe noisy measurements $\boldsymbol{y} = [y_1, \ldots, y_M]^\top$ at inputs $\boldsymbol{x}$. The observation model relates the latent function $f(\cdot)$ to the observations via additive Gaussian noise:

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2). \tag{II.25}$$

The likelihood $p(y_i|f(x_i))$ is thus given by:

$$p(y_i|f(x_i)) = \mathcal{N}(y_i|f(x_i), \sigma_\epsilon^2). \tag{II.26}$$

The prior distribution over functions $f(\cdot)$ is assumed to be a zero-mean Gaussian process:

$$f(\cdot) \sim \mathcal{GP}(0, k(\cdot,\cdot)). \tag{II.27}$$

The kernel function $k(\cdot,\cdot)$ is usually chosen as the **SE!** (**SE!**) kernel, also known as the **RBF!** (**RBF!**), defined as:

$$k(x_i, x_j) = \sigma_{SE}^2 \exp(-\frac{1}{2\ell^2} \|x_i - x_j\|^2), \tag{II.28}$$

where σ_{SE} and ℓ are vertical and horizontal scale parameters, which control the vertical span and smoothness of the function, respectively.

To obtain the optimal regression performance, the hyperparameters $\varphi = \{\sigma_{SE}, \ell, \sigma_\epsilon\}$ must be optimized. In **GPR!**, this is achieved by maximizing the log marginal likelihood. By integrating out the latent function values f, the marginal distribution of the data $\boldsymbol{y}$ depends only on the inputs $\boldsymbol{x}$ and hyperparameters φ:

$$p(\boldsymbol{y}|\boldsymbol{x}, \varphi) = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \boldsymbol{K} + \sigma_\epsilon^2 \boldsymbol{I}). \tag{II.29}$$

The optimization objective $\mathcal{J}(\varphi)$ is the logarithm of this marginal likelihood:

$$\mathcal{J}(\varphi) = \log p(\boldsymbol{y}|\boldsymbol{x}, \varphi) = \underbrace{-\frac{1}{2}\boldsymbol{y}^\top (\boldsymbol{K} + \sigma_\epsilon^2 \boldsymbol{I})^{-1} \boldsymbol{y}}_{\text{Data Fit}} \underbrace{-\frac{1}{2}\log |\boldsymbol{K} + \sigma_\epsilon^2 \boldsymbol{I}|}_{\text{Complexity Penalty}} - \frac{M}{2}\log 2\pi. \tag{II.30}$$

The optimal hyperparameters $\hat{\varphi}$ are found by maximizing this function, typically using gradient-based optimizers.

The **GPR!** is widely used for regression tasks owing to its non-parametric nature and its capacity to quantify uncertainty. However, its computational complexity scales cubically with the number of training points, i.e., $\mathcal{O}(D^3)$ for a D-dimensional input. Additionally, calculating Gaussian processes requires storing the entire training dataset, incurring a memory cost of $\mathcal{O}(M^2)$, which grows quadratically with the dataset size M. These limitations hinder its scalability to high-dimensional inputs and large datasets. In addition, the expressiveness of **GPR!** is constrained by the choice of kernel function and its incapacity to perform hierarchical feature extraction.

II.3.1.2 Parameter Identification using GPR

Feature	**Direct Inverse Regression (?)**	**Surrogate-Based Inference (Forward) (?)**
Core Concept	The **GPR!** learns the inverse mapping directly from observation space to parameter space, acting as amortized Bayesian inference.	The **GPR!** acts as a surrogate for the physical simulator to approximate the likelihood function, accelerating the simulation process. The posterior distribution of the parameters is obtained by **MCMC!**.
GP Input ($\boldsymbol{x}$)	The observed system data $\mathcal{D}$ (or extracted feature vector).	The system parameter $\boldsymbol{\theta}$.
GP Output ($\boldsymbol{y}$)	The estimated system parameter $\boldsymbol{\theta}$.	The simulated system response $\hat{\boldsymbol{y}} = \text{Sim}(\boldsymbol{\theta})$.
Posterior $p(\boldsymbol{\theta}\|\mathcal{D})$	Obtained immediately from the GP's predictive distribution.	Obtained via Bayes' rule using the GP as the likelihood term: $p(\boldsymbol{\theta}\|\mathcal{D}) \propto p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta})$.
Pros & Cons	**Pros:** Computationally fast at inference time. **Cons:** Ill-posed if the mapping is non-unique (multiple $\boldsymbol{\theta}$ produce same $\mathcal{D}$).	**Pros:** Scientifically rigorous; correctly handles multi-modal distributions and uncertainty. **Cons:** Requires iterative sampling (MCMC).

Table II.3: Comparison of Gaussian Process strategies for parameter identification.

In the context of system identification, we aim to estimate the posterior distribution of system parameters given observed system data, denoted as $p(\boldsymbol{\theta}|\mathcal{D})$. There are two primary strategies to achieve this, which are compared in Table **??**.

II.3.1.3 Deep Gaussian Process

To improve expressiveness, **DGP!** (**DGP!**) (**?**) is proposed as a multilayer hierarchical generalization of Gaussian processes. The input $\boldsymbol{x}$ is passed through multiple layers of Gaussian processes to obtain the output $\boldsymbol{y}$. For an L-layer **DGP!**, the model extends the Gaussian process (**??**) by:

$$y_i = f_L(f_{L-1}(\dots f_1(x_i) + \epsilon_{i,1} \dots)) + \epsilon_L, \quad f_l(\cdot) \sim \mathcal{GP}(0, k_l(\cdot,\cdot)), \\ l = 1, \dots, L,\ i = 1, \dots, M. \quad \text{(II.31)}$$

The deep architecture enables Gaussian processes to learn more complex data patterns and hierarchical representations. It can be seen as a deep neural network with infinitely wide layers (**?**), where each layer applies a Gaussian process instead of a nonlinear matrix transformation. However, due to computational cost and storage requirements, the **DGP!** is still cumbersome to apply in practice.

II.3.2 Bayesian Neural Network

Another approach of deep probabilistic methods involves integrating Bayesian inference with deep neural networks. A **BNN!** (**BNN!**) (**??**) places prior distributions over the network's weights and biases, allowing uncertainty to be modeled directly within the architecture.

Unlike **VI!**, where the posterior of the target parameters is directly approximated, the objective of a **BNN!** is to optimize a probabilistic model for estimating the target posterior. Thus, the aim is to infer the posterior distribution over the model parameters $\tilde{\boldsymbol{\theta}} = [\boldsymbol{w}, \boldsymbol{b}]$ given the training data $\mathcal{D}$, i.e., $p(\tilde{\boldsymbol{\theta}}|\mathcal{D})$. **VI!** is commonly employed to approximate the posterior by optimizing the **ELBO!** (**??**). Here, we inherit the notation from **VI!**. The parameters $\tilde{\boldsymbol{\theta}}$ are not the system parameters to be identified.

To facilitate gradient-based optimization via backpropagation—similar to standard neural networks—the reparameterization trick (**?**) is used. This technique expresses random variables as deterministic variables of noise, enabling efficient and differentiable sampling during training. For example, for a Gaussian variational distribution $q_{\boldsymbol{\phi}}(\tilde{\boldsymbol{\theta}}) = \mathcal{N}(\tilde{\boldsymbol{\theta}}|\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$, we can reparameterize $\tilde{\boldsymbol{\theta}}$ as:

$$\tilde{\boldsymbol{\theta}} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \quad \text{(II.32)}$$

where $\odot$ represents the Hadamard product, i.e., the element-wise product, and the variational parameters here are $\boldsymbol{\phi} = [\boldsymbol{\mu}, \boldsymbol{\sigma}]$. To train a **BNN!** on large datasets, **SGD!** (**SGD!**) (**?**) is typically employed to optimize the **ELBO!**. Unlike standard gradient descent, **SGD!**

estimates the gradient at each iteration using a randomly selected subset of the data. Then, the variational objective of a **BNN!** using the **ELBO!** (**??**) becomes:

$$\mathcal{L}(\tilde{\boldsymbol{\theta}}, \boldsymbol{\phi}) = \frac{1}{M} \sum_{i=1}^{M} [D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\tilde{\boldsymbol{\theta}}) || p(\tilde{\boldsymbol{\theta}})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\tilde{\boldsymbol{\theta}})} \log p(\mathcal{D}_i | \tilde{\boldsymbol{\theta}})], \tag{II.33}$$

where each mini-batch $\mathcal{D}_i \subset \mathcal{D}$ is drawn from equally sized subsets of the training data $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_M\}$. During each optimization step, the auxiliary noise $\boldsymbol{\epsilon}$ is sampled multiple times from $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to perform Monte Carlo estimation of the expectation terms in the loss function $\mathcal{L}(\tilde{\boldsymbol{\theta}}, \boldsymbol{\phi})$. The gradient $\nabla_{\boldsymbol{\phi}} \mathcal{L}(\tilde{\boldsymbol{\theta}}, \boldsymbol{\phi})$ is then computed via backpropagation and averaged across all samples. Finally, the variational parameter $\boldsymbol{\phi}$ is updated by

$$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \alpha \nabla_{\boldsymbol{\phi}} \mathcal{L}(\tilde{\boldsymbol{\theta}}, \boldsymbol{\phi}), \tag{II.34}$$

where α is the learning rate. This procedure is called **BbB!** (**BbB!**) (**?**).

In comparison to **DGP!**, **BNN!** demonstrates better scalability to large datasets and improved representational capacity. Moreover, the **BbB!** method facilitates the compatibility of **BNN!**s with deep learning techniques and thus enables flexible implementation in different neural network architectures, such as **CNN!** (**CNN!**) (**?**), **RNN!** (**RNN!**) (**?**), and Transformer (**?**).

However, a fundamental challenge remains. Neither **DGP!**s (typically employed as forward maps) nor **BNN!**s (which can operate in either mapping direction) inherently model the posterior uncertainty of parameters in a direct, closed-form manner. Instead, both act as functional mappings that require an external probabilistic framework to characterize the distribution of interest, such as marginalization over latent functions or variational optimization over weights.

Consequently, several works raise concerns with BNNs, including incorrect uncertainty estimation between separated regions of observations (**??**), poor generalization ability under domain shift (**?**), and a high sensitivity to the choice of prior distributions (**?**). These limitations, arising from the difficulty of capturing complex, multi-modal posterior landscapes using standard approximate inference, motivate the application of more powerful generative models, such as variational autoencoders and **NF!**s, which are discussed in the following sections.

II.3.3 Variational Autoencoder

One way to address the limitations of **BNN!** and directly model the posterior distribution of parameters is to use a **DGM!** (**DGM!**). Generally speaking, **DGM!** is a model that learns a probabilistic mapping from latent variables $\boldsymbol{z}$ to observed data $\boldsymbol{x}$, enabling the

generation of new samples and the inference of latent representations.

VAE! is one of the most widely used **DGM!**s. For a dataset $\mathcal{D} = \{\boldsymbol{x}^{(i)}\}_{i=1}^{M}$, the samples are assumed to be **i.i.d.!** (**i.i.d.!**) and drawn from an unchanging underlying distribution $p(\boldsymbol{x})$. A **VAE!** maps input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$ using a parameterized inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, which is also called an *encoder* or *recognition model*. Then, the **VAE!** generates data $\boldsymbol{x}$ from the latent variables $\boldsymbol{z}$ by another parameterized model $p_{\boldsymbol{\varphi}}(\boldsymbol{x}|\boldsymbol{z})$, which is also called a *decoder*. The decoder is often described as a "generative" model. However, according to the formal definition (**?**), a generative model learns a joint distribution $p_{\boldsymbol{\varphi}}(\boldsymbol{z}, \boldsymbol{x}) = p_{\boldsymbol{\varphi}}(\boldsymbol{z})p_{\boldsymbol{\varphi}}(\boldsymbol{x}|\boldsymbol{z})$, including both sampling $\boldsymbol{z} \sim p_{\boldsymbol{\varphi}}(\boldsymbol{z})$ and sampling $\boldsymbol{x} \sim p_{\boldsymbol{\varphi}}(\boldsymbol{x}|\boldsymbol{z})$.

The latent variables connect the inference model and the generative model. From the encoder side, the inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ approximates the true posterior distribution $p_{\boldsymbol{\varphi}}(\boldsymbol{z}|\boldsymbol{x})$ of the generative model, which is intractable. From the decoder side, the distribution of latent variables is a prior distribution, which is usually chosen to be a standard normal distribution, i.e., $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I})$. The whole architecture is illustrated in Figure **??**.

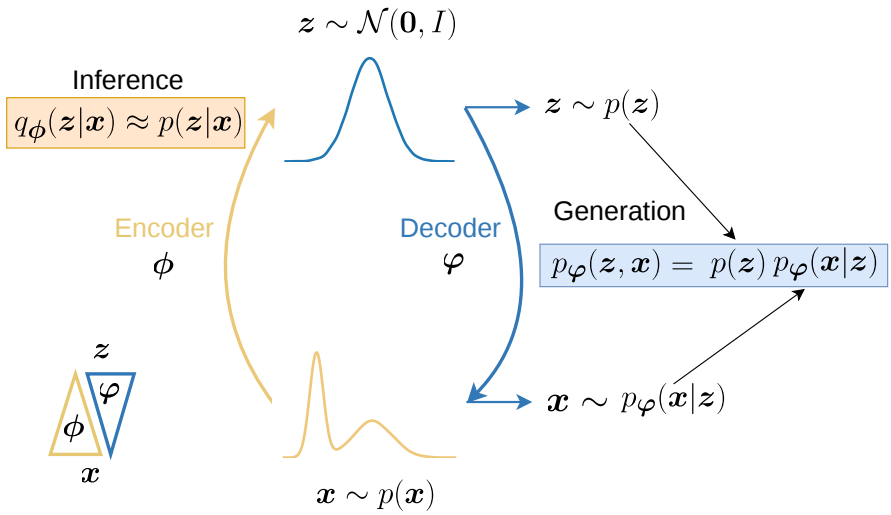


Figure II.3: The architecture of a Variational Autoencoder. The encoder (with neural network parameters $\boldsymbol{\phi}$) maps the input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$, and the decoder (with neural network parameters $\boldsymbol{\varphi}$) reconstructs the input data from the latent variables.

The same reparameterization trick as in the **BNN!** is used to enable gradient-based optimization. The latent variables $\boldsymbol{z}$ become:

$$\boldsymbol{z} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{II.35}$$

where $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x})$ and $\boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x})$ are also neural networks.

Unlike the mini-batch optimization in the **BNN!**, we use a single encoder neural network to perform posterior inference over all samples in the dataset. This strategy of sharing variational parameters across data points is called *amortized variational inference* (**??**), which improves the optimization efficiency.

The objective of the inference model is to minimize the **KL!**-divergence between the inferred and true posterior $D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\varphi}}(\boldsymbol{z}|\boldsymbol{x}))$. Recall the **ELBO!** in **VI!** (**??**, **??**), the **ELBO!** of a **VAE!** is given by:

$$\log p_{\boldsymbol{\varphi}}(\boldsymbol{x}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\varphi}}(\boldsymbol{x}|\boldsymbol{z}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\varphi}}(\boldsymbol{z})). \tag{II.36}$$

Maximizing the **ELBO!** concurrently optimizes both the inference model and the generative model. The marginal likelihood $p_{\boldsymbol{\varphi}}(\boldsymbol{x})$ is maximized, meaning the generated data maximally resembles the input data. Meanwhile, the distance between the approximated and the true posterior is minimized. Changing the prior distribution in the **ELBO!**, $p_{\boldsymbol{\varphi}}(\boldsymbol{z}) := p(\boldsymbol{z})$, the overall loss function of a **VAE!** is given by:

$$\mathcal{L}(\boldsymbol{\varphi}, \boldsymbol{\phi}) = \underbrace{-\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\varphi}}(\boldsymbol{x}|\boldsymbol{z})}_{\text{reconstruction}} + \underbrace{D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z}))}_{\text{regularization}}. \tag{II.37}$$

The first term is a reconstruction loss, which encourages the decoder to reconstruct the input data from the latent variables. The second term is a regularization term, which forces the approximate posterior to be close to the prior distribution.

VAE!s have proven to be versatile and powerful in diverse applications, such as representation learning, anomaly detection, and generative modeling (**?**). To enable **VAE!**s to perform Bayesian inference of variables $\boldsymbol{x}$ given observations $\boldsymbol{y}$, the observations should be inserted into both the encoder and decoder as conditions. This method is known as conditional **VAE!** (**?**). The inference model becomes $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{y})$, the prior distribution becomes $p(\boldsymbol{z}|\boldsymbol{y})$, and the generative model becomes $p_{\boldsymbol{\varphi}}(\boldsymbol{z}, \boldsymbol{x}|\boldsymbol{y})$. The conditions are often constrained to be low-dimensional variables, such as class labels (**?**) or physical parameters (**?**). For high-dimensional and complex conditions, a powerful decoder is required to capture the relationship between the variables and observations. In this case, the latent variables may be ignored and thus the variability is lost. This issue is known as posterior collapse (**??**), which limits the application of **VAE!**s in conditional inference tasks.

II.4 Normalizing Flow and Other Deep Generative Models

Another **DGM!** is the **NF!**s (**NF!**s). **?** (**?**) proposed **NF!** as a method for approximating posterior distributions by transforming a simple base distribution through a sequence of

invertible mappings into complex target distributions.

A variety of architectures have been developed to realize **NF!**s. A GitHub website (**?**) summarizes the key publications and implementations in this domain. As outlined in the comprehensive review by **?**, successful architectures include coupling flows, autoregressive flows, residual flows, and infinitesimal (continuous) flows. Throughout this work, we focus primarily on coupling flows, which offer structural invertibility and have been extensively utilized in **NF!**s. Infinitesimal flows characterize distributional transformations through continuous dynamics, establishing deep theoretical connections with other probabilistic frameworks such as diffusion models. Thus, we also introduce continuous **NF!**s following the description of coupling flows.

Continuous **NF!**s provide a bridge to other generative models, including flow matching and diffusion models, which are also introduced. In particular, flow matching is often referred to as a member of normalizing flows due to its close relationship with the normalizing flow framework. Moreover, stochastic interpolants generalize and unify the concepts of flow matching and diffusion models, providing a comprehensive framework for understanding these generative processes.

II.4.1 Basics

We simplify a **NF!** as an invertible and differentiable function $f : \mathbb{R}^D \to \mathbb{R}^D$ (known as a diffeomorphism) that maps a random variable $\boldsymbol{z}$ in latent space to another random variable $\boldsymbol{x}$ in data space:

$$\boldsymbol{x} = f(\boldsymbol{z}), \tag{II.38a}$$

$$\boldsymbol{z} = f^{-1}(\boldsymbol{x}). \tag{II.38b}$$

The variable $\boldsymbol{x}$ follows an underlying distribution of data $p_X(\boldsymbol{x})$. The variable $\boldsymbol{z}$ is drawn from a simple base distribution $p_Z(\boldsymbol{z})$, such as a standard normal distribution. Understanding the transformation between the two density functions is essential for comprehending **NF!**s.

II.4.1.1 Change of Variables

According to the change of variables formula (**?**), when both the latent and data spaces are smooth and the function f is differentiable, we have:

$$\int \Psi(\boldsymbol{x}) d\boldsymbol{x} = \int \Psi(\boldsymbol{x} = f(\boldsymbol{z})) \left|\det \boldsymbol{J}_f(\boldsymbol{z})\right| d\boldsymbol{z}, \tag{II.39}$$

where Ψ is a continuous function, $\boldsymbol{J}_f(\boldsymbol{z})$ is the Jacobian of the mapping f and $\det|\cdot|$ stands for determinant:

$$|\det \boldsymbol{J}_f(\boldsymbol{z})| = \left|\det \frac{\partial f(\boldsymbol{z})}{\partial \boldsymbol{z}}\right|. \tag{II.40}$$

Equation (**??**) implies probability conservation under transformation.

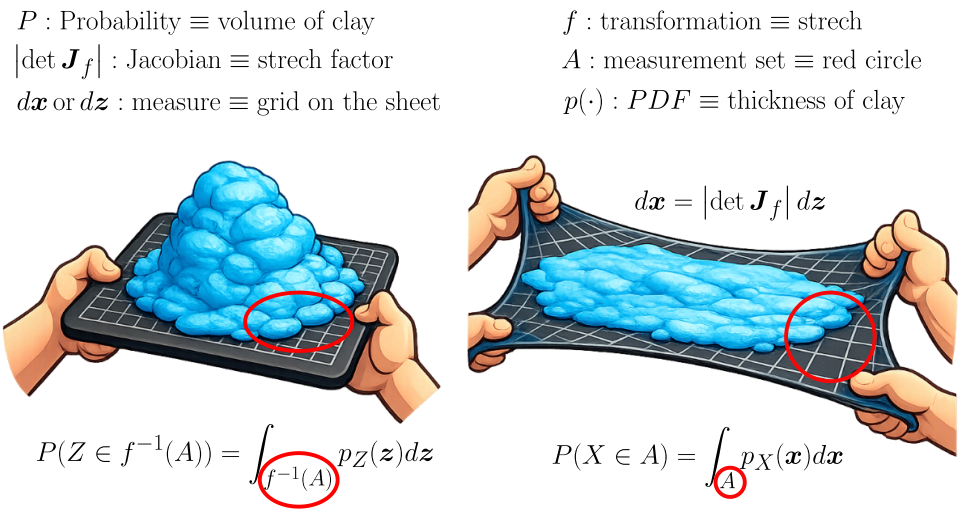


Figure II.4: The Clay Volume Analogy for Change of Variables. Probability mass is represented as clay on a rubber sheet. Transforming the variable ($\boldsymbol{z} \to \boldsymbol{x}$) stretches the sheet. While the total amount of clay (probability P) remains constant, the local thickness (density $p(\cdot)$) decreases as the area increases.

In probability theory, the *Radon–Nikodym derivative* (**?**) provides a rigorous method to express how one probability measure changes with respect to another. In the context of continuous random variables, the **PDF!** (**PDF!**) is essentially the Radon–Nikodym derivative of the probability measure (probability mass of an event) with respect to the Lebesgue measure (standard geometric volume). This relationship generalizes the familiar link between a **CDF!** (**CDF!**) and a **PDF!**. To provide a physical intuition for this abstract concept, we introduce the "Clay Volume" Analogy illustrated in Figure **??**. The Radon–Nikodym derivative corresponds to the local thickness of the clay at any given point.

For a random variable $\boldsymbol{x}$ and a measurement set A, the probability measure is defined by the integral of the density:

$$P(X \in A) = \int_A p_X(\boldsymbol{x}) d\boldsymbol{x}. \tag{II.41}$$

To relate this to a transformed variable $\boldsymbol{z}$ via an invertible mapping (Equation (**??**)), we rely on the invariance of probability. An event occurs in A for $\boldsymbol{x}$ if and only if it occurs in the pre-image $f^{-1}(A)$ for $\boldsymbol{z}$:

$$P(X \in A) = P(Z \in f^{-1}(A)), \tag{II.42a}$$

$$\int_A p_X(\boldsymbol{x}) d\boldsymbol{x} = \int_{f^{-1}(A)} p_Z(\boldsymbol{z}) d\boldsymbol{z}. \tag{II.42b}$$

We apply the multivariate change of variables formula (Equation **??**) to the right-hand integral in Equation (**??**). This transforms the domain of integration from $f^{-1}(A)$ back to A, introducing the Jacobian determinant to account for local volume distortion:

$$\int_{f^{-1}(A)} p_Z(\boldsymbol{z}) d\boldsymbol{z} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right| d\boldsymbol{x}. \tag{II.43}$$

Here, the Jacobian determinant of the inverse mapping f^{-1} is related to the forward mapping by:

$$\left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right| = \left|\det \frac{\partial f^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}}\right| = \left|\det \frac{\partial \boldsymbol{z}}{\partial f(\boldsymbol{z})}\right| = \frac{1}{\left|\det \boldsymbol{J}_f(\boldsymbol{z})\right|}. \tag{II.44}$$

By substituting (**??**) into (**??**), we obtain an equality of two integrals defined over the same domain A:

$$\int_A p_X(\boldsymbol{x}) d\boldsymbol{x} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right| d\boldsymbol{x}. \tag{II.45}$$

Crucially, Equation (**??**) holds for any arbitrary measurable set A. In measure theory, if two continuous functions integrate to the same value over every possible subset of the domain, the functions themselves must be identical almost everywhere. Therefore, we can equate the integrands directly:

$$p_X(\boldsymbol{x}) = p_Z(f^{-1}(\boldsymbol{x})) \left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right|. \tag{II.46}$$

Substituting $\boldsymbol{x}$ and $\boldsymbol{z}$ with equations **??** and **??** respectively, it can also be formulated using the forward mapping f:

$$p_Z(\boldsymbol{z}) = p_X(f(\boldsymbol{z})) \left|\det \boldsymbol{J}_f(\boldsymbol{z})\right|. \tag{II.47}$$

These two equations define the **change of variables** formula applied to **NF!**s. They are often formulated in logarithmic form:

$$\log p_X(\boldsymbol{x}) = \log p_Z(f^{-1}(\boldsymbol{x})) + \log \left| \det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right|, \tag{II.48}$$

$$\log p_Z(\boldsymbol{z}) = \log p_X(f(\boldsymbol{z})) + \log \left| \det \boldsymbol{J}_f(\boldsymbol{z}) \right|. \tag{II.49}$$

The derivation of the probability density in Equation (**??**) relies fundamentally on the Change of Variables theorem. Consequently, for a Normalizing Flow to be both mathematically valid and computationally tractable, the transformation f cannot be arbitrary; it must satisfy three strict structural constraints:

1. **Invertibility (Bijectivity):** The mapping $f : Z \to X$ must be bijective. This ensures that the inverse f^{-1} exists and is unique, allowing distinct data points $\boldsymbol{x}$ to be uniquely mapped to latent representations $\boldsymbol{z}$ (inference) and vice versa (sampling).

2. **Differentiability:** The function f and its inverse must be differentiable. This is a prerequisite for the existence of the Jacobian matrix $\boldsymbol{J}_f$. In differential geometry, a map satisfying conditions (1) and (2) is termed a *diffeomorphism*.

3. **Computational Efficiency of the Jacobian:** While the first two properties ensure mathematical validity, this third property ensures practical feasibility. Training an **NF!** involves maximizing the log-likelihood, which requires calculating the log-determinant of the Jacobian, $\log |\det \boldsymbol{J}_f(\boldsymbol{z})|$. For an arbitrary high-dimensional function with a dimension D, this calculation scales with $\mathcal{O}(D^3)$. To make training efficient, Normalizing Flows are explicitly designed such that this term can be computed in linear time, $\mathcal{O}(D)$, often by restricting the Jacobian to be triangular or diagonal.

Beyond the structural constraints of individual mappings, a powerful feature of **NF!**s is composability (**?**). Given a series of transformations $f_L \circ \cdots \circ f_1$, the following equations hold:

$$\boldsymbol{x} = f_L(f_{L-1}(\cdots f_1(\boldsymbol{z}) \cdots)), \tag{II.50a}$$

$$f^{-1} = (f_L \circ \cdots \circ f_1)^{-1} = f_1^{-1} \circ \cdots \circ f_L^{-1}, \tag{II.50b}$$

$$\boldsymbol{J}_f(\boldsymbol{z}) = \boldsymbol{J}_{f_L}(\boldsymbol{z}_{L-1}) \cdot \boldsymbol{J}_{f_{L-1}}(\boldsymbol{z}_{L-2}) \cdots \boldsymbol{J}_{f_1}(\boldsymbol{z}_0). \tag{II.50c}$$

Equation (**??**) applies the multivariate **chain rule**. A crucial consequence for training is that the log-determinant of the Jacobian decomposes into a sum of terms:

$$\log |\det \boldsymbol{J}_f(\boldsymbol{z})| = \sum_{k=1}^{L} \log |\det \boldsymbol{J}_{f_k}(\boldsymbol{z}_{k-1})|. \tag{II.51}$$

This allows the total change in density to be computed by simply summing the log-determinants of each layer, enabling the construction of deep, expressive flows while maintaining linear computational complexity.

II.4.1.2 Using Normalizing Flows for Modeling and Inference

Invertibility also provides an easy way to fit **NF!** models by minimizing the divergence between the true data distribution and the estimated one. For a **NF!** with parameters $\boldsymbol{\varphi}$, the KL-divergence is given by:

$$D_{\mathrm{KL}}(p_X(\boldsymbol{x})||p_{\boldsymbol{\varphi}}(\boldsymbol{x})) = \mathbb{E}_{\boldsymbol{x}\sim p_X}[\log p_X(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x}\sim p_X}[\log p_{\boldsymbol{\varphi}}(\boldsymbol{x})]. \tag{II.52}$$

In this objective function, the first term (the entropy of the data) depends only on the true distribution and is constant with respect to $\boldsymbol{\varphi}$. Consequently, minimizing the KL divergence is equivalent to maximizing the second term, which corresponds to the expected log-likelihood. Given a dataset $\mathcal{D} = \{\boldsymbol{x}^{(m)}\}_{m=1}^{M}$ drawn from $p_X(\boldsymbol{x})$, we can approximate this expectation using the sample mean (Monte Carlo simulation):

$$\mathbb{E}_{\boldsymbol{x}\sim p_X}[\log p_{\boldsymbol{\varphi}}(\boldsymbol{x})] \approx \frac{1}{M}\sum_{m=1}^{M} \log p_{\boldsymbol{\varphi}}(\boldsymbol{x}^{(m)}). \tag{II.53}$$

This confirms that minimizing the KL divergence is equivalent to **MLE!** (**MLE!**). By substituting the change of variables formula (Equation (**??**)) into the log-likelihood, we obtain the final loss function to be minimized:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\varphi}) &= -\frac{1}{M}\sum_{m=1}^{M}\left[\log p_Z(f_{\boldsymbol{\varphi}}^{-1}(\boldsymbol{x}^{(m)})) + \log\left|\det \boldsymbol{J}_{f_{\boldsymbol{\varphi}}^{-1}}(\boldsymbol{x}^{(m)})\right|\right], \\ &= \frac{1}{M}\sum_{m=1}^{M}\left[\underbrace{\frac{1}{2}\left\|f_{\boldsymbol{\varphi}}^{-1}(\boldsymbol{x}^{(m)})\right\|_2^2}_{\text{inference}} \underbrace{-\log\left|\det \boldsymbol{J}_{f_{\boldsymbol{\varphi}}^{-1}}(\boldsymbol{x}^{(m)})\right|}_{\text{preparation for generation}}\right], \quad \text{when } p_Z(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I}).\end{aligned} \tag{II.54}$$

The parameters $\boldsymbol{\varphi}$ are then optimized using stochastic gradient-based methods.

The loss function of **NF!**s suggests that during training, the learned latent representation is compared with the true latent distribution, while the Jacobian determinant term ensures that this comparison is transformed back to the data space. Recalling the **VAE!**, the first term can be seen as inference, while the second term can be implicitly regarded as "preparation for generation". After the **NF!** model is trained, new data can be generated

by sampling the latent variables and transforming them via the forward mapping. It is important to emphasize that **NF!**s are conventionally trained in one direction at a time. Simultaneously bidirectional training would necessitate computing gradients through the composition of a function and its inverse, leading to problematic gradient flows of the form $\partial\mathcal{L}/\partial\boldsymbol{\varphi}_f(\boldsymbol{\varphi}\boldsymbol{f}^{-1})$, which creates numerical instabilities and computational inefficiencies.

II.4.1.3 Conditional Normalizing Flows

The ability to estimate complex densities is desirable in Bayesian inference of the posterior distribution of variables given observations $p(\boldsymbol{x}|\boldsymbol{y})$. Similar to **VAE!**, amortized Bayesian inference (**?**) is applied in **NF!** to achieve a general posterior estimation across all data points. Obtaining the posterior distribution is represented by a conditional density transformation via a **CNF!** (**CNF!**). The change of variables for a **CNF!** extends the standard one (**??**) by the condition $\boldsymbol{y}$:

$$p_{X|Y}(\boldsymbol{x}|\boldsymbol{y}) = p_{Z|Y}(f^{-1}(\boldsymbol{x},\boldsymbol{y})|\boldsymbol{y}) \left|\det \frac{\partial f^{-1}(\boldsymbol{x},\boldsymbol{y})}{\partial \boldsymbol{x}}\right|, \tag{II.55}$$

where the mapping $f : Z \to X \times Y$ is bijective between X and Z, and $\boldsymbol{z} = f^{-1}(\boldsymbol{x},\boldsymbol{y})$.

Designing such a **CNF!** requires addressing two key considerations in advance: (1) how to manage the conditioned latent variables, and (2) how to incorporate conditions into the flow architecture while preserving invertibility. **?** (**?**) proposed to build the conditional prior $p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y})$ as a diagonal Gaussian:

$$p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y}) = \mathcal{N}(\boldsymbol{z}|\mu(\boldsymbol{y}), \mathrm{diag}[\sigma^2(\boldsymbol{y})]). \tag{II.56}$$

For simpler conditioning scenarios, incorporating the conditional prior could accelerate the transformation by leveraging information from the conditions. For complex conditions, summarizing the conditions through mean and variance may result in information loss, thereby degrading posterior estimation quality. This limitation likely explains why most existing work on **CNF!**s adopts a standard Gaussian as the latent base distribution, bypassing the need for an explicitly conditioned prior and relying instead on the normalizing flows to capture conditional information.

In terms of architectures of **CNF!**s, a direct adaptation based on the **INN!** (**INN!**) is the **cINN!** (**cINN!**) (**?**). It takes the condition as an augmented input to the neural networks. In cases of higher-dimensional conditions, such as images, **?** (**?**) apply a staged **cINN!** coupled with a staged condition network, wherein the conditions are progressively processed, and each intermediate representation is fed into its corresponding **cINN!** block.

In scenarios involving complexly structured conditions and low-dimensional target

variables, for example, parameter identification from observed time series signals, BayesFlow (**?**) offers a general framework by extracting informative representations through a summary network and incorporating them, alongside the inputs as augmented inputs to the **cINN!**. This framework is particularly well-suited for parameter identification from time series observations, where the summary network is typically chosen as a time series modeling architecture, such as **RNN!**. The architecture is illustrated in Figure **??**. Both the summary network and **cINN!** are optimized together by minimizing the **KL!**-divergence between the true posterior density and the estimated one (similar to Equation (**??**)):

$$D_{\mathrm{KL}}(p_{X|Y}(\boldsymbol{x}|\boldsymbol{y})||p_{\boldsymbol{\varphi},\boldsymbol{\phi}}(\boldsymbol{x}|\boldsymbol{y})) \approx const.- \\ \frac{1}{M}\sum_{m=1}^{M}[\log p_Z(f_{\boldsymbol{\varphi}}^{-1}(\boldsymbol{x}^{(m)}, h_{\boldsymbol{\phi}}(\boldsymbol{x}^{(m)}))) + \log\left|\det \boldsymbol{J}_{f_{\boldsymbol{\varphi}}^{-1}}(\boldsymbol{x}^{(m)}, h_{\boldsymbol{\phi}}(\boldsymbol{x}^{(m)}))\right|], \quad \text{(II.57)}$$

where f is the **cINN!** parameterized by $\boldsymbol{\varphi}$ and h is the summary network parameterized by $\boldsymbol{\phi}$.

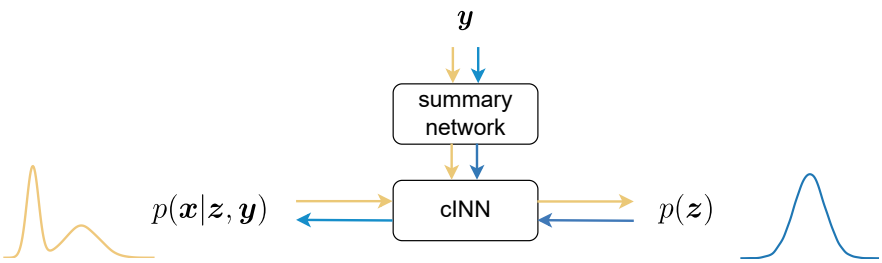


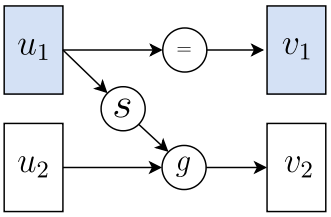
Figure II.5: The architecture of BayesFlow. The blue arrows indicate the forward direction for training BayesFlow, while the yellow arrows indicate the reverse direction for inference. The summary network extracts informative representations from the observations, which are then fed into the **cINN!** alongside the target variables.

The joint optimization approach encourages the summary network to deliver maximally useful information to the **cINN!** and thus improve inference capability. In this work, we adopt the BayesFlow framework as a flexible and scalable solution for amortized Bayesian inference, adapting its summary network to accommodate diverse time series observations. This framework forms the foundation of our proposed methods for parameter identification, system dynamics modeling, and state estimation.

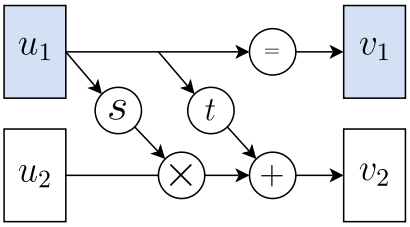
II.4.2 Coupling Flows

The basic idea of a coupling flow is to partition the input variables into two subsets: one subset is processed by neural networks while being passed directly to the output variables,

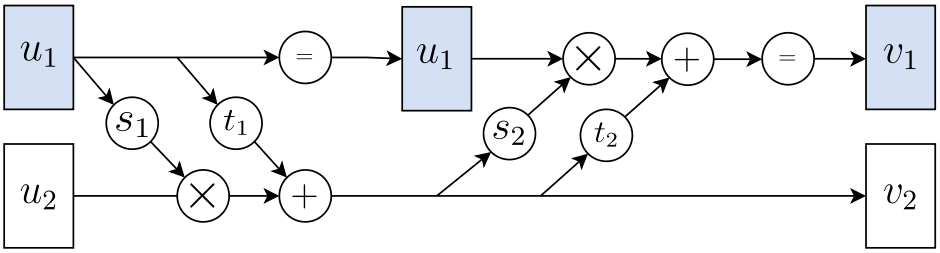
and the other subset is combined with the processed subset through affine functions.



(a) The architecture of a **NICE!** (**NICE!**) layer. s is a neural network, and g is an affine function.



(b) The architecture of a **RealNVP!** (**RealNVP!**) layer. s and t are neural networks. The symbol $\times$ denotes element-wise multiplication by the exponential function over the input.



(c) The architecture of an **INN!** layer. s_1, s_2, t_1, and t_2 are neural networks. The symbol $\times$ denotes element-wise multiplication by the exponential function over the input.

Figure II.6: The architectures of coupling flows. The input of the layer is $\boldsymbol{u}$ and the output is $\boldsymbol{v}$.

The first proposed coupling flow was **NICE!** (**?**). The input variables $\boldsymbol{u}$ are split into two parts $[\boldsymbol{u}_1, \boldsymbol{u}_2]$, and the corresponding output variables are $\boldsymbol{v} = [\boldsymbol{v}_1, \boldsymbol{v}_2]$. The architecture in the forward direction is formulated by the following functions:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = g(\boldsymbol{u}_2; s(\boldsymbol{u}_1)), \tag{II.58}$$

where s is a neural network and g is an affine function $g(a; b) = a + b$ or $a \odot b$ (element-wise multiplication). The forward architecture is illustrated in Figure **??**. Then, the inverse functions are easily calculated:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = g^{-1}(\boldsymbol{v}_2; s(\boldsymbol{u}_1)). \tag{II.59}$$

From the above functions, the Jacobian matrix is:

$$\frac{\partial \boldsymbol{v}}{\partial \boldsymbol{u}} = \begin{bmatrix} \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_2 \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_2 \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2 \end{bmatrix} \tag{II.60}$$

Because of the triangular matrix, the determinant is $\partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2$. By composing multiple coupling layers, we can construct increasingly complex hierarchical transformations. Since each coupling layer preserves a subset of its input unchanged, it is essential to alternate the roles of the partitioned subsets across successive layers. This ensures that all input dimensions are eventually processed. Analyzing the Jacobian reveals that a minimum of three coupling layers is required for full inter-dimensional interaction. In practice, we typically employ four layers to achieve sufficient expressiveness.

An improved architecture is **RealNVP!**, where one part of the variables is processed by two neural networks. The forward transformation is:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = \boldsymbol{u}_2 \odot \exp(s(\boldsymbol{u}_1)) + t(\boldsymbol{u}_1). \tag{II.61}$$

The inverse transformation is:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = (\boldsymbol{v}_2 - t(\boldsymbol{u}_1)) \odot \exp(-s(\boldsymbol{u}_1)). \tag{II.62}$$

The Jacobian determinant $\det |\partial \boldsymbol{v}/\boldsymbol{u}|$ is then $\exp(\Sigma_i s(\boldsymbol{u}_{1,i}))$. The architecture is shown in Figure **??**. To interact all variables in one block, **INN!** (**?**) extends **RealNVP!** by another coupling block of the same form, as shown in Figure **??**.

A further enhancement of **RealNVP!** is Glow (**?**), which inserts an invertible normalization layer and an invertible permutation layer before affine transformation. **HINT!** (**HINT!**) (**?**) partitions the inputs of coupling blocks recursively to construct a dense, triangular Jacobian. Another method to increase interaction among variables is **MAF!** (**MAF!**) (**?**), which can be considered as a generalized **RealNVP!** that leverages autoregressive transformations between variables. Apart from affine coupling, **NSF!** (**NSF!**) (**?**) applies monotonic rational-quadratic spline functions, enabling more expressive and smooth transformations, but at the cost of higher complexity.

II.4.3 Residual Flows

Such flows build stepwise invertible transformations between distributions, enabling tractable Jacobian calculation and density estimation. A notable drawback lies in the strict architectural constraint on coupling blocks. To relax this limitation, residual flows (**??**) introduce invertible mapping through specified residual blocks. A residual block (**?**) takes the form $f(x) = x + F(x)$, where F is a neural network. When F is constrained to

be Lipschitz continuous and the Lipschitz constant is smaller than 1, the function $f(x)$ is invertible.

The definition of a Lipschitz continuous function follows the work of **?** (**?**).

Definition 1. *(**Lipschitz continuous function**) A function $f : \mathbb{R}^d \to \mathbb{R}^K$, defined on some domain $dom(f) \subseteq \mathbb{R}^d$, is called C-Lipschitz continuous w.r.t. some α-norm, if $\forall \boldsymbol{x}, \boldsymbol{y} \in dom(f) : \|f(\boldsymbol{x}) - f(\boldsymbol{y})\|_\alpha \leq C \|\boldsymbol{x} - \boldsymbol{y}\|_\alpha$. C is called the Lipschitz constant of f and $0 < C < \inf$.*

Lemma 1. *(**Invertibility of Residual Units**) Let $f(\mathbf{x}) = \mathbf{x} + F(\mathbf{x})$. If F is C-Lipschitz continuous with $C < 1$, then f is a bijection.*

Proof. To show f is invertible, we must show that for any $\mathbf{y}$, there exists a unique $\mathbf{x}$ such that $\mathbf{y} = \mathbf{x} + F(\mathbf{x})$. This can be rewritten as a fixed-point problem:

$$\mathbf{x} = \mathbf{y} - F(\mathbf{x}) = G(\mathbf{x}). \tag{II.63}$$

By the definition of G, for any $\mathbf{x}_1, \mathbf{x}_2$:

$$\begin{aligned} \|G(\mathbf{x}_1) - G(\mathbf{x}_2)\| &= \|(\mathbf{y} - F(\mathbf{x}_1)) - (\mathbf{y} - F(\mathbf{x}_2))\| && \text{(II.64)} \\ &= \|F(\mathbf{x}_2) - F(\mathbf{x}_1)\| && \text{(II.65)} \\ &\leq C \|\mathbf{x}_2 - \mathbf{x}_1\| . && \text{(II.66)} \end{aligned}$$

Since $C < 1$, G is a contraction mapping. According to the *Banach Fixed-Point Theorem* (**?**), G has a unique fixed point $\mathbf{x}^*$ for every $\mathbf{y}$, implying f is bijective. □

The residual block can be seen as an Euler discretization of an **ODE!** (**ODE!**) (**???**), which is the foundation of continuous **NF!**s. A simplified comparison between the residual flow, continuous flow, and flow matching is illustrated in Figure **??**.

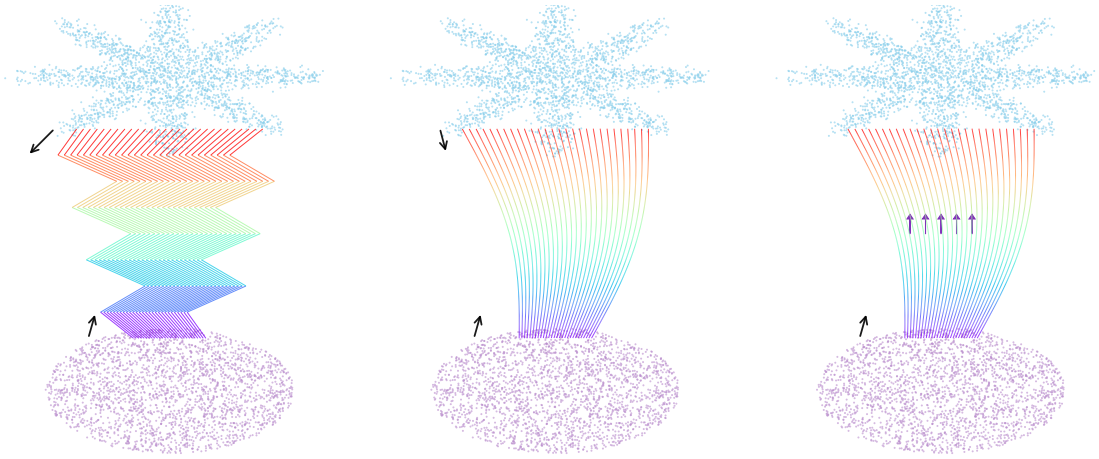
II.4.4 Continuous Flows

When the transformation from latent space to data space is modeled by a continuous dynamical system, the **NF!** is called a *continuous **NF!***. The continuous dynamical system can be formulated by a Neural **ODE!**, which is introduced as follows.

II.4.4.1 Neural Ordinary Differential Equations

For a dynamical system with states $\boldsymbol{h}$, an **ODE!** is:

$$\frac{d\boldsymbol{h}(t)}{dt} = f(\boldsymbol{h}(t), t; \boldsymbol{\varphi}), \tag{II.67}$$



(a) Residual Flow **(b)** Continuous Flow **(c)** Flow Matching

Figure II.7: Illustration of the Residual Flow, Continuous Flow, and Flow Matching. The bottom point cloud is the base distribution in the latent space, while the top one in the star shape is the target distribution in the data space. The residual flow and continuous flow are trained from the data to latent space and sampled from latent to the data space, while Flow Matching is trained by learning the vector field and sampled from the latent to the data space. The reverse direction is not used by Flow Matching.

where $\boldsymbol{\varphi}$ denotes system parameters. When the **ODE!** is modeled by neural networks, it becomes a Neural **ODE!** (**?**). Given the initial state $\boldsymbol{h}(0)$, the final state $\boldsymbol{h}(T)$ is the solution of the initial value problem at the time step T using a numerical solver. The loss function of training a Neural **ODE!** is:

$$\mathcal{L}(\boldsymbol{h}(T)) = \mathcal{L}(\int_0^T f(\boldsymbol{h}(t), t; \boldsymbol{\varphi})dt), \tag{II.68}$$

where f is a neural network parameterized by $\boldsymbol{\varphi}$. A straightforward application of Neural **ODE!**s is learning continuous time-series models. Moreover, when the time step is interpreted as architectural depth, the distributional transformation in **NF!**s can also be constructed with a Neural **ODE!**. The Neural **ODE!** replaces an infinite number of transformations by change of variables with a single neural network, which represents the derivative of transformation at each depth. This enables more efficient and expressive probabilistic modeling and inference.

Taking $\mathcal{U}_\tau$ as the state variable at flow time τ, we define a continuous **NF!** as a Neural **ODE!**:

$$\frac{d\mathcal{U}_\tau}{d\tau} = f(\mathcal{U}_\tau, \tau; \boldsymbol{\varphi}), \tag{II.69}$$

where $\mathcal{U}_\tau$ follows the density $p(\mathcal{U}_\tau)$ at an instantaneous step $\tau \in [\tau_0, \tau_1]$. The **NF!** f is parameterized by $\boldsymbol{\varphi}$. To apply the Neural **ODE!** as continuous **NF!**s, the function f should be uniformly Lipschitz continuous in the states $\mathcal{U}_\tau$ and continuous in depth τ.

The Lipschitz condition guarantees that there exists a unique solution curve for every initial condition $\mathcal{U}_0 := \boldsymbol{z}$. This is known as the Picard–Lindelöf theorem (**?**). If the dynamics function f is smooth, then it is a *diffeomorphism*, which means we can recover the same $\mathcal{U}_0$ by running the **ODE!** backward. Recalling the residual flows, where the Lipschitz constant must be smaller than 1 (see **??**), invertibility is ensured naturally by a smooth and Lipschitz continuous **ODE!**. In practice, neural networks with smooth continuous activation functions satisfy this requirement.

A central technical challenge in training continuous-depth networks lies in executing reverse-mode differentiation (backpropagation) through the ODE solver. Two primary strategies exist for this task. The first is the backward auto-differentiation (*discretize-then-optimize*), which differentiates directly through the operations of the numerical solver. While this method is numerically stable and standard for stiff differential equations, it typically incurs a high memory cost as it requires storing intermediate states of the solver operations.

To mitigate this memory bottleneck, the continuous adjoint method (**???**) is often employed (*optimize-then-discretize*). This method computes gradients by solving an augmented ODE backward in time. Specifically, instead of storing intermediate states, the solver reconstructs the state trajectory backward from the final time point, effectively re-solving the dynamics in reverse alongside the adjoint state. This approach reduces memory cost to $\mathcal{O}(1)$ with respect to integration time. However, because the system dynamics must be evaluated a second time during this reconstruction, it results in increased computational runtime. This trade-off makes the adjoint method ideal for high-dimensional variables and long sequences, but computationally slower for models with expensive architecture dynamics.

II.4.4.2 Continuous Normalizing Flows

In addition to the ODE **??**, the log-density of states changes according to another **ODE!**, the **instantaneous change-of-variables formula** (**?**):

$$\frac{\partial \log p(\mathcal{U}_\tau)}{\partial \tau} = -\mathrm{Tr}\left(\frac{\partial f(\mathcal{U}_\tau, \tau; \boldsymbol{\varphi})}{\partial \mathcal{U}_\tau}\right). \tag{II.70}$$

Directly calculating the trace is computationally expensive. For B backward passes to get the diagonal, $\mathcal{O}(B)$ computational cost and $\mathcal{O}(B^2)$ memory cost are required. Alternatively,

the trace can be approximated through Skilling-Hutchinson trace estimator (**???**):

$$\mathrm{Tr}(\frac{\partial f}{\partial \mathcal{U}_\tau}) = \mathbb{E}_{p(\boldsymbol{\epsilon})}[\boldsymbol{\epsilon}^T \frac{\partial f}{\partial \mathcal{U}_\tau}\boldsymbol{\epsilon}], \tag{II.71}$$

where $\boldsymbol{\epsilon}$ is a noise vector sampled from a standard normal distribution and kept unchanged to maintain determinism of the **ODE!**. The part of vector-Jacobian product $\boldsymbol{\epsilon}^T \partial f / \partial \mathcal{U}_\tau$ is computed with automatic differentiation. In this way, the computational cost is reduced to $\mathcal{O}(1)$ with respect to the number of backward passes (one backward pass for each trace approximation). There is no need to store the full Jacobian matrix. The trade-off is that the estimator introduces variance, as it yields a stochastic approximation of the trace rather than the exact value; in practice, one or a few samples per step are typically sufficient to keep training stable.

For a continuous **NF!** between the data distribution $p(\mathcal{U}_1) = p_X(\boldsymbol{x})$ and the latent distribution $p(\mathcal{U}_0) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, we define the augmented states as $[\mathcal{U}_\tau, \log p_X(\boldsymbol{x}) - \log p(\mathcal{U}_\tau)]$. In the original formulation (**?**), the second term is $\log p(\mathcal{U}_\tau)$ itself; in that case, one must still calculate $\log p_X(\boldsymbol{x}) - \log p(\mathcal{U}_\tau)$. The difference-based formulation used here yields a consistent form for both initial values and solutions. The total dynamics becomes:

$$\frac{d}{d\tau}\begin{bmatrix} \mathcal{U}_\tau \\ \log p_X(\boldsymbol{x}) - \log p(\mathcal{U}_\tau) \end{bmatrix} = \begin{bmatrix} f(\mathcal{U}_\tau, \tau; \boldsymbol{\varphi}) \\ \mathrm{Tr}\left(\frac{\partial f}{\partial \mathcal{U}_\tau}\right) \end{bmatrix} \tag{II.72}$$

Then, building the continuous **NF!** is equivalent to solving the initial value problem (**?**):

$$\underbrace{\begin{bmatrix} \mathcal{U}_0 \\ \log p_X(\boldsymbol{x}) - \log p(\mathcal{U}_0) \end{bmatrix}}_{\text{solutions}} = \underbrace{\begin{bmatrix} \mathcal{U}_1 = \boldsymbol{x} \\ \log p_X(\boldsymbol{x}) - \log p(\mathcal{U}_1) = 0 \end{bmatrix}}_{\text{initial values}} + \int_1^0 \underbrace{\begin{bmatrix} f(\mathcal{U}_\tau, \tau; \boldsymbol{\varphi}) \\ \mathrm{Tr}\left(\frac{\partial f}{\partial \mathcal{U}_\tau}\right) \end{bmatrix}}_{\text{dynamics}} d\tau. \tag{II.73}$$

It is worth noting that the dynamics are integrated backward from $\tau = 1$ to $\tau = 0$, meaning from the data side to the latent side. The flow time span can also be optimized to further improve the computational efficiency (**?**).

Continuous **NF!**s release the architectural constraints of discrete flows, allowing more flexible and expressive density estimation through the integration of Neural **ODE!**s. This improved expressiveness, however, comes at a cost: computing the Jacobian trace and the associated ODE integrals is significantly more expensive than in stacked coupling flows.

II.4.5 Flow Matching

FM! (**FM!**) (**?**) is a recent approach for training continuous **NF!**s. The key idea is to learn a vector field that directly matches the stepwise transport (infinitesimal transport)

between the data distribution and the latent distribution. This is achieved by minimizing the expected squared error between the learned vector field and the true velocity field defined by the optimal transport plan. The optimal transport plan is typically a linear interpolation between the latent samples $\mathcal{U}_0$ and the data samples $\boldsymbol{x} = \mathcal{U}_1$:

$$\boldsymbol{v}^{OT}(\mathcal{U}_\tau, \tau) = \frac{\mathcal{U}_1 - \mathcal{U}_0}{1 - 0}. \tag{II.74}$$

Along this path, the marginal distribution at flow time τ is denoted $p_t(\mathcal{U}_\tau)$. For the linear interpolation with $\mathcal{U}_\tau = (1-\tau)\mathcal{U}_0 + \tau\,\boldsymbol{x}$, where $\mathcal{U}_0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ (latent distribution) and $\boldsymbol{x} \sim p_X(\boldsymbol{x})$ (data distribution), $p_t(\mathcal{U}_\tau)$ is the joint distribution through the interpolation:

$$p_t(\mathcal{U}_\tau) = (1-\tau)\, p(\mathcal{U}_0) + \tau\, p_X(\boldsymbol{x}). \tag{II.75}$$

Thus, sampling $\mathcal{U}_\tau \sim p_t(\mathcal{U}_\tau)$ amounts to drawing a random point on the transport path at flow time τ. The training objective is formulated as:

$$\mathcal{L}(\boldsymbol{\varphi}) = \mathbb{E}_{\tau\sim\mathbb{U}(0,1),\, \mathcal{U}_\tau\sim p_t(\mathcal{U}_\tau)} \left[\left\| v_{\boldsymbol{\varphi}}(\mathcal{U}_\tau, \tau) - \boldsymbol{v}^{OT}(\mathcal{U}_\tau, \tau) \right\|^2 \right], \tag{II.76}$$

where $v_{\boldsymbol{\varphi}}(\mathcal{U}_\tau, \tau)$ is the learned vector field parameterized by $\boldsymbol{\varphi}$, and $\boldsymbol{v}^{OT}(\mathcal{U}_\tau, \tau)$ is the true velocity field derived from the optimal transport plan. Simply put, the **FM!** loss regresses the vector field $\boldsymbol{v}^{OT}$ with a neural network $v_{\boldsymbol{\varphi}}$. Upon reaching zero loss, the learned continuous **NF!** model will generate data samples by integrating the learned vector field from $\tau = 0$ to $\tau = 1$.

The main difference from other **NF!**s is that **FM!** is not trained by maximizing likelihood via change of variables between distributions. Instead, it focuses on learning the vector field that defines the continuous transformation between distributions. This approach avoids the computational overhead of solving **ODE!**s during training, making it more efficient and scalable to high-dimensional data. However, **FM!** cannot be directly applied to exact density estimation because it does not transform the data distribution into a known latent distribution. The sample density it generates is only an approximation of the data density. For exact estimation, **FM!** must invert the generative process via a backward **ODE!**, which makes it computationally heavier than the single-pass inference of other **NF!**s.

II.4.6 Diffusion Models

Diffusion models are another major family of **DGM!**s and have gained increasing attention because of their extraordinary expressive power (**???**). They are closely related to **FM!**: both define an artificial flow time along the transformation from a latent noise distribution

to the data distribution, and both learn a neural field that drives samples from the source side to the data side. The main difference is that **FM!** learns a deterministic transport velocity $v_{\boldsymbol{\varphi}}(\mathcal{U}_\tau, \tau)$, whereas diffusion models usually learn the reverse of a stochastic noising process, either through a noise-prediction network or through a score field.

In most diffusion-model literature, diffusion time is written in the opposite direction from the flow-matching convention in Section **??**: the forward noising process starts from data at $t = 0$ and ends at noise at $t = T$, while generation runs backward. To keep the notation consistent in this work, we use a scaled flow time $\tau \in [0, 1]$ with $\tau = 0$ denoting noise and $\tau = 1$ denoting data.

DDPM! (**DDPM!**) (**?**) defines a discrete Markov chain that gradually adds Gaussian noise to data. The noise-adding process is

$$q(\mathcal{U}_\tau|\mathcal{U}_{\tau-1}) = \mathcal{N}\left(\mathcal{U}_\tau; \sqrt{1-\beta_\tau}\mathcal{U}_{\tau-1}, \beta_\tau \boldsymbol{I}\right), \tag{II.77}$$

where β_τ is a prescribed noise schedule. After many steps, the distribution approaches a standard Gaussian. The generative model $\epsilon_{\boldsymbol{\varphi}}$ learns the reverse transition $p_{\boldsymbol{\varphi}}(\mathcal{U}_{\tau-1}|\mathcal{U}_\tau)$ by predicting the added noise. The loss function is simply the mean-squared error between the predicted noise and the true noise:

$$\mathcal{L}(\boldsymbol{\varphi}) = \mathbb{E}_{\tau\sim\mathbb{U}(0,1),\, \mathcal{U}_\tau\sim p_\tau(\mathcal{U}_\tau)}\left[\|\epsilon_{\boldsymbol{\varphi}}(\mathcal{U}_\tau, \tau) - \epsilon\|^2\right]. \tag{II.78}$$

After training, the generative process is performed by sampling from the learned reverse transition $p_{\boldsymbol{\varphi}}(\mathcal{U}_{\tau-1}|\mathcal{U}_\tau)$ from the noise side to the data side, where the noise is gradually removed.

SBM!s (**SBM!**s) extend this idea to continuous time and describe generation through **SDE!**s (**?**). An **SDE!** that perturbs data into noise is defined as

$$d\mathcal{U}_\tau = f(\mathcal{U}_\tau, \tau)d\tau + g(\tau)d\,W_\tau, \quad \mathcal{U}_0 \sim p_X(\boldsymbol{x}), \tag{II.79}$$

where τ denotes diffusion time in the standard noising direction, $f(\mathcal{U}_\tau, \tau)$ is the drift term, $g(\tau)$ is the diffusion coefficient, and W_τ is a standard Wiener process. In the reverse-time process, the drift term of the **SDE!** has to be compensated by a score function $\nabla_{\mathcal{U}_\tau} \log p_\tau(\mathcal{U}_\tau)$, which is learned by a neural network $s_{\boldsymbol{\varphi}}$. The generative process is then defined by the reverse-time **SDE!**:

$$d\mathcal{U}_\tau = \left[f(\mathcal{U}_\tau, \tau) - g(\tau)^2\nabla_{\mathcal{U}_\tau} \log p_\tau(\mathcal{U}_\tau)\right] d\tau + g(\tau)d\,W_\tau. \tag{II.80}$$

Similar to the **DDPM!**, the training objective is to minimize the expected squared error between the learned score function and the true score function. Once the generative

model s_φ is trained, generation is performed by integrating from the noise side toward the data side.

In contrast to **FM!**, diffusion models learn a stochastic generative process, which can represent more flexible intermediate distributions and often provides stronger mode coverage for complex data. However, this comes at a much higher computational cost, as both the iterative noise-removal procedure in **DDPM!** and the numerical **SDE!**-solving procedure in **SBM!**s require many sequential function evaluations during sampling.

II.4.7 Stochastic Interpolant

The previously introduced **FM!** and diffusion models have a similar design. They both predefine a transformation from latent to data space, and then learn the transformation function. After training, new data samples are generated by integrating the learned function. This design is unified by the definition: Stochastic Interpolant (**?**).

Let $\mathcal{U}_\tau$ denote a trajectory-valued random variable defined over the artificial flow time interval $\tau \in [0,1]$. This variable is distinct from the physical state process x_t used in dynamical systems, where t denotes system time.

Definition 2. *Given two probability density functions $p_0, p_1 : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$, a stochastic interpolant between p_0 and p_1 is a stochastic process $\mathcal{U}_\tau$ of the form*

$$\mathcal{U}_\tau = \mathcal{I}(\tau, \mathcal{U}_0, \mathcal{U}_1) + \gamma(\tau)\epsilon, \quad \epsilon \sim \mathcal{N}(0, \boldsymbol{I}_d), \quad \tau \in [0,1]. \tag{II.81}$$

Here,

1. *$\mathcal{I}(\tau, \mathcal{U}_0, \mathcal{U}_1)$ is a deterministic interpolation function satisfying $\mathcal{I}(0, \mathcal{U}_0, \mathcal{U}_1) = \mathcal{U}_0$ and $\mathcal{I}(1, \mathcal{U}_0, \mathcal{U}_1) = \mathcal{U}_1$, together with smoothness and regularity conditions that yield well-defined flow dynamics;*

2. *$\gamma(\tau)$ is a noise schedule satisfying $\gamma(0) = \gamma(1) = 0$ and $\gamma(\tau) > 0$ for $\tau \in (0,1)$, so that stochastic perturbations vanish at the endpoints and remain well behaved in the interior;*

3. *$\mathcal{U}_\tau$ has marginal distribution p_τ for each τ, with p_τ continuously deforming from p_0 to p_1 as τ increases from 0 to 1;*

4. *$\mathcal{U}_0$ and $\mathcal{U}_1$ are random variables with distributions p_0 and p_1, respectively, and are independent of the noise ϵ.*

This framework connects flow matching, continuous normalizing flows, diffusion models, score-based generative models, Schrödinger bridges, and entropy-regularized optimal transport. Figure **??** provides a schematic overview. Its flexibility makes it possible to design stochastic processes tailored to a specific application or data distribution.

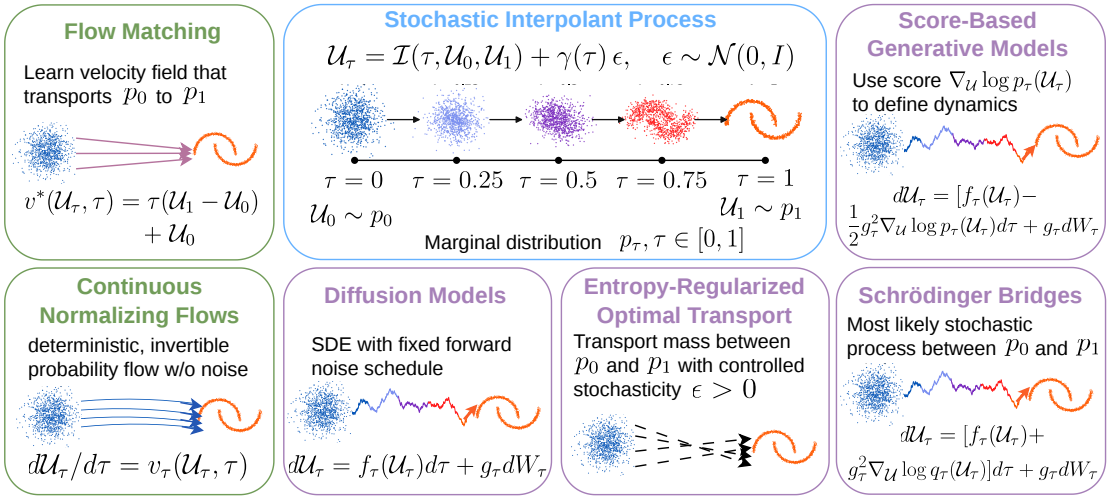


Figure II.8: Schematic of the stochastic interpolant framework. The interpolation function I and noise schedule γ determine the stochastic process and its associated dynamics, thereby unifying several generative modeling frameworks.

II.5 From Bayesian Inference to Control

This section bridges the Bayesian inference methodologies discussed above to their practical applications in control engineering. We first review how Bayesian approaches are employed for system identification and state estimation. Next, we introduce **MPC!** (**MPC!**), which is a widely used model-based control method for optimal decision-making based on the system identification results. Finally, we discuss **RL!** (**RL!**) as a data-driven control paradigm, highlighting its ability to learn control policies directly from interactions with the environment without requiring explicit system models. Together, these approaches illustrate the synergy between Bayesian inference and modern control strategies, enabling robust and adaptive solutions for complex dynamical systems.

II.5.1 Bayesian System Identification

An essential goal of system modeling and identification is to provide a rigorous framework for predicting and controlling the future evolution of system outputs. For any control strategy, the decision-maker needs to predict how the system will behave under any possible control input. Consider a discrete-time dynamical system with inputs $\boldsymbol{u}_n$ and outputs $\boldsymbol{y}_n$ at time t_n. The system evolution is governed by a (possibly stochastic) transition function:

$$\boldsymbol{y}_n = g(\boldsymbol{u}_n, \mathcal{D}_{0:(n-1)}; \boldsymbol{\theta}) + \boldsymbol{\epsilon}_n, \tag{II.82}$$

where g is the system function, $\boldsymbol{\theta}$ are unknown parameters, $\mathcal{D}_{0:(n-1)}$ denotes the history of input-output pairs up to time t_{n-1}, and $\boldsymbol{\epsilon}_n$ is process or measurement noise. Here, we apply the same-time convention ($\boldsymbol{u}_n \to \boldsymbol{y}_n$) instead of the one-step-delaying convention ($\boldsymbol{u}_n \to \boldsymbol{y}_{n+1}$). Given this setup, the joint probability distribution of the future input-output sequence from t_1 to t_N, conditioned on the initial data $\mathcal{D}_0$, can be factorized using the chain rule of probability (**?**):

$$p(\mathcal{D}_{1:N}|\mathcal{D}_0) = \prod_{n=1}^{N} p(\boldsymbol{y}_n|\boldsymbol{u}_n, \mathcal{D}_{0:(n-1)}) p(\boldsymbol{u}_n|\mathcal{D}_{0:(n-1)}), \tag{II.83}$$

where $\mathcal{D}_n = \{\boldsymbol{u}_n, \boldsymbol{y}_n\}$ denotes the input-output pair at time step t_n. The conditional probability density $p(\boldsymbol{u}_n|\mathcal{D}_{0:(n-1)})$ describes the process of determining the input $\boldsymbol{u}_n$ on the basis of the known past history. The left-hand term $p(\boldsymbol{y}_n|\boldsymbol{u}_n, \mathcal{D}_{0:(n-1)})$ represents the dependence of the output on the corresponding input and the known past history, which can be interpreted as the system function g. Identifying this system consists of two steps: model selection and parameter identification. The process of finding the most plausible model from a group of candidates is model selection. System identification then focuses on inferring the parameters of the chosen system model.

II.5.1.1 Bayesian Model Selection

Among different model structures $\{\mathcal{M}_i\}$, $i = 1, \ldots, |\mathcal{M}|$, the probability density of the model $\mathcal{M}_i$ given the data $\mathcal{D}$ measures whether the model is suitable for the dataset, which can also be written using Bayes' theorem:

$$p(\mathcal{M}_i|\mathcal{D}) = \frac{p(\mathcal{D}|\mathcal{M}_i)p(\mathcal{M}_i)}{p(\mathcal{D})}. \tag{II.84}$$

Model selection is also known as model comparison. The relative probability density of two different models given the same data $\mathcal{D}$ is:

$$\frac{p(\mathcal{M}_i|\mathcal{D})}{p(\mathcal{M}_j|\mathcal{D})} = \frac{p(\mathcal{D}|\mathcal{M}_i)p(\mathcal{M}_i)}{p(\mathcal{D}|\mathcal{M}_j)p(\mathcal{M}_j)}. \tag{II.85}$$

Since a model has multiple undetermined parameters, the distribution $p(\mathcal{D}|\mathcal{M})$ is obtained by integrating over all parameter variants. This integral is usually intractable. Instead, **DIC!** (**DIC!**) (**???**) is applied as the model selection criterion. A deviance is defined as follows:

$$\mathrm{dev}(\boldsymbol{\theta}) = -2 \log p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M}), \tag{II.86}$$

where $\boldsymbol{\theta}$ denotes the parameters of the model $\mathcal{M}$. Then, the **DIC!** is defined as:

$$\mathrm{DIC}(\mathcal{M}) = \mathbb{E}_{\mathcal{M}}[\mathrm{dev}(\boldsymbol{\theta})] + P_D(\mathcal{M}), \tag{II.87}$$

where the first term is interpreted as a Bayesian measure of model fit:

$$\mathbb{E}_{\mathcal{M}}[\mathrm{dev}(\boldsymbol{\theta})] = \int p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})\mathrm{dev}(\boldsymbol{\theta})d\boldsymbol{\theta}, \tag{II.88}$$

and the second term is used to measure the model complexity:

$$P_D(\mathcal{M}) = \mathbb{E}_{\mathcal{M}}[\mathrm{dev}(\boldsymbol{\theta})] - \mathrm{dev}(\overline{\boldsymbol{\theta}}), \quad \overline{\boldsymbol{\theta}} = \int p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})\boldsymbol{\theta}d\boldsymbol{\theta}. \tag{II.89}$$

The posterior $p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})$ is obtained by **MCMC!**. **DIC!** balances the goodness of fit (how well the model explains the data) and the model complexity (the number of parameters in the model). A lower **DIC!** value indicates better model fit for the given dataset.

II.5.1.2 Bayesian Parameter Identification

For parameter identification, we restrict the discussion to a simpler model class where the output depends only on the current input and parameters, without explicit history dependence (i.e., a memoryless or static mapping). We define the chosen system model with unknown parameters $\boldsymbol{\theta}$:

$$\boldsymbol{y}_n = f(\boldsymbol{u}_n, \boldsymbol{\theta}) + \boldsymbol{\epsilon}_n. \tag{II.90}$$

The optimal parameters are selected by **MAP!** (**MAP!**) estimation:

$$\boldsymbol{\theta}^*_{MAP} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{D}). \tag{II.91}$$

Applying Bayes' rule (**??**), it becomes:

$$\boldsymbol{\theta}^*_{MAP} = \arg\max_{\boldsymbol{\theta}} \left[\prod_{n=1}^{N} p(\boldsymbol{y}_n|\boldsymbol{u}_n, \boldsymbol{\theta})\right] p(\boldsymbol{\theta}), \tag{II.92}$$

where $\mathcal{D} = \{(\boldsymbol{u}_n, \boldsymbol{y}_n)\}_{n=1}^N$ and $p(\mathcal{D})$ is dropped because it is constant with respect to $\boldsymbol{\theta}$. The noise $\boldsymbol{\epsilon}_n$ is commonly assumed to be Gaussian. Thus, the distribution $p(\boldsymbol{y}_n|\boldsymbol{u}_n, \boldsymbol{\theta})$ has a closed form, which makes this term equivalent to a weighted mean squared error. The prior density term $p(\boldsymbol{\theta})$ acts as $L2$ regularization. In addition to parameter estimates, confidence intervals for the parameters can also be obtained from the posterior distribution.

II.5.2 Bayesian State Estimation

The basic idea of Bayesian state estimation is to apply the estimated posterior of system parameters as a prior and update the posterior with new data (**?**). State estimation involves inference of the internal system states, which are not directly measurable. Expressed in a probabilistic framework, the goal of state estimation is to infer the posterior distribution of the states given observations $p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n})$. Assuming a Markovian system, the posterior of the state chain is factorized as:

$$\begin{aligned} p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n}) &= p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_n, \boldsymbol{y}_{1:n-1}) \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_{0:n}, \boldsymbol{y}_{1:n-1})\, p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \quad \text{[Bayes' rule]} \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)\, p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \quad \text{[Markov property]} \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)\, p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})}\, p(\boldsymbol{x}_{0:n-1}|\boldsymbol{y}_{1:n-1}), \quad \text{[recursion]} \end{aligned} \tag{II.93}$$

where $p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})$ is referred to as *evolution model*, or *state transition model*, and $p(\boldsymbol{y}_n|\boldsymbol{x}_n)$ is called an *observation model*, or *measurement model*. The posterior $p(\boldsymbol{x}_{0:n-1}|\boldsymbol{y}_{1:n-1})$ is the result of the previous time step. Once the observation $\boldsymbol{y}_n$ is received, the term $p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})$ is a constant with respect to $\boldsymbol{x}_{0:n-1}$. Thus, it acts as a normalization factor. The equation above describes a recursive process of state estimation. States are indexed $0:n$ (including the initial $\boldsymbol{x}_0$); observations $1:n$ (no observation at step zero). The prior $p(\boldsymbol{x}_0)$ is the base case of the recursion.

II.5.2.1 Bayesian Filters

When only one state $\boldsymbol{x}_n$ is considered at a time, the problem is called marginal state estimation, or filtering. The equation becomes:

$$\begin{aligned} p(\boldsymbol{x}_n|\boldsymbol{y}_{1:n}) &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)\, p(\boldsymbol{x}_n|\boldsymbol{y}_{1:n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \quad \text{[update]} \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \int p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1}) p(\boldsymbol{x}_{n-1}|\boldsymbol{y}_{1:n-1}) d\boldsymbol{x}_{n-1}. \quad \text{[prediction]}. \end{aligned} \tag{II.94}$$

From this equation, the terms with $\boldsymbol{y}_n$ (a new observation) are separated from the terms with $\boldsymbol{x}_{n-1}$ (the previous state). The first part is the update step, which incorporates the new observation $\boldsymbol{y}_n$ to refine the belief about the current state $\boldsymbol{x}_n$. The second part is the prediction step, which propagates the previous belief $p(\boldsymbol{x}_{n-1}|\boldsymbol{y}_{1:n-1})$ through the evolution model $p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})$ to predict the current state before considering the new observation.

These two steps form the basis of Bayesian filtering methods such as Kalman filters and particle filters (**?**). The main challenge in this equation is to calculate the integral. For arbitrary probability distributions and system equations, this integral has no analytical (closed-form) solution. Therefore, some approximations and restrictions have to be made to make the filtering problem tractable. The most common approximations are linearization and Gaussian assumptions, which lead to the **KF!** (**KF!**) and its variants. For more general cases, Monte Carlo methods are applied to approximate the integral, leading to **PF!** (**PF!**).

The **KF!** (**?**) can be directly derived from Equation (**??**) under the assumptions of linear systems and Gaussian noise. For a linear system:

$$\boldsymbol{x}_n = F_n \boldsymbol{x}_{n-1} + \boldsymbol{w}_n, \quad \boldsymbol{w}_n \sim \mathcal{N}(\boldsymbol{0}, \Sigma_{w_n}), \tag{II.95a}$$

$$\boldsymbol{y}_n = H_n \boldsymbol{x}_n + \boldsymbol{v}_n, \quad \boldsymbol{v}_n \sim \mathcal{N}(\boldsymbol{0}, \Sigma_{v_n}), \tag{II.95b}$$

the iterative algorithm of filtering is:

$$\textbf{Prediction:} \quad \boldsymbol{x}'_n = F_n \boldsymbol{x}_{n-1}, \qquad \Sigma'_n = F_n \Sigma_{n-1} F_n^T + \Sigma_{w_n}. \tag{II.96a}$$

$$\textbf{Update:} \quad K_n = \Sigma'_n H_n^T (H_n \Sigma'_n H_n^T + \Sigma_{v_n})^{-1}, \tag{II.96b}$$

$$\boldsymbol{x}_n = \boldsymbol{x}'_n + K_n(\boldsymbol{y}_n - H_n \boldsymbol{x}'_n), \qquad \Sigma_n = (I - K_n H_n)\Sigma'_n. \tag{II.96c}$$

For nonlinear systems, the **EKF!** (**EKF!**) (**?**) obtains the matrices F_n and H_n by linearizing the nonlinear functions at the current estimate, i.e., $F_n = \nabla_{\boldsymbol{x}} f_n|_{\boldsymbol{x}_n}$ and $H_n = \nabla_{\boldsymbol{x}} h_n|_{\boldsymbol{x}_n}$. In such a way, the **EKF!** can be considered as the first-order approximation to the optimal terms. This approximation, however, can introduce large errors, which may lead to suboptimal performance and sometimes divergence of the filter. The **UKF!** (**UKF!**) (**?**) addresses the approximation issues by carefully choosing a minimal set of sample points, which are called *sigma points*. The sigma points completely capture the mean and covariance of states, which are treated as Gaussian random variables. These sample points are then propagated through the nonlinear functions, and the mean and covariance of the estimate are reconstructed from the transformed points.

Relaxing the assumptions on model linearity and Gaussian noise, **PF!** (**??**) is a more general Bayesian filtering method. The integral in Equation (**??**) is approximated through Monte Carlo estimation, yielding an average of all weighted particles. Each particle represents a possible state of the system, and its weight reflects the likelihood of that state given the observations. In the prediction step, each particle is propagated through the system dynamics. In the update step, weights are adjusted according to each particle based on the likelihood of the observation given the particle's state. Unlike Kalman filters, in each iteration of the **PF!**, a resampling step is included to avoid weight degeneracy,

meaning that most particles carry very small weights over time. In the resampling step, particles are replaced with equally weighted ones, replicating high-weight particles and discarding low-weight ones.

Table **??** summarizes the comparison of the three Bayesian filtering methods. The **KF!** is efficient and optimal for linear systems with Gaussian noise, while the **EKF!** and **UKF!** extend the applicability to nonlinear systems under Gaussian assumptions. The **PF!** provides a flexible framework for general nonlinear and non-Gaussian systems, albeit at a higher computational cost.

Table II.4: Comparison of Bayesian filtering methods.

Bayesian Filter	System Model	Noise
KF!	linear	Gaussian
EKF! / UKF!	nonlinear	Gaussian assumption
PF!	general	arbitrary

II.5.2.2 State-Space Models

When the evolution and observational models are unknown, Kalman filters can not be directly applied. **SSM!**s (**SSM!**s) (**?**) are then used for modeling system dynamics entirely from data. An **SSM!** consists of a latent state vector $\boldsymbol{x}_n$ and an observation vector $\boldsymbol{y}_n$. The state vector captures the internal dynamics of the system, while the observation vector represents the measurable outputs. $\boldsymbol{x}_n$ follows a Markov process, meaning that the previous state $\boldsymbol{x}_{n-1}$ carries all information to obtain the current state $\boldsymbol{x}_n$. $\boldsymbol{y}_n$ depends only on the latent state $\boldsymbol{x}_n$. These assumptions lead to the following factorization of the joint distribution:

$$p(\boldsymbol{y}_{1:N}, \boldsymbol{x}_{0:N}) = p(\boldsymbol{x}_0) \prod_{n=1}^{N} p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1}) p(\boldsymbol{y}_n|\boldsymbol{x}_n). \tag{II.97}$$

In the simplest case, the state transition distribution $p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})$ and the observation distribution $p(\boldsymbol{y}_n|\boldsymbol{x}_n)$ are assumed to be linear Gaussian models:

$$\boldsymbol{x}_0 \sim \mathcal{N}(\boldsymbol{0}, \Sigma_0), \tag{II.98a}$$

$$\boldsymbol{x}_n = \boldsymbol{F}\boldsymbol{x}_{n-1} + \boldsymbol{w}_n, \quad \boldsymbol{w}_n \sim \mathcal{N}(\boldsymbol{0}, \Sigma_{w_n}) \tag{II.98b}$$

$$\boldsymbol{y}_n = \boldsymbol{H}\boldsymbol{x}_n + \boldsymbol{v}_n, \quad \boldsymbol{v}_n \sim \mathcal{N}(\boldsymbol{0}, \Sigma_{v_n}). \tag{II.98c}$$

In the previous section, we showed state estimation using Kalman filters with known system matrices ($\boldsymbol{F}$, $\boldsymbol{H}$) and noise covariance matrices ($\Sigma_{w_n}, \Sigma_{v_n}$). When these matrices are unknown, they can be estimated from data using the **EM!** (**EM!**) algorithm (**?**). The

EM! algorithm iteratively maximizes the expected log-likelihood of the complete data (both observed and latent variables) given the current parameter estimates. The algorithm consists of two main steps:

- E-step: Compute the expectations of the latent states using the current parameter estimates:

$$E_n = E[\boldsymbol{x}_n], \quad E_{n,n} = E[\boldsymbol{x}_n \boldsymbol{x}_n^T], \quad E_{n,n-1} = E[\boldsymbol{x}_n \boldsymbol{x}_{n-1}^T].$$

  The states are obtained by running the Kalman filter (Equation (**??**), (**??**)) on the current parameters.
- M-step: Update parameters using the expectations:

$$\boldsymbol{F} = \left(\sum_{n=1}^{N} E_{n,n-1}\right)\left(\sum_{n=1}^{N} E_{n-1,n-1}\right)^{-1}, \quad \boldsymbol{H} = \left(\sum_{n=0}^{N} \boldsymbol{y}_n E_n^T\right)\left(\sum_{n=1}^{N} E_{n,n}\right)^{-1},$$
$$\Sigma_{w_n} = \frac{1}{N}\left(\sum_{n=1}^{N} E_{n,n} - \boldsymbol{F}\sum_{n=1}^{N} E_{n,n-1}^T\right), \quad \Sigma_{v_n} = \frac{1}{N}\left(\sum_{n=0}^{N} \boldsymbol{y}_n \boldsymbol{y}_n^T - \boldsymbol{H}\sum_{n=0}^{N} \boldsymbol{y}_n E_n^T\right).$$

The E-step and M-step are iterated until convergence. The **EM!** algorithm guarantees that the likelihood of the observed data does not decrease with each iteration, leading to convergence to a local maximum. Similar to Kalman filters, for nonlinear models, **EKF!**s and **UKF!**s are applied in the E-step; while for nonlinear and non-Gaussian **SSM!**s, **PF!**s are used to approximate the expectations in the E-step (**?**).

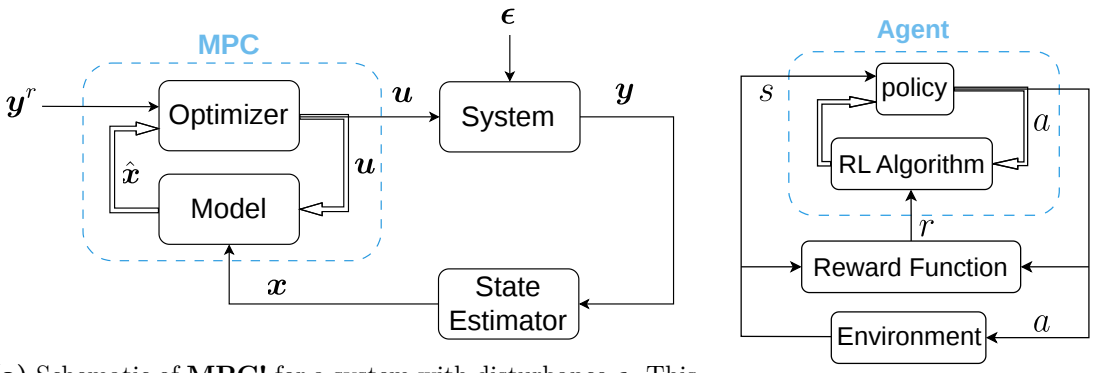
II.5.2.3 Stochastic Differential Equations

A key strength of **SSM!**s is that they explicitly model latent states and noise as distinct components, when this separation is well posed, for example under additive noise and identifiable structures. When we upgrade **SSM!**s from discrete time to continuous time, **SDE!**s (**SDE!**s) (**?**) are a natural choice for modeling the evolution of latent states. For a stochastic process $X(t)$ for variables $\boldsymbol{x}$, the general form of an **SDE!** is:

$$dX(t) = \mu(X(t), t)\, dt + \sigma(X(t), t)\, dW(t), \tag{II.100}$$

where $dX(t)$ is the infinitesimal change in the state, $\mu(X(t), t)$ is the drift function representing the deterministic part of the dynamics, $\sigma(X(t), t)$ is the diffusion function representing the stochastic part, and $W(t)$ is a Wiener process (introduced in Section **??**) capturing the random fluctuations.

II.5.3 Model Predictive Control



(a) Schematic of **MPC!** for a system with disturbance ϵ. This figure is adapted from **?**.

(b) Block diagram of **RL!**.

Figure II.9: MPC! and **RL!**.

After investigating Bayesian system identification and state estimation, we now turn our attention to control strategies, meaning the optimization of system inputs given target and system observations. Control methods can be broadly categorized into model-based and model-free approaches. Model-based control relies on an explicit model of the system dynamics to predict future states and optimize control actions accordingly. In contrast, model-free control approaches do not require a state transition model of the system, instead learning control strategies directly through interactions with the system. A prominent example of model-based control is **MPC!** (**MPC!**) (**??**).

II.5.3.1 Nominal Model Predictive Control

A system to be controlled is first modeled in discrete time:

$$\boldsymbol{x}_{n+1} = f(\boldsymbol{x}_n, \boldsymbol{u}_n), \tag{II.101a}$$

$$\boldsymbol{y}_n = h(\boldsymbol{x}_n), \tag{II.101b}$$

where $\boldsymbol{x}_n$ is the system state, $\boldsymbol{u}_n$ and $\boldsymbol{y}_n$ are the control input and system output at time step n, respectively. The function f describes the state transition, and h is the measurement function.

Assuming that a full measurement or estimate of the state $\boldsymbol{x}_n$ is available, the objective of **MPC!** is to find an optimal sequence of control inputs $\boldsymbol{u}_{n:n+N-1}$ over a finite prediction

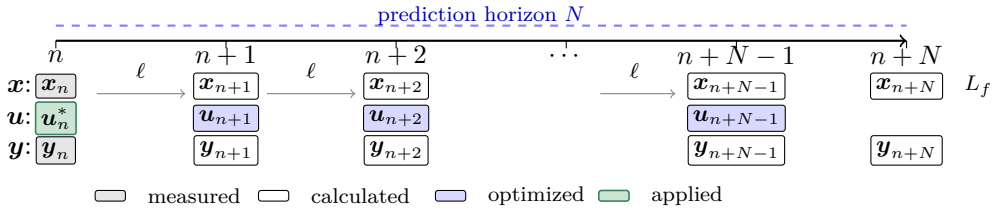


Figure II.10: Time dependency of variables in **MPC!**. The state $\boldsymbol{x}_n$ is measured at the current step. The optimizer finds $\boldsymbol{u}_{n:n+N-1}$; only $\boldsymbol{u}_n^*$ is applied. Dynamics f and output h propagate along the horizon. Stage costs ℓ sum over $i = n, \ldots, n+N-1$; L_f is the terminal cost.

horizon N. The objective function is:

$$\min_{\boldsymbol{u}_{n:n+N-1}} \sum_{i=n}^{n+N-1} \ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r) + L_f(\boldsymbol{y}_{n+N}^r, \boldsymbol{y}_{n+N}), \tag{II.102a}$$

$$\ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r) = (\boldsymbol{y}_i - \boldsymbol{y}_i^r)^T Q_y (\boldsymbol{y}_i - \boldsymbol{y}_i^r) + \boldsymbol{u}_i^T Q_u \boldsymbol{u}_i, \tag{II.102b}$$

$$\text{subject to} \quad \boldsymbol{x}_{i+1} = f(\boldsymbol{x}_i, \boldsymbol{u}_i), \quad \boldsymbol{y}_i = h(\boldsymbol{x}_i), \tag{II.102c}$$

$$g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0, \quad j = 1, \ldots, n_g, \quad i = n, \ldots, n+N-1, \tag{II.102d}$$

$$\boldsymbol{u}_{min} \leq \boldsymbol{u}_i \leq \boldsymbol{u}_{max}, \quad i = n, \ldots, n+N-1. \tag{II.102e}$$

Figure **??** illustrates the time dependency of the variables in the **MPC!** formulation. At the current time step n, the state $\boldsymbol{x}_n$ is measured or estimated. The optimizer computes the control sequence $\boldsymbol{u}_{n:n+N-1}$; only the first element $\boldsymbol{u}_n^*$ is applied to the system. The dynamics f and measurement h propagate states and outputs over the prediction horizon. The stage cost ℓ is evaluated at each step $i = n, \ldots, n+N-1$, and the final cost L_f at the terminal step $n+N$.

The cost function consists of a sum of stage costs ℓ and a final cost function L_f. In the stage cost, weighting matrices Q_y and Q_u are used to balance the trade-off between tracking performance and control effort. The reference trajectory $\boldsymbol{y}^r$ represents the desired output over the prediction horizon. The constraints $g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0$ can represent physical limitations of the system or other operational constraints. A simplified schematic of **MPC!** is shown in Figure **??**.

The optimization problem in **MPC!** can be solved using various numerical methods, such as sequential quadratic programming, interior-point methods, or gradient-based approaches (**?**). The choice of method depends on the specific characteristics of the system model and constraints. The first step of the solution (Equation (**??**)) $\boldsymbol{u}_n^*$ is applied to the system, and the whole process is repeated at the next time step with updated state

information. This is why **MPC!** is also known as receding horizon control (**?**).

II.5.3.2 Stochastic Model Predictive Control

Taking system uncertainties and disturbances into account, robust **MPC!** (**?**) aims to satisfy system stability and constraints despite uncertainties and disturbances. There exist several sound robust **MPC!** methods, including Min-Max**MPC!** (**?**), scenario **MPC!** (**?**), and stochastic **MPC!** (**??**). The Min-Max **MPC!** chooses control actions that minimize the worst possible cost under bounded disturbances. The scenario **MPC!** samples a finite number of random uncertainty realizations as scenarios and optimizes control actions that satisfy constraints for each scenario. Stochastic **MPC!** incorporates the probabilistic descriptions of uncertainties into a stochastic optimal control problem through defining chance constraints. A mathematical formulation of stochastic **MPC!** is as follows (**?**) :

$$\min_{\boldsymbol{u}_{n:n+N-1}} \sum_{i=n}^{n+N-1} \mathbb{E}[\ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r)] + \mathbb{E}[L_f(\boldsymbol{y}_{n+N}^r, \boldsymbol{y}_{n+N})], \tag{II.103a}$$

$$\text{subject to} \quad \boldsymbol{x}_{i+1} = f(\boldsymbol{x}_i, \boldsymbol{u}_i, \boldsymbol{w}_i), \quad \boldsymbol{y}_i = h(\boldsymbol{x}_i, \boldsymbol{v}_i), \tag{II.103b}$$

$$P[g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0] \geq 1 - \beta_j, \quad \beta_j \in (0, 1], \quad j = 1, \ldots, n_g, \quad i = n, \ldots, n+N-1, \tag{II.103c}$$

$$\boldsymbol{u}_{min} \leq \boldsymbol{u}_i \leq \boldsymbol{u}_{max}, \quad i = n, \ldots, n+N-1. \tag{II.103d}$$

The chance constraints P regularize the inequality constraints g_i, allowing for a small probability β_j of constraint violation. The expectation in the cost function can be approximated using Monte Carlo simulation or analytical methods if the distributions are known.

II.5.4 Reinforcement Learning for Control

When an accurate model of the system is unavailable or difficult to obtain, model-free control methods can be employed. Classical model-free control methods are **PID!** (**PID!**) controllers (**?**). In recent years, data-driven control methods attract increasing attention. Especially, **RL!** (**RL!**) can be viewed as a model-free framework for solving optimal control problems, and has been successfully applied in various domains, including robotics, game playing, and autonomous systems.

We first introduce the terminology of **RL!**:

- agent: the controller,
- environment: the to-be-controlled system,
- action: the control input, a_t,

- state: the complete state of the environment, s_t,
- policy: the control strategy, which can be deterministic ($a_t = \mu(s_t)$) or probabilistic ($a_t \sim \pi(\cdot|s_t)$),
- reward: the feedback on the agent's success, $r_t = R(s_t, a_t)$.

The foundation of **RL!** is the **MDP!** (**MDP!**) (**?**). An **MDP!** is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P(s_{t+1}|s_t, a_t)$ is the state transition probability, $R(s_t, a_t)$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The interaction process between the agent and the environment at each time step t is as follows: the agent observes the current state s_t, selects an action a_t according to its policy $\pi(a_t|s_t)$, receives a reward $r_t = R(s_t, a_t)$ from the environment, and transitions to a new state s_{t+1} based on the state transition probability $P(s_{t+1}|s_t, a_t)$. The goal of the agent is to learn an optimal policy π^* that maximizes the expected return, i.e., the value function:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \tag{II.104}$$

In comparison to other optimal control methods, **RL!** methods aim to optimize long-term sequential decision-making performance. Besides, generality and scalability are the two main advantages of **RL!** methods. They can handle nonlinear, high-dimensional, and stochastic dynamics and also non-quadratic reward functions. The work by **?** (**?**) provides a comprehensive review of **RL!** for control.

It is worth noting that there is also an **RL!** method applying a surrogate model for the sake of improving the policy, which is called model-based **RL!** (**??**). The surrogate model is learned from data in advance. Then, the model can also be used for planning, simulation, or generating synthetic data to augment real-world data. In comparison to the model-free **RL!**, the model-based **RL!** can reduce the required amount of samples gathered directly from the environment. From the viewpoint of control, model-based **RL!** is also referred to learning-based control, when the surrogate model is also adaptive online. From the perspective of leveraging a state transition model, model-based **RL!** can be viewed as a hybrid approach that combines elements of both model-based and model-free control strategies.

RL! provides a flexible framework for learning control policies directly from data, enabling adaptation to changing environments and complex system dynamics. However, it often requires a large amount of data and may suffer from explainability, stability, and reliability issues. In safety-critical applications, model-based control methods are often preferred.

Chapter III

Literature Review

This chapter reviews the relevant literature on applications of deep probabilistic models in three areas: Bayesian parameter identification, Bayesian state estimation, and deep generative models as control policies. The deep probabilistic models considered here include **DGP!**s (**DGP!**s), **BNN!**s (**BNN!**s), **CVAE!**s (**CVAE!**s), **CNF!**s (**CNF!**s), conditional diffusion models, and flow matching models.

III.1 Bayesian Parameter Identification

III.1.1 Deep Gaussian Processes

DGP!s are sequential compositions of Gaussian processes, where the outputs of one Gaussian process are the inputs to the next. They extend the single-layer **GPR!** (**GPR!**) and can represent compositional or hierarchical structures among variables. The survey by **?** reviews the development of **DGP!**s, highlighting advances in model architectures, scalable inference methods, and approximation techniques that have significantly improved their practical applicability.

? (**?**) proposed a **DGP!**-based method for Bayesian inference of compositional parameters from **PDE!**s (**PDE!**s). The parameters are assigned a **DGP!** prior that models compositional functions. Then, the posterior distribution of the parameters is optimized via variational inference.

Applications of **DGP!**s for Bayesian inference span multiple domains. For instance, **?** (**?**) used **DGP!**s to identify contaminant release sources, while **?** (**?**) used them for air quality estimation.

In biomedical contexts, **?** (**?**) used **DGP!**s to assess patient survival under competing risks.

These studies show promising results compared with **GPR!**s as well as other machine learning methods, and highlight the versatility and effectiveness of **DGP!**s in addressing parameter inference challenges. However, computational and storage costs remain the main drawbacks of **DGP!**s, especially for high-dimensional data.

III.1.2 Bayesian Neural Networks

Applying **BNN!**s to Bayesian inference usually requires combining them with other methods to better extract information from observational data. For example, **?** (**?**) integrated a **BNN!** with **PINN!**s (**PINN!**s) (B-PINN) to infer parameters in **PDE!**s with Gaussian noise. **PINN!**s construct a neural network to approximate the forward solution of the underlying **ODE!** (**ODE!**) or **PDE!** problem by minimizing the data reconstruction loss while being regularized by the residual loss of the governing **ODE!** or **PDE!**. Meanwhile, model parameters can also be estimated probabilistically via **MCMC!** (**MCMC!**) or variational inference. The B-PINN achieved more accurate results under stronger observational noise.

? (**?**) used an autoregressive **BNN!** to estimate system parameters, together with a nonparametric noise model to capture complex noise characteristics in the data. The **BNN!** network parameters are approximated using an **HMC!** (**HMC!**) sampling approach. **BNN!**s can be flexibly integrated with other models and can provide uncertainty quantification, at the cost of higher computational complexity. However, applying **BNN!**s to Bayesian inference is not straightforward and must be combined with other architectures, such as CNN, RNN, or Transformer. Moreover, uncertainty in network weights does not necessarily translate to the uncertainty of the target variables.

III.1.3 Conditional Variational Autoencoders

CVAE!s represent a conditional extension of the **VAE!** (**VAE!**) framework. **CVAE!**s have been widely adopted for amortized Bayesian inference across various scientific domains. For example, **?** (**?**) introduced a **CVAE!**-based approach for efficient Bayesian inference in scientific simulators.

In astronomy, **CVAE!**s have enabled Bayesian parameter estimation from gravitational wave data, as demonstrated by **?** (**?**), **?** (**?**), and **?** (**?**). Additionally, **?** (**?**) applied **CVAE!**s to infer the equation of state of neutron stars.

In biomedical engineering, **CVAE!**s have been used for neuroimaging analysis under varying correlated parameters (**?**), and for parameter estimation in mouse cardiac muscle studies (**?**).

These examples highlight the versatility and effectiveness of **CVAE!**s for Bayesian inference with high-dimensional data. One shortcoming reported in (**?**) is that the **CVAE!** tends to estimate broader posterior distributions compared to other methods, indicating a potential limitation in capturing precise uncertainty.

III.1.4 Conditional Normalizing Flows

BayesFlow (Section **??**) (**?**) provides a general framework for amortized Bayesian inference using **CNF!**s. Beyond our work, it has also been applied to various scientific domains. For instance, **?** (**?**) applied BayesFlow in multiple cognitive models. **?** (**?**) demonstrated its effectiveness in damage detection in structural health monitoring. **?** (**?**) used BayesFlow to infer parameters from system dynamics, such as the random walk model.

Several other works share ideas similar to BayesFlow. **?** (**?**) referred to the summary network as an embedding neural network. **?** (**?**) extended the condition with richer synthetic data, which is generated by a **CVAE!**. **?** (**?**) constructed a **CNF!** by taking the conditioning variable as an additional input to each coupling layer and enlarging the latent space.

These works demonstrate the effectiveness of **CNF!**s for amortized Bayesian inference. In particular, the design of the summary network needs to be tailored to the specific characteristics of the data, which is crucial for extracting informative representations from observations and precisely inferring parameters.

III.1.5 Conditional Diffusion Models

Diffusion models have recently emerged as powerful tools for Bayesian inference due to their ability to model complex, multimodal distributions. **?** (**?**) introduced a diffusion-based framework for parameter identification in dynamic power systems, demonstrating improved accuracy over traditional methods.

Conditional diffusion models have also been applied to infer parameters, unobservable states, and system dynamics in **PDE!**s (**?**).

Furthermore, **?** (**?**) developed a framework for conditional score-based diffusion models in infinite-dimensional function spaces to perform Bayesian inference for linear inverse problems.

? (**?**) proposed a conditional diffusion model for amortized posterior estimation, showing superior performance compared to **CNF!**s, particularly in reconstructing multimodal distributions and capturing variable boundaries. Nevertheless, the computational cost of inference with diffusion models is much higher than that of other **DGM!**s (**DGM!**s) because of their iterative sampling procedure.

III.2 Bayesian State Estimation

After introducing versatile applications of deep probabilistic models for Bayesian parameter identification, this section focuses on applications of deep probabilistic approaches to state estimation, including data-assisted Bayesian filters and deep state-space models.

III.2.1 Data-Assisted Bayesian Filters

As discussed in Section **??**, estimating states from noisy observations strongly depends on the completeness of system models and strong assumptions about noise. Under model mismatches or incompleteness, deep probabilistic alternatives have been proposed to integrate Bayesian filters with offline data. Data-assisted Bayesian filters take advantage of rigorous state prediction and update steps, achieving better explainability and performance than purely data-driven methods.

For instance, **?** (**?**) introduced a Bayesian gated recurrent framework for non-Markov dynamical systems, which consists of a memory update gate for capturing temporal regularities in state evolution, a state prediction gate with evolution mismatch compensation, and a state update gate with observation mismatch compensation; it achieved stronger state estimation performance than standard **RNN!**s (**RNN!**s) and Transformer baselines.

? (**?**) proposed the Kalman **VAE!**, which combines a **VAE!** with a linear Gaussian state-space model. The **VAE!** encoder learns a low-dimensional latent representation of the observations, while the Kalman filter performs recursive Bayesian state estimation in the latent space. The decoder reconstructs the observations from filtered latent states, allowing uncertainty-aware tracking of temporal dynamics. Similar ideas have been explored in (**??**).

NF!s (**NF!**s) have also been combined with linear Gaussian state-space models. **?** (**?**) used a **NF!** to transform nonlinear models with non-Gaussian noise into latent space, where a Kalman filter is applied. The major advantage of **NF!**s compared with **VAE!**s lies in their invertibility, which allows efficient sampling of the target filtering distribution in the original space. The authors show that the flow-based Bayesian filter achieves better accuracy and computational efficiency compared to conventional filtering methods in challenging high-dimensional settings.

Another way of applying **NF!** is proposed by **?** (**?**). A **CNF!** is employed to construct flexible proposal distributions in differentiable particle filters. The **NF!** is conditioned on the observed data to migrate particles to areas closer to the true posterior. The integration of **CNF!**s improved accuracy in a visual tracking task.

III.2.2 Deep State-Space Models

Constructing the state transition and observation models in **SSM!**s (**SSM!**s) with neural networks enables the representation of a wide range of dynamic behaviors because of their flexibility and expressive capability. **?** provide a selective review of recent advancements in deep neural network-based approaches for **SSM!**s, and presents a unified perspective for discrete-time deep state-space models and continuous-time ones such as latent neural **ODE!**s and neural **SDE!**s (**SDE!**s). Recent advancements in sequence modeling have led to

the emergence of structured **SSM!**s as an efficient alternative to **RNN!**s and Transformers, addressing challenges in long-range dependency modeling and computational efficiency. A comprehensive survey of structured **SSM!**s is provided by **?** (**?**).

Similar to the work of **?** (**?**), leveraging **NF!**s to model noise in **SSM!**s also enables fast and accurate system dynamics learning, high-dimensional state estimation, and time-series forecasting. **?** (**?**) proposed an efficient transformed Gaussian process **SSM!** that integrates a single shared **GP!** (**GP!**) with input-dependent **NF!**s, yielding an expressive implicit process prior that captures complex, nonstationary transition dynamics while significantly reducing model complexity.

III.3 Deep Probabilistic Models and Time Series Modeling

Unlike Bayesian state estimation methods, deep probabilistic models offer a more flexible and general approach to handling uncertainty in time series data. Here, we place state estimation in a broader data-driven perspective by considering it as a time-series modeling task and reviewing combinations of deep probabilistic models with time-series modeling architectures, which provide inspiration for extending normalizing flows to state estimation tasks. Moreover, we summarize the applications of normalizing flows for modeling stochastic processes, which are closely related to our proposed method FlowSSM in Chapter **??**.

III.3.1 Integration of Deep Probabilistic Models with Time Series Modeling Architectures

Identifying nonlinear system dynamics solely from input-output data is challenging with time-series modeling methods alone. In contrast, deep probabilistic models, especially **DGM!**s, have shown great potential in capturing complex nonlinear relationships while providing uncertainty quantification. Combinations of these methods have been explored in recent years.

? (**?**) stacked **LSTM!** (**LSTM!**) layers with a probabilistic layer, obtaining a Bayesian inference model for nonlinear system identification, which captures nonlinear dynamics while providing uncertainty quantification.

? (**?**) introduced a framework integrating a **VAE!** with **NARX!** (**NARX!**) for identifying nonlinear systems using input-output data. The **VAE!** encodes the input-output data into a latent space, and the **NARX!** decodes the latent space information and reconstructs the system outputs. This approach effectively captures nonlinear relationships and improves robustness through the **VAE!** compared with **NARX!** alone.

? (**?**) proposed an amortized Bayesian inference method for learning hierarchical nonlinear dynamical systems. The method extends the **VAE!** by incorporating the

structural information of the system as hierarchical conditions. The encoder performs amortized inference over latent variables conditioned on observed trajectories and system structure. The decoder employs differentiable **ODE!**s to describe the system dynamics, which can include both known mechanistic components and learned neural components. This method enables the integration of knowledge about system structure with data-driven learning, improving interpretability and generalization.

III.3.2 Normalizing Flows and Stochastic Processes

Another viewpoint on probabilistic time-series modeling is to consider it as learning the underlying stochastic process that generates the data. In this section, we focus on the research on leveraging **NF!**s to model stochastic processes.

SCNF! (**SCNF!**) (**?**) treats **SDE!**s as random **ODE!**s. Random **ODE!**s are differential equations in which the vector field includes a stochastic process that is approximated by deterministic functions. Random **ODE!**s are trained in the manner of continuous normalizing flows. However, **SCNF!** suffers from the overfitting issue of the vanishing diffusion term.

CTFP! (**CTFP!**) (**?**) models stochastic processes by transforming latent Wiener processes via continuous normalizing flows. The continuous normalizing flows are trained in the forward direction by mapping data samples to latent variables. Time-Changed Normalizing Flow (**?**) extends the base Wiener process in **CTFP!** with a time function. Continuous Latent Process Flow (**?**), as an improvement of **CTFP!**, applies a LatentSDE (**?**) as the encoder to generate latent variables, and a continuous normalizing flow as the decoder. Although these methods have shown good performance in generating probabilistic time series, they are computationally expensive and unable to solve initial-value problems directly or model dynamical systems with control inputs, making them unsuitable for many engineering applications.

Although several works have demonstrated the potential of normalizing flows for modeling stochastic processes, their application to system dynamics modeling and state estimation remains limited. Key challenges include high computational cost, difficulty handling control inputs and initial-value problems, and risks of overfitting or vanishing diffusion. Addressing these issues represents a promising direction for our research.

III.4 Deep Generative Models as Control Policies

Moreover, deep generative approaches for capturing input-output relationships are well suited for integration into model-free **RL!** (**RL!**) frameworks. The modeled uncertainty plays a crucial role in enabling robust decision-making. A survey (**?**) provides a comprehen-

sive overview of recent research efforts focused on the application of diffusion models within the domain of reinforcement learning. Diffusion models demonstrate significant flexibility and capacity as general-purpose tools for planning, policy generation, and data synthesis. The recent convergence with powerful large language models (LLMs) has significantly enhanced this capability. This integration unlocks the multimodal strength of diffusion models, enabling them to tackle diverse and complex tasks in robotics (**?**).

However, in specific single-task scenarios, **NF!**s may prove more suitable. **NF!**s often offer a superior balance between computational cost and the strength of accurate density estimation. This is particularly required in domains like physics-based modeling, where the underlying noise distribution is often less complex than in general image/language generation tasks. In particular, **?** (**?**) argued that the probabilistic modeling and inference capabilities of **NF!**s are better suited to **RL!** frameworks than diffusion models, including when acting as a policy, as a Q-function or value model, or for modeling state densities.

In the following, we summarize different **RL!** methods equipped with **NF!** or **FM!** (**FM!**).

Table III.1: Normalizing Flows in Reinforcement Learning

Flow Model	**Role in RL**	**Main Contribution**
Discrete NF (**??**)	Policy of SAC	Improved expressiveness of the policy distribution and efficient exploration under sparse rewards.
Neural Spline Flow (**?**)	Visuomotor Policy	Comparable performance to Diffusion Policy and improved sample efficiency in training while being up to 30x faster in inference.
Neural Spline Flow (**?**)	Expert policy in inverse RL	Good recovery of the true reward signal and imitation from observation.
Causal NF (**?**)	Learning transition dynamics and reward function in offline policy training	Improved generalization in sequential decision-making tasks, revealing a high potential for OoD adaptation in offline RL.
Coupling Flow (**??**)	Diffeomorphic transformation from state to action (policy)	Application in complex continuous control tasks while ensuring stability.

Continued on next page...

Table III.1: Normalizing Flows in RL (Continuation)

Flow Model	Role in RL	Main Contribution
Coupling Flow (**?**)	Constrained policy of SAC	Ensuring constraint satisfaction without reward engineering while offering enhanced interpretability.
Coupling Flow (**?**)	Policy in VLA models	Comparable performance to Diffusion Policy and improved parameter efficiency while being faster in inference.
RealNVP (**?**)	Policy of DDPG	Significant reduction of constraint violations.
RealNVP (**?**)	Learning control policy in MPC	FlowMPPI with projection outperforms state-of-the-art MPC baselines on both in-distribution and OoD environments.
RealNVP (**?**)	Policy of DAC	Enabling imitation of complex continuous control policies.
Autoregressive Flow (**?**)	Motion planner in IL	Achieving good performance in real-world autonomous driving tasks.
RealNVP (**?**)	Policy representation	Capturing multimodal action distributions.
RealNVP (**?**)	Policy gradient in model-based RL	Capturing multimodal action distributions.
MAF and Glow (**?**)	Learning dynamics model	Improved representation of stochastic dynamics while ensuring stability.
MAF (**?**)	Distribution matching in IL	Better capture of the state-action distribution.
FM (**?**)	Augmenting data	Achieving stable convergence in model-based RL.
FM (**?**)	Generating trajectories	Outperforming other baselines in continuous and discrete environments with stochastic dynamics.
FM (**?**)	Optimal behavior-constrained policy in online RL	Regularization of learned policy through an implicit guidance mechanism; achieving stable training and improved sample efficiency.
FM (**?**)	Policy of SAC	Improved performance and sample efficiency in continuous control tasks.

Continued on next page...

Table III.1: Normalizing Flows in RL (Continuation)

Flow Model	**Role in RL**	**Main Contribution**
FM (**?**)	Estimating dynamics gap	Improved generalization for learning online dynamics; outperforming state-of-the-art RL baselines with shifted dynamics.
FM (**?**)	Policy in IL	Fast policy generation based on 3D vision; achieving improved success rates and faster inference time for robotic manipulation.
FM (**?**)	Goal-conditioned policy	Estimating distributional bounds.
FM (**?**)	Policy in PGRL	Capturing multimodal action distributions in continuous control tasks.
FM (**?**)	Student & teacher policy in IL	Improved generalization of learned policy and the robustness against suboptimal expert data.
FM (**?**)	Modeling value distribution (Critic)	Reduction of gradient variance.
Energy-based NF (**?**)	Policy of Maximum Entropy RL	Integration of policy evaluation steps with policy improvement steps, resulting in a single objective training process.
Energy-guided FM (**?**)	Policy for offline RL	Competitive performance and constant training time in flow sampling steps.
Energy-guided FM (**?**)	Policy in offline RL	Faster inference.
Rectified Flow (FM) (**?**)	Policy in online RL	Surpassing diffusion policy in continuous robotic control.

Chapter IV

Methodology and Numerical Validation

IV.1 Parameter Identification

As introduced in Section **??**, various deep probabilistic approaches have been proposed for Bayesian parameter identification. In many practical applications, the available observations are multimodal, heterogeneous, and high-dimensional, leading to posterior distributions that exhibit strong nonlinear dependencies, multimodality, and complex uncertainty structures. Deep generative models provide a flexible framework for learning such distributions directly from data while preserving a probabilistic interpretation. Furthermore, through amortized inference, they enable efficient posterior estimation for new observations without repeatedly solving computationally expensive Bayesian inference problems. For these reasons, deep generative models have emerged as a powerful approach for scalable Bayesian parameter identification. We therefore introduce a universal framework based on deep generative models for amortized Bayesian inference. This framework can be viewed as a generalization of BayesFlow (**?**).

IV.1.1 Universal Framework with Deep Generative Models for Amortized Bayesian Inference

? (**?**) proposed BayesFlow as an amortized Bayesian inference approach with normalizing flows. Amortized inference means to train a shared inference network that can be used to approximate the posterior distribution for new observations, instead of solving the inference problem from scratch for each new observation. BayesFlow consists of a summary network for extracting information from the observations, and a conditional normalizing flow for learning the posterior distribution of parameters conditioned on the extracted information. This framework can be generalized to other deep generative models, such as **VAE!** (**VAE!**), **FM!** (**FM!**), and diffusion models. Figure **??** illustrates the framework. The accompanying table (Table **??**) summarizes how different **DGM!**s (**DGM!**s) differ in their training objectives and inference mechanisms. Algorithm **??** details the two-stage procedure: amortized learning from data followed by posterior sampling given new observations. We call the summary network here the condition network to emphasize that the embeddings of observations function as condition in the **DGM!**s. The major advantages of applying deep generative models as the Bayesian inference backbone are

threefold:

- **DGM!**s consider the parameter identification as posterior inference, which naturally provides trustworthiness of the estimates;
- **DGM!**s seek to model the posterior distribution on a large dataset, which is suitable for amortized Bayesian inference;
- compared with other architectures that learn only a direct input-output mapping, **DGM!**s learn both an inference process from data to latent variables and a generative process from latent variables back to data. This separate modeling enables **DGM!**s to learn expressive representations and capture complex data distributions.

In many classical generative tasks, such as text, image, and video generation, the condition information relatively simple, often represented by a low-dimensional feature vector or class label. In contrast, for solving inverse problems, such as parameter identification, the generative model is supposed to generate variables under some more complex conditions. Thus, a well-designed condition network is essential to extract most informative features from the observations. This universal framework automatically encourages the condition network to deliver desirable embeddings to the deep generative model. Besides, each observation is only required to be processed once while sampling the trained generative model for multiple times, which is computationally efficient.

IV.1.2 Normalizing Flows for Parameter Identification

Why are **NF!**s (**NF!**s) more suitable for parameter identification tasks given time series observations? In this section, we will showcase the benefits of **NF!**s in terms of

① Inference accuracy and precision,

② Reliability, i.e., whether the ground truth lies in the posterior,

③ Speed, i.e., inference computational time,

④ Sample efficiency, i.e., posterior contraction while increasing data amount,

⑤ Correlation, i.e., identifying linearly correlated parameters,

⑥ Complexity, i.e., handling multimodal distributions.

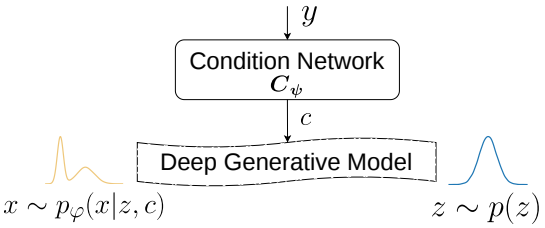


Figure IV.1: Universal Framework of Using Deep Generative Models for Amortized Bayesian Inference. The corresponding algorithm is shown on the right (Algorithm **??**).

Model	Training Objective	Inference
VAE	Maximize ELBO	Amortized encoder
NF	Maximize log-likelihood	Direct deterministic mapping
FM	Minimize matching cost (vector field)	Integrate ODE
SBM	Minimize matching cost (score function)	Integrate SDE
DDPM	Minimize matching cost (added noise)	Integrate SDE

Table IV.1: Training Objectives and Inference Across **DGM!**s.

Algorithm 1 DGM-ABI Training & Inference

Require: Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{M}$
Require: Prior $p(z)$, Condition network C_ψ
Require: Generator $p_\varphi(x|z,c)$, (Optional: Inference $q_\phi(z|x,c)$)

Stage 1: Amortized Learning
while not converged **do**
  Sample batch $\{(x,y)\} \sim \mathcal{D}$
  Compute embeddings: $c \leftarrow C_\psi(y)$
  Sample latent: $z \sim p(z)$
  Compute objective gradient:
  $g \leftarrow \nabla_{\varphi,\psi,\phi} \mathcal{L}$
  Update parameters: $(\varphi, \psi, \phi) \leftarrow (\varphi, \psi, \phi) - \eta \cdot g$
end while
Stage 2: Bayesian Inference
Given observed condition y_{obs}:
$c_{\text{obs}} \leftarrow C_\psi(y_{\text{obs}})$
for $k = 1$ to K **do**
  Sample $z^{(k)} \sim p(z)$
  Generate $x^{(k)} \sim p_\varphi(x|z^{(k)}, c_{\text{obs}})$
end for
return Posterior samples $\{x^{(k)}\}_{k=1}^{K}$

IV.1.2.1 Overall Settings

In this section, we take a classic mass-spring-damper system as an example, which has three parameters including mass m, spring constant k, and damping coefficient c. As observations, we take the position and velocity trajectories. For evaluation of ① - ④, an external force is introduced. The **ODE!** (**ODE!**) is:

$$\dot{x} = v, \ \dot{v} = \frac{1}{m}(F_t - kx - cv), \ F_t = 10 \cdot \sin(3t). \tag{IV.1}$$

For the ⑤ linear correlation case, the external force is removed, and the parameters m and k are correlated, i.e., $k = 10 \cdot m$.

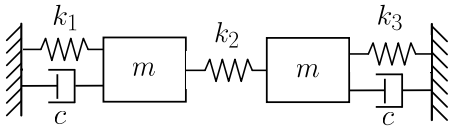


Figure IV.2: Mass-Spring-Damper System with Multimodal Parameters. The two spring constants k_1 and k_3 are exchangeable, leading to a multimodal posterior distribution.

To investigate the ⑥ multimodal case, the system is extended into a two-mass-three-spring topology, as shown in Figure **??**. When the parameters m, k_2, and c are fixed as constant, and we take the middle points of the positions and velocities of the two masses as observations, the value of k_1 and k_3 cannot be uniquely distinguished. The **ODE!** becomes

$$\dot{x}_1 = v_1,\ \dot{v}_1 = \frac{1}{m}(-k_1 x_1 + k_2(x_2 - x_1) - c v_1); \tag{IV.2}$$

$$\dot{x}_2 = v_2,\ \dot{v}_2 = \frac{1}{m}(-k_3 x_2 - k_2(x_2 - x_1) - c v_2). \tag{IV.3}$$

The two-mass-three-spring topology is designed to expose a fundamental limitation of Bayesian inference: structural symmetry leading to multimodal posteriors. In this configuration, the symmetric coupling of the two outer springs (k_1 and k_3) to identical masses creates an indistinguishability: swapping the values of k_1 and k_3 produces identical observational data at the interior measurement point. Testing multimodal recovery is essential for assessing whether deep generative models can capture the full posterior complexity rather than collapsing to spurious point estimates.

We compare two normalizing flow architectures **RealNVP!** (**RealNVP!**) and **NSF!** (**NSF!**) with **FM!**, **DDPM!** (**DDPM!**), and **SBM!** (**SBM!**), as well as **VAE!** and **BNN!** (**BNN!**). As the condition network, we use an **LSTM!** (**LSTM!**) layer followed by two linear layers with **ELU!** (**ELU!**) activations. The evaluation is conducted on a test set of 200 samples, with each sample estimated through 100 posterior samples generated by the model. For a group of estimated $\{\widehat{x}^{(m)}\}_{m=1}^{M}$ and true parameters $\{x^{(m)}\}_{m=1}^{M}$ with the mean of the true parameters $\overline{x}$, the following evaluation metrics are applied:

- Accuracy: **RMSE!** (**RMSE!**) and R^2 score:

$$RMSE = \sqrt{\frac{1}{M}\sum_{m=1}^{M}(x^{(m)} - \widehat{x}^{(m)})^2}\,, \tag{IV.4}$$

$$R^2 = 1 - \sum_{m=1}^{M}\frac{(x^{(m)} - \widehat{x}^{(m)})^2}{(x^{(m)} - \overline{x})^2}\,. \tag{IV.5}$$

- Precision: **MCIW!** (**MCIW!**). For the m-th parameter by sampling it L times, the

standard deviation is $\sigma_{\widehat{x}^{(m)}}$, and we have:

$$CI^{(m)}(95\,\%) = [L_m, U_m] = \widehat{x}^{(m)} \pm 1.96\frac{\sigma_{\widehat{x}^{(m)}}}{\sqrt{L}}. \tag{IV.6}$$

Then the MCIW can be obtained by:

$$MCIW = \frac{1}{M}\sum_{m=1}^{M}(U_m - L_m). \tag{IV.7}$$

- Reliability: **CICP!** (**CICP!**). The larger **CICP!** indicates higher reliability of estimation.

$$CICP = \frac{1}{M}\sum_{m=1}^{M} c_m,\ c_m = \begin{cases} 0 & \text{if } x^{(m)} \notin [L_m, U_m], \\ 1 & \text{otherwise.} \end{cases} \tag{IV.8}$$

IV.1.2.2 Performance Comparison

Table IV.2: Performance Benchmarking of Deep Bayesian Inference Methods on the Mass-Spring-Damper System. All metrics represent averages across the three system parameters of 200 test samples. Predictive accuracy is assessed via RMSE (lower is better) and R^2 (higher is better). Uncertainty quantification is evaluated through the CICP and MCIW together. Ideal performance is characterized by a CICP near the 95 % calibration target; significantly lower values indicate overconfidence, while excessively high MCIW values indicate model imprecision. Inference time denotes the total computational time for a single test scenario based on 100 posterior samples.

Model	**RMSE** ↓	R^2 ↑	**CICP** ↑	**MCIW** ↓	**Inference Time** (s) ↓
BNN	0.0393	0.9965	0.9900	0.1874	0.0209
VAE	0.0287	**0.9987**	0.7250	**0.0497**	**0.0018**
DDPM	0.0915	0.9852	0.9883	0.5338	0.2310
SBM	0.0699	0.9909	0.9700	0.3289	0.0534
FM	0.0547	0.9938	0.9933	0.3730	0.0050
RealNVP	**0.0278**	0.9986	**0.9917**	**0.1223**	0.0043
NSF	0.0401	0.9966	0.9917	0.1845	0.0069

In Table **??**, the evaluation of different models is shown. The observed time series are from 0 s to 1 s with the time step of 0.05 s, and no noises are added onto them. In terms of accuracy, **RealNVP!** (RMSE: 0.0278) and VAE (RMSE: 0.0287) are the most accurate models. Their highest R^2 scores indicate that they explain almost all the variance in the mass-spring-damper system. **DDPM!** (RMSE: 0.0915) has the highest error in this group. While an R^2 of 0.985 is still objectively good, it is significantly less precise

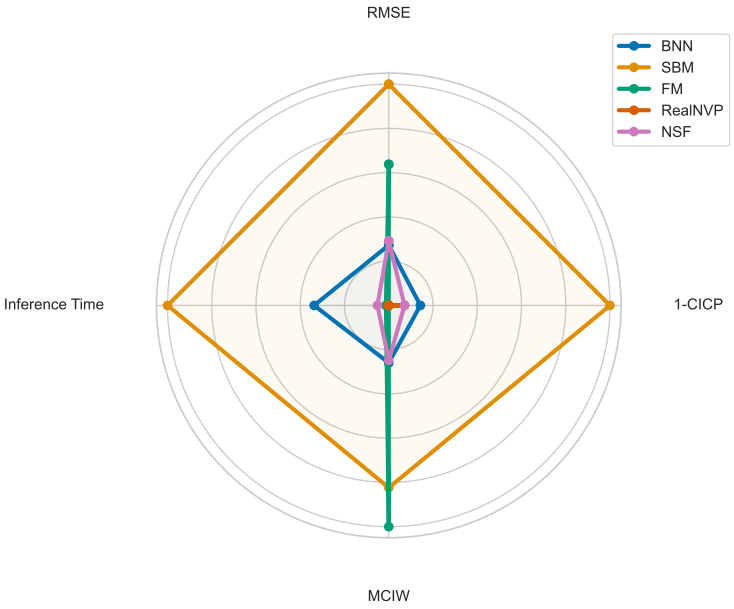


Figure IV.3: Detailed Comparison of Deep Bayesian Inference Methods on the Mass-Spring-Damper System. The values are the same as those in Table **??**. 1-CICP is shown for the sake of consistency of comparison, meaning that for all metrics in the spider chart, lower is better. The center of the radar plot represents optimal performance.

than the **NF!**s (**RealNVP!**, **NSF!**) for this specific physical task. Overall, the flow-based methods (**RealNVP!**, **NSF!**, and **FM!**) achieve higher accuracy than diffusion-based methods (**DDPM!** and **SBM!**). The **BNN!** is comparable with the **NF!**s.

In terms of uncertainty quantification, almost all models (except VAE) show CICP values near 1.0. Theoretically, the ideal CICP with 95 % confidence intervals should also be 0.95. Higher CICPs mean that they are conservative and under-confident, whereas the lower CICPs indicate overfitting. Besides, in an ideal noise-free scenario, the posterior distribution for the mass-spring-damper parameters should approach a Dirac delta function. However, in the practice of solving inverse problems, the interval should never be zero, because different parameters might produce very similar trajectories. Moreover, the models themselves have uncertainty due to limited data. Therefore, the CICPs higher than 0.95 mean that the inference models prioritize the reliability over precision. Consequently, the MCIW should be considered together with CICP. **VAE!** (CICP: 0.7250 and MCIW: 0.0497) is overconfident. Its intervals are so narrow that the truth falls outside of them 27.5 % of the cases. **DDPM!** (CICP: 0.9883 and MCIW: 0.5338) is imprecise. It achieves high coverage simply by producing very wide intervals.

In terms of computational efficiency, **VAE!** exhibits the highest inference speed. Flow-based methods demonstrate comparable performance to one another, though they are approximately 5× slower than the **VAE!** baseline. **BNN!** follows, showing higher latency than the flow-based models but maintaining a clear advantage over **SBM!**. Finally,

DDPM! is identified as the most computationally intensive approach, recording the slowest inference times.

With **VAE!** and **DDPM!** excluded due to miscalibration and computational overhead respectively, Figure **??** visualizes the performance trade-offs of the remaining five architectures. **RealNVP!** exhibits the most optimal profile, represented by the smallest polygon area; it successfully minimizes **RMSE!** and **MCIW!** while maintaining near-perfect coverage. **NSF!** and **FM!** show comparable speed and accuracy but yield less precise (wider) credible intervals. The **BNN!** maintains high reliability (low 1-CICP) but suffers from increased inference time. Finally, **SBM!** shows the least favorable performance, characterized by the largest predictive error, the widest uncertainty bounds, and the longest computational time, indicating a difficulty in parameter identification compared to flow-based alternatives. To sum up, **RealNVP!** emerges as the winner for overall performance comparison.

IV.1.2.3 Sample Efficiency

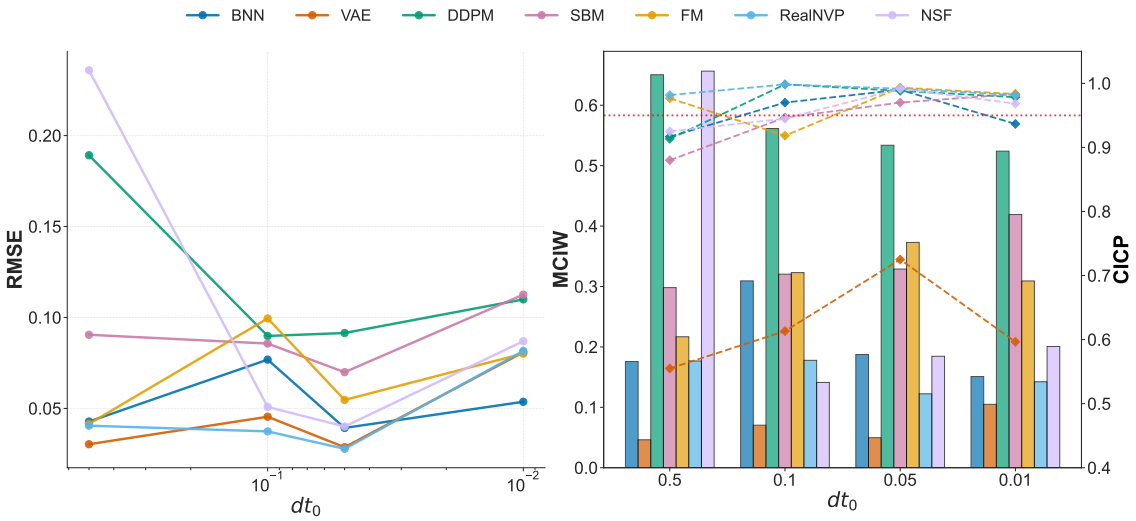


Figure IV.4: Sample Efficiency Comparison of Deep Bayesian Inference Methods under Varying Observation Densities. The left panel illustrates the predictive accuracy (RMSE) across time step $dt_0 = [0.5\,\text{s},\ 0.1\,\text{s},\ 0.05\,\text{s}$ and $0.01\,\text{s}]$ plotted on a logarithmic scale where smaller dt_0 represents higher sampling density. The right panel provides a hybrid uncertainty analysis: the grouped bars indicate the MCIW (uncertainty precision), while the superimposed dashed lines represent the CICP (calibration reliability). The horizontal dotted line denotes the 95 % coverage target.

Figure **??** shows how observation density affects posterior estimation. As dt_0 de-

creases from 0.5 s to 0.05 s, all models show consistent improvement in predictive accuracy, attributed to improved identifiability of the system dynamics. Accuracy degrades at $dt_0 = 0.01$ s—likely because the encoder is not designed for long time series. Adding more points adds redundancy without new constraints; this can induce overfitting in the latent space, where the model interprets numerical rounding errors or minor architectural biases as significant physical features. For this system, 0.05 s is the optimal resolution. **BNN!**, **RealNVP!**, and **VAE!** are less sensitive to sample efficiency; diffusion-based methods and **NSF!** require more data. While almost all models reach their accuracy peak at $dt_0 = 0.05$ s, **FM!** favors lower resolution, possibly because vector-field learning benefits from larger differences between data samples.

Regarding uncertainty quantification, **VAE!** is overfitted with narrow intervals and inadequate coverage. As data density increases, **DDPM!** and **RealNVP!** show consistent posterior shrinkage—as the likelihood becomes more peaked, the posterior contracts around the true parameters. Other architectures exhibit more volatile trends: the MCIW of **BNN!** initially increases before dropping; **NSF!** shows a decrease-then-rise pattern; **FM!** and **SBM!** show paradoxical MCIW increases, as the longer sequences introduce numerical complexity or perceived contradictions in the latent space that temporarily outweigh the additional information. The reliability requirement (**CICP!** ≥ 0.95) is hardest under sparse sampling. At $dt_0 = 0.5$ s, only **RealNVP!** and **FM!** are calibrated, highlighting their robustness for sparse trajectories. As density improves to $dt_0 = 0.1$ s, **FM!** temporarily drops out of the reliable regime (its MCIW rise not aligned with predictive accuracy), while **SBM!** and **NSF!** reach the target via distinct mechanisms—**SBM!** through corrective expansion of posterior variance, **NSF!** through efficient shrinkage. At $dt_0 = 0.05$ s, all models satisfy the requirement; 0.05 s provides sufficient information without numerical saturation.

At the highest density ($dt_0 = 0.01$ s), models break down. **DDPM!** and **BNN!** continue to shrink MCIW; for **BNN!**, intervals shrink faster than error, causing it to miss the ground truth and become overconfident. Conversely, **SBM!**, **RealNVP!**, and **NSF!** expand MCIW defensively, as the numerical difficulty of processing high-frequency noise-free data broadens uncertainty bounds to maintain coverage. **FM!** reverses its trend and eventually finds a more stable numerical representation than the other flow variants. In summary, more observations do not always improve performance. **RealNVP!** is most precise under sparse data (excluding **VAE!**); **BNN!** shrinks bounds consistently but its variance may reflect weight convergence rather than physical correctness. Notably, at $dt_0 = 0.1$ s, **SBM!** and **NSF!** define the boundary of the reliable regime, whereas **BNN!** and **DDPM!** remain under-calibrated despite increased data density. Overall, **RealNVP!** demonstrates superior sample efficiency across varying observation densities.

IV.1.2.4 Linear Correlation

When parameters are linearly correlated, the observations only constrain their ratio, rendering the individual parameters non-identifiable. While **DGM!**s are capable of capturing these dependencies, the task is computationally harder than identifiable ones. The ill-posedness manifests as a manifold of equivalent solutions, making it difficult for optimization algorithms to distinguish between physically distinct systems that produce identical observations. When the correlation between m and k are fixed to be $k = 10 \cdot m$, the damping coefficient is also proportional to the mass, i.e., $c \propto m$. Because the damping coefficient c is related to the damping ratio ζ (which determines if the system is under-damped, critically damped, or over-damped) by the formula:

$$c = 2\zeta\sqrt{mk}. \tag{IV.9}$$

Therefore, the following relation holds:

$$c/m = 2\sqrt{10}\,\zeta. \tag{IV.10}$$

Figure **??** shows two identification examples of linear correlated parameters.

As illustrated in Figure **??**, the **BNN!** and **DDPM!** fail to capture the underlying dependencies, resulting in wide, uninformative posterior clouds that lack alignment with the physical constraints. While the **VAE!** recovers the k/m correlation, it fails significantly on c/m, where the posterior mass is biased and poorly aligned with the diagonal line. In contrast, the remaining four methods demonstrate higher fidelity.

Among the successful models, **SBM!** and the **NF!** architectures (**RealNVP!** and **NSF!**) show their superior capability in identifying and enforcing linear parameter correlations. Their k/m estimates are tightly aligned with the theoretical diagonal, and their c/m estimates manifest as narrow ellipses centered on the ground truth. On the other hand, **FM!** successfully captures the linear trend, but it exhibits noticeably higher variance compared to **SBM!**, **RealNVP!**, and **NSF!**, particularly in the c/m correlations. This suggests that while **FM!** identifies the correlation, it is less precise in enforcing the constraint than other architectures.

Linear correlations represent a low-dimensional manifold within the parameter space. Figure **??** demonstrates that **NF!**s (**RealNVP!**/**NSF!**) and **SBM!** are superior at manifold learning by effectively collapsing the posterior mass onto a line, whereas **BNN!** and **DDPM!** struggle to resolve such sharp constraints. Besides, the **VAE!**'s failure in the c/m correlation is a classic symptom of posterior collapse or architectural under-fitting for specific parameter dimensions.

The performance gap between the two diffusion models is particularly striking: **SBM!**

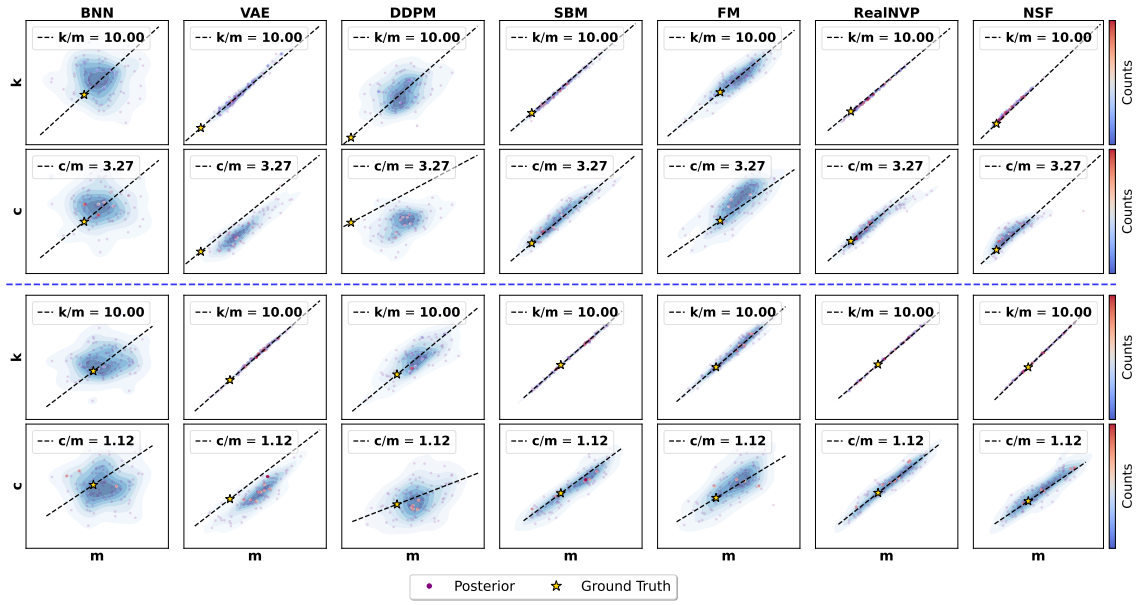


Figure IV.5: Comparative analysis of deep Bayesian inference architectures in identifying linearly correlated parameters. The 4×7 grid displays the joint posterior distributions for seven different models (columns) across two test scenarios (separated by a horizontal blue dashed line in the middle). Rows 1 and 3 visualize the $m - k$ correlation, while Rows 2 and 4 visualize the $m - c$ correlation. Within each sub-figure, the gold star represents the ground truth, and the black dashed line illustrates the theoretical linear constraint (k/m or c/m). Individual posterior samples are plotted as purple points, overlaid on a logarithmic density heatmap where warm hues indicate regions of high probability mass. Note: To ensure high visibility of the posterior topology across models with varying precision, axis limits and density color-scales are individually optimized for each sub-figure. Each color map suits the whole row.

exhibits a far greater capability for recovering precise physical constraints than **DDPM!**. Despite **DDPM!**'s established success in generating high-dimensional complex data, it fails to resolve the specific linear parameter correlations in this task. Similarly, despite their theoretical proximity, **FM!** appears more dispersed than **SBM!**. This is primarily because **FM!** is a deterministic transport method that is more sensitive to cumulative approximation errors along the integration path, whereas the stochastic nature of **SBM!** provides a corrective mechanism through noise-driven sampling.

Within the **NF!** architectures, a notable distinction exists between **RealNVP!** and **NSF!** when we zoom in on the c/m figures. While both successfully identify the correlation, **RealNVP!** exhibits a higher degree of centralization along the diagonal correlation axes. While this leads to a narrower, line-like distribution with high precision, it also places the

ground-truth (yellow star) at the extreme periphery of the high-density core. In contrast, **NSF!** estimates the density with higher variance, which is less desirable in this case. In summary, **SBM!**, **RealNVP!**, and **NSF!** emerge as the winners for linear correlation identification, demonstrating superior capability in capturing low-dimensional manifold structures.

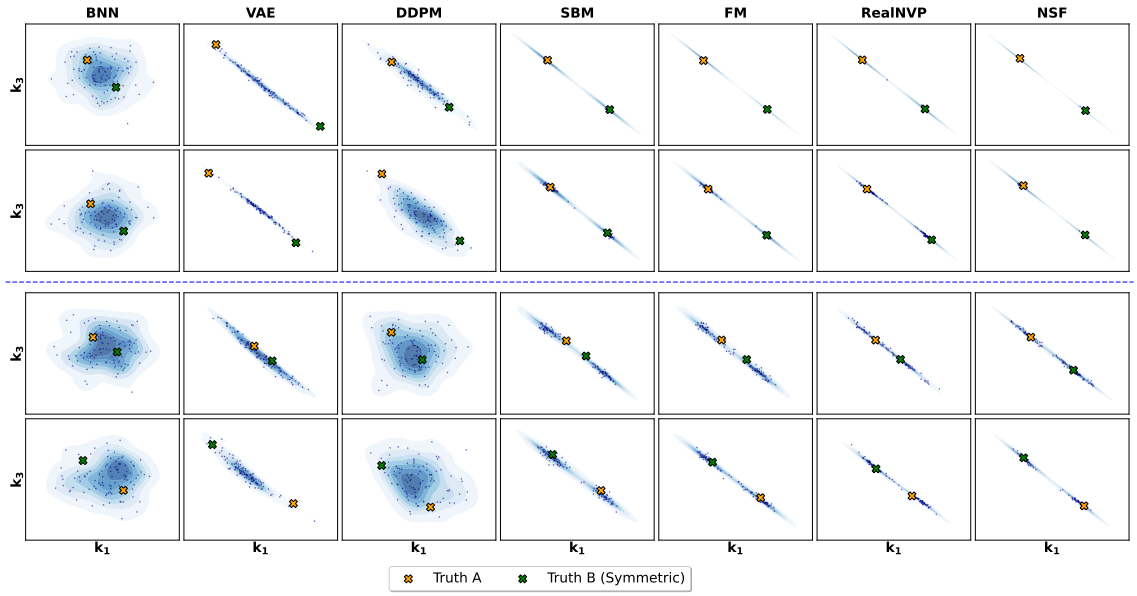


Figure IV.6: Comparative evaluation of deep Bayesian architectures in recovering multimodal posterior distributions. The 4×7 grid displays the joint posterior of k_1 and k_3 for seven different models (columns) across four distinct test scenarios (rows). The top two rows represent scenarios with well-separated modes, while the bottom two rows represent scenarios with proximal (overlapping) modes (partitioned by a blue dashed line). Within each subplot, the orange cross denotes the target ground truth, while the green cross represents the symmetric ground truth (swapped k_1 and k_3 values), highlighting the models' ability to capture the inherent exchangeable symmetry of the parameter space. Blue contours represent the estimated posterior density. Note: To ensure high visibility of the posterior topology across models with varying precision, axis limits and density color-scales are individually optimized for each sub-figure.

IV.1.2.5 Multimodal Distribution

The ability to capture multimodal distributions is a key advantage of applying **DGM!**s for Bayesian inference. Different **DGM!**s exhibit different characteristics. Figure **??** illustrates these differences. As test scenarios, we consider two cases: one where the values of k_1 and k_3 are far apart (the first two samples), and another where they are close to each other

(the last two samples).

As demonstrated in Figure **??**, the performance is consistent with the linear correlation results. **SBM!**, **FM!**, **NSF!**, and **RealNVP!** significantly outperform **BNN!**, **VAE!**, and **DDPM!** in resolving the symmetric multimodal structure. The **BNN!** produces broad, uninformative posterior clouds that fail to recognize any specific multimodal topology. The **VAE!** exhibits over-concentration onto narrow, biased ridges; in all these four scenarios, the ground truth and its symmetric counterpart lie entirely outside the high-density regions, indicating the posterior collapse again. **DDPM!** successfully resolves the two distinct modes only in the first scenario (Row 1), but degrades into a single diffuse, biased cluster in the remaining test cases.

In contrast, **SBM!** and the three **NF!** architectures (**RealNVP!**, **NSF!**, and **FM!**) demonstrate a superior capacity for multimodal recovery. In well-separated scenarios (top two rows), all architectures successfully resolve two distinct high-probability modes. Notably, however, **RealNVP!** exhibits a slight bias, failing to precisely align its right posterior mode with the ground truth. As the modes become proximal (bottom two rows), the identification task becomes increasingly ill-posed, with models struggling to decouple the two ground-truth points and producing a higher density of samples in the interstitial region. While **NSF!** maintains high fidelity by successfully resolving both modes and accurately centering the posterior on the ground-truth coordinates, the remaining three models exhibit varying degrees of dislocation. This suggests that while most architectures can handle discrete multimodality, only **NSF!** remains robust in proximal scenarios where structural symmetry leads to significant parametric overlap. Consequently, **NSF!** emerges as the winner for multimodal distribution recovery.

IV.1.2.6 Characteristics Comparison

Table **??** summarizes the characteristics of the five **DGM!** families in Table **??** based on the performance, sample efficiency, linear correlation, and multimodal distribution experiments.

IV.1.3 Relationship Between Bayesian Inference and Sensitivity Analysis

Previous experiments applying mass-spring-damper models demonstrate that Bayesian inference is intrinsically linked to parameter identifiability. Our findings indicate that spring constants converge rapidly during training, which will be further elucidated through sensitivity analysis. Conversely, linearly correlated parameters lack unique identification, and in symmetric coupled models, individual spring constants become indistinguishable. These varying levels of identifiability can be rigorously analyzed through **SA!** (**SA!**), which studies how uncertainty in the output of a mathematical model can be partitioned and

Method	Performance	Sample efficiency	Linearity	Multimodality
VAE!	Fast; accurate mean, but under-calibrated.	Stable with data amount, but overconfident.	Partial trend; posterior collapse in some dimensions.	Weak; biased ridges and missed symmetric modes.
NF!	Best accuracy–calibration–cost balance; **RealNVP!** strongest standard case.	Best, especially **RealNVP!**; robust with sparse data.	**RealNVP!**/**NSF!** sharply recover the constraint manifold.	Strong; **NSF!** best for close or overlapping modes.
FM!	Competitive, but posteriors are more dispersed.	Robust under sparse and dense observations.	Correct trend, less sharp than **NF!**s/**SBM!**.	Good for separated modes; weaker with overlap.
SBM!	Costly with wider uncertainty; weaker standard benchmark.	Data- and compute-intensive; useful for structured posteriors.	Strong manifold recovery through score correction.	Strong, but less precise than **NSF!** in proximal cases.
DDPM!	Weakest; largest error and high inference cost.	Poor; iterative denoising is inefficient.	Diffuse posteriors; weak sharp-correlation recovery.	Limited; only easiest separated modes are resolved.

Table IV.3: Characteristics comparison of the five **DGM!** families in Table **??** based on the performance, sample-efficiency, linear-correlation, and multimodal-distribution experiments.

allocated to different sources of uncertainty in its inputs. This framework aligns naturally with the objectives of amortized Bayesian inference. For purely data-driven methods, **SA!** provides essential insights into the physical model and practical training suggestions: weakly identifiable or correlated parameters that require additional observational data for resolution.

Unlike local **SA!**, which examines instantaneous changes in output relative to specific parameter perturbations, our research focuses on total trajectories, necessitating a global **SA!**.

IV.1.3.1 Sobol' Index

A common approach for global **SA!** is the Sobol' index, which is a variance-based method that decomposes the total output variance into contributions from individual parameters and their interactions. For a model f with inputs $X_1, \ldots, X_d$ and the output Y, V_i represents the variance contribution of the i-th input, V_{ij} represents the interaction effect between inputs X_i and X_j, and so on. The symbol $\boldsymbol{X}_{\setminus i}$ denotes the vector of all input variables except X_i, i.e. $(X_1, \ldots, X_{i-1}, X_{i+1}, \ldots, X_d)$.

The total variance of the model output is then:

$$\mathrm{Var}(Y) = \sum_i V_i + \sum_{i<j} V_{ij} + \cdots + V_{1,2,\ldots,d}, \tag{IV.11}$$

where V_i is given by:

$$V_i = \mathrm{Var}_{X_i}\left[\mathbb{E}_{\boldsymbol{X}_{\setminus i}}(Y|X_i)\right]. \tag{IV.12}$$

The inner expectation can be interpreted as an average response of the model when X_i is fixed, while the outer variance measures how much this average response varies as X_i changes. Dividing the variance contribution of each parameter by the total variance yields the first-order Sobol' indice:

$$S_i^1 = \frac{V_i}{\mathrm{Var}(Y)}, \tag{IV.13}$$

which quantifies the relative importance of each parameter in determining the output variability. The first-order Sobol' index captures the main effect of a parameter.

The total Sobol' index accounts for the total contribution of a parameter to the output variance, including both its main effect and all higher-order interactions with other parameters. The total Sobol' index of X_i is obtained by substracting the variance contribution of all other parameters from the total variance:

$$S_i^T = 1 - \frac{\mathrm{Var}_{\boldsymbol{X}_{\setminus i}}[\mathbb{E}_{X_i}(Y|\boldsymbol{X}_{\setminus i})]}{\mathrm{Var}(Y)}. \tag{IV.14}$$

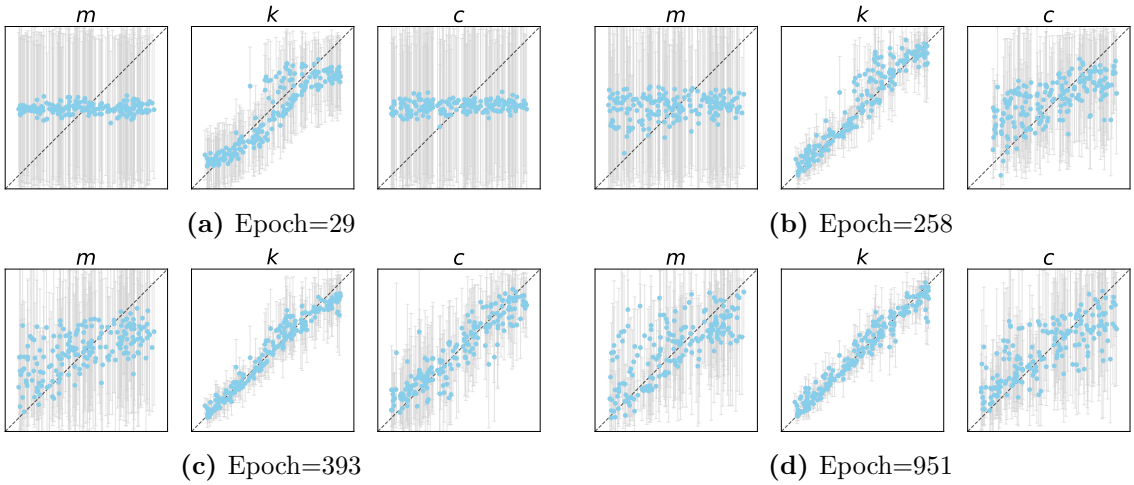
Another key issue of using variance-based **SA!** is to sample the parameter space efficiently. The standard random sampling often creates clusters or gaps in the parameter space. Instead, Sobol' sequences are applied, which are low-discrepancy sequences. For the sake of stability, a larger number of samples should be set.

By comparing these two indices, we can diagnose the structural complexity of the mass-spring-damper system. For instance, a parameter with a high total Sobol' index but a low first-order Sobol' index indicates that while the parameter is influential, its effect is primarily driven by interactions with other components of the model. If a parameter exhibits a total Sobol' index near zero, it suggests that the model output is insensitive to that parameter across its entire prior range. Such parameters are unidentifiable under the provided observed data.

Ultimately, integrating global **SA!** into the amortized Bayesian workflow allows us to preventively identify blind spots in the model. By detecting parameters with low sensitivity or high correlation, we can strategically focus on data collection efforts or reparameterize the model to ensure that the posterior distributions are well-defined and physically meaningful. When parameters exhibit significantly different levels of identifiability, a sensitivity-guided iterative identification strategy is recommended. In such cases, strongly identifiable parameters (those with high S_i^1 values, like the primary spring constants) should be identified first to stabilize the model. Once these dominant parameters are constrained, the remaining weakly identifiable or highly correlated parameters can be addressed in subsequent stages by adaptation of observational data or by refining the prior distributions. This hierarchical approach prevents the variance of dominant parameters from obscuring the signals of secondary parameters, ensuring a more robust and physically meaningful

calibration of the complex system.

The sensitivity of parameters also reflects the convergence speed of training deep Bayesian inference models as well. Taking a closer look at the mass-spring-damper model with external force, we get very close first-order and total-order indices, i.e., $S_m^1 = 0.07, S_k^1 = 0.88, S_c^1 = 0.03$ and $S_m^T = 0.07, S_k^T = 0.89, S_c^T = 0.05$. It suggests that for the single-degree-of-freedom system under external excitation, the stiffness parameter k dominates the system response and there is hardly any parametric interaction. This means the model is effectively additive in the chosen parameter ranges. From an identifiability perspective, the k should be much easier to estimate than the other two parameters.



(a) Epoch=29 **(b)** Epoch=258

(c) Epoch=393 **(d)** Epoch=951

Figure IV.7: Parameter identification results of **RealNVP!** at the early training stages. The figures show the estimation results of parameters m, k, c. The horizontal axis represents the ground truth, while the vertical shows the estimated values. The perfect convergence is demonstrated by the diagonal dashed line.

Figure **??** illustrates the convergence behavior of the three parameters during the early stages of training. By the 29th epoch, the stiffness k begins to converge toward the diagonal parity line, whereas the other two parameters remain unresolved, centered at the mean of their respective ranges. At the 258th epoch, c begins to show a slight tendency toward convergence, while k is further refined; however, m shows no significant improvement. By the 393rd epoch, m also begins to converge. Finally, at the 951st epoch, m and c reach a similar level of accuracy, while the estimate for k continues to improve. This development reflects the sensitivity of parameters. It is worth noting that faster convergence of c than m results from the larger data range of c. The convergence speed of both parameters eventually synchronizes.

Figure **??** compares the sensitivity analysis of the free and coupled mass-spring-models.

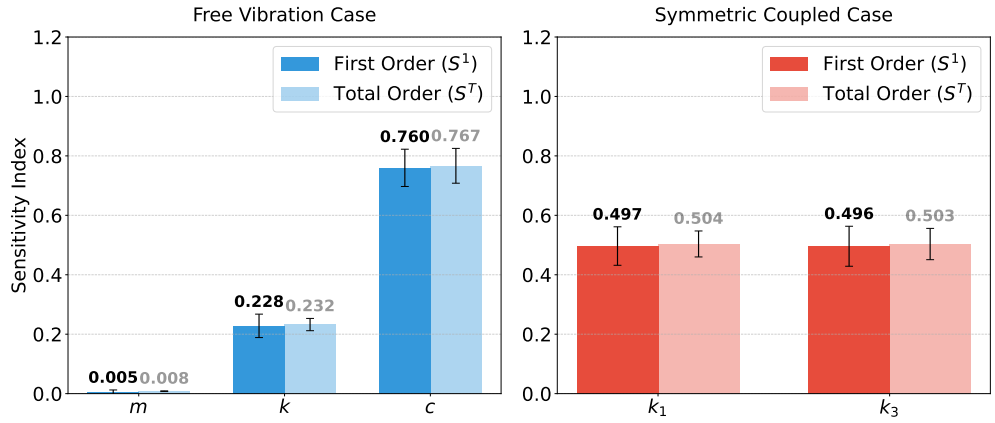


Figure IV.8: Global **SA!** comparison in free-vibration and symmetric coupled mass-spring-damper systems. The bar charts display the first-order (S^1) and total-order (S^T) Sobol' indices, with error bars representing the 95 % confidence interval. (Left) In the free-vibration case, the parameters m, k, c have highly different sensitivities. (Right) In the symmetric coupled two-mass system, k_1 and k_3 exhibit nearly identical sensitivity.

In the free-vibration case, the external force is removed. The behavior of the mass-spring-model is heavily influenced by how fast the energy loses, which is caused by the damping c. The global **SA!** of the free-vibration system reveals a clear hierarchy of parameter identifiability. The damping coefficient c dominates the output variance ($S_c^T = 0.767$), followed by the stiffness k ($S_k^T = 0.232$), and the mass m exhibits negligible sensitivity ($S_m^T = 0.008$). Furthermore, the near-equality of S^1 and S^T across all parameters indicates a lack of higher-order interactions. This suggests m is weakly identifiable, whereas the other two parameters can be identified independently.

The global **SA!** of the symmetric coupled two-mass system reveals a perfect parity between the boundary stiffness parameters. Both k_1 and k_3 exhibit nearly identical first-order ($S^1 \approx 0.496$) and total-order ($S^T \approx 0.503$) indices. The near-unity sum of their first-order indices ($S_{k_1} + S_{k_3} \approx 1$) suggests that these two parameters account for almost the entire variance of the system's dynamic response, with negligible higher-order interactions ($S^T - S^1 < 0.01$). From an identifiability perspective, this symmetry presents a unique challenge for Bayesian inference. While the high sensitivity ensures that the parameters will converge quickly away from the prior mean, the identical nature of their influence results in a bimodal posterior distribution. This confirms the indistinguishability and multimodality: the amortized inference model can identify the required stiffness levels but cannot uniquely assign them to k_1 or k_3 without additional asymmetric constraints or localized observations.

IV.1.3.2 Mutual Information

Global **SA!** does not always provide a reliable measure of parameter identifiability, particularly when observations are corrupted by additive noise. Consider a model $f(\boldsymbol{X})$ whose observations contain additive Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma_n^2)$, such that $Y = f(\boldsymbol{X}) + \epsilon$. Assuming the noise is independent of the parameters, the total observed variance becomes $\mathrm{Var}(Y) = \mathrm{Var}(f(\boldsymbol{X})) + \sigma_n^2$. Under this condition, the first-order and total-order Sobol' indices behave differently:

$$S_i^1 = \frac{V_i}{\mathrm{Var}(Y) + \sigma_n^2} < \frac{V_i}{\mathrm{Var}(Y)}; \tag{IV.15}$$

$$S_i^T = 1 - \frac{\mathrm{Var}_{\boldsymbol{X}_{\setminus i}}[\mathbb{E}_{X_i}(Y|\boldsymbol{X}_{\setminus i})]}{\mathrm{Var}(Y) + \sigma_n^2} > 1 - \frac{\mathrm{Var}_{\boldsymbol{X}_{\setminus i}}[\mathbb{E}_{X_i}(Y|\boldsymbol{X}_{\setminus i})]}{\mathrm{Var}(Y)}. \tag{IV.16}$$

As the noise ratio increases, the first-order Sobol' index decreases because the denominator grows while the numerator remains constant. In contrast, the total-order Sobol' index increases by adding the variance of noise in the denominator. This behavior is highly counter-intuitive: a higher total Sobol' index usually suggests that a parameter is more influential, yet in this case, the increase is driven entirely by noise, which actually makes the parameter more difficult to identify. Consequently, variance-based metrics can fail to distinguish between true parameter sensitivity and the presence of measurement uncertainty.

To address these limitations, **MI!** (**MI!**) (**?**) offers a robust alternative for evaluating the practical identifiability of parameters. Unlike global **SA!**, which only considers the second moment (variance), **MI!** is an information-theoretic measure that quantifies the total information gain from the prior to the posterior distribution given the data:

$$\mathrm{MI}(X_i, Y) = H(p(X_i)) - H(p(X_i|Y)), \tag{IV.17}$$

where H denotes the entropy or Shannon entropy. In information theory, the entropy of a (discrete) random variable X is defined as:

$$H(X) = -\sum p(x) \log p(x), \tag{IV.18}$$

which measures the expected amount of information needed to describe the state of the variable.

The **MI!** captures the reduction in uncertainty about a parameter X_i after observing Y. A higher **MI!** indicates that the data provides more information about the parameter, suggesting better identifiability. In contrast, a lower **MI!** implies that the observations are less informative, indicating that the parameter is poorly identifiable under the given noise

conditions.

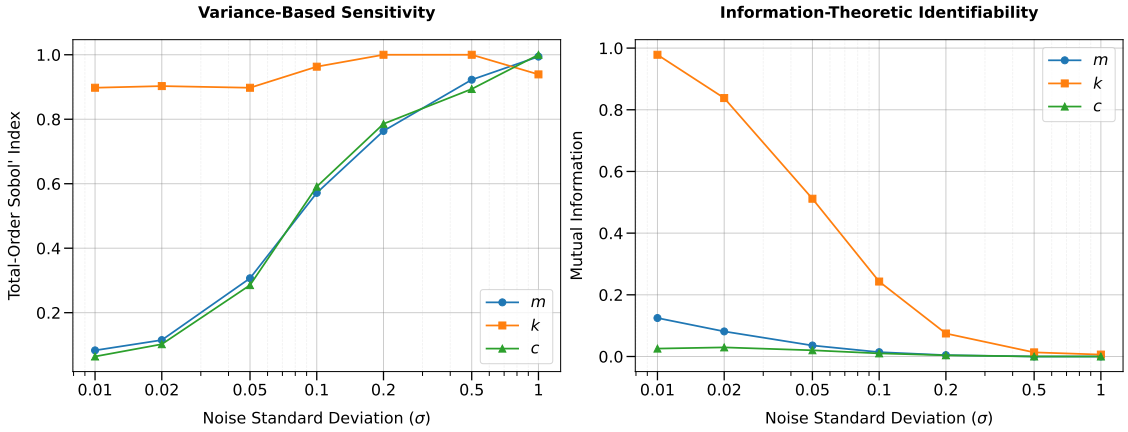


Figure IV.9: Comparison of parameter sensitivity and identifiability under varying noise levels. The left panel displays the total-order Sobol' index, while the right panel shows the Mutual Information (MI) for the parameters m, k, c of a forced mass-spring-damper model. Both metrics were estimated using 1024 samples generated from a low-discrepancy sequence. The horizontal axis represents the noise ratio (scaling a standard normal distribution) on a logarithmic scale.

A comparison between the total-order Sobol' index and **MI!** across varying noise levels is shown in Figure **??**. While S^T suggests a rising sensitivity as σ increases, which results from the noise contributing to the total variance, the MI captures the actual degradation of the signal-to-noise ratio. The divergence between these metrics demonstrates that sensitivity can be a misleading proxy for identifiability in noisy environments. Beyond a threshold of $\sigma = 0.02$, the sharp drop in MI indicates that the parameters cannot be identified as perfectly as the clean observations. Beyond a threshold of $\sigma = 0.2$, all parameters are poorly identifiable, meaning the observations no longer provide sufficient information to constrain the prior.

IV.1.4 When Does the Framework Fail?

From the previous sections, we have known the framework frequently struggles with posterior identifiability; specifically, when the system exhibits multimodality, generative models may fail to resolve disjoint parameter regimes. Apart from identifiability, the efficacy of amortized Bayesian inference via deep generative models is intrinsically tied to the fidelity of the latent representations extracted by the condition network. This framework encounters significant bottlenecks under three primary conditions. First, in the presence of high-dimensional, noisy, or highly stochastic time-series data, the condition network

often fails to distill sufficient summary statistics, leading to information loss. Second, the framework is susceptible to model misspecification. When a discrepancy exists between the training simulator and real-world dynamics (the sim-to-real gap), the amortized posterior yields biased and poorly calibrated estimates. Third, the framework fails to generalize to parameter ranges outside its training boundaries, even if the underlying physics remain the same.

We evaluate the robustness of **NF!**s (including **FM!**, **RealNVP!**, and **NSF!**) by subjecting the framework to the following stress-test scenarios:

- Information Extraction Bottlenecks: Increasing observational noise in both training and test sets to assess the limits of the condition network's feature extraction.
- Structural Misspecification: Evaluating models trained on linear dynamics against data generated from non-linear physical systems.
- **OOD!** (**OOD!**) Generalization: Testing the framework on parameter ranges that lie outside the support of the training prior.

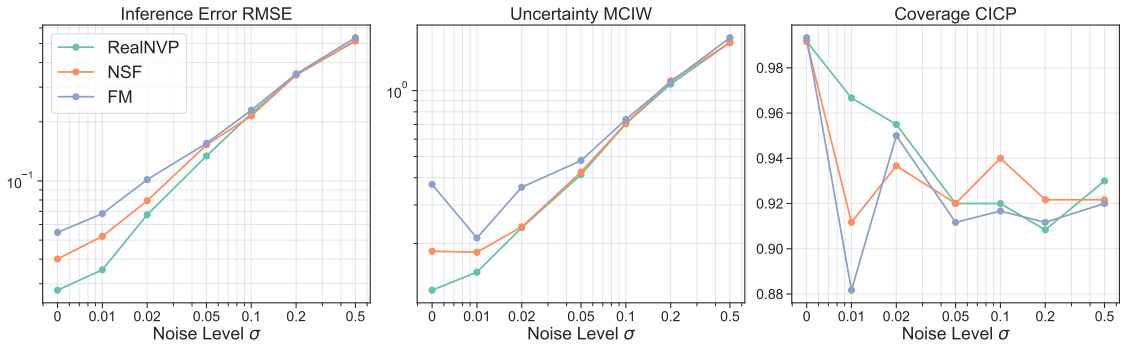


Figure IV.10: Stress test of Normalizing Flow architectures under increasing additive Gaussian noise. The panels illustrate the evolution of predictive accuracy (RMSE), uncertainty quantification (MCIW), and posterior calibration (CICP) for **RealNVP!**, **NSF!**, and **FM!** as the noise intensity σ changes. Logarithmic scaling is applied to both the noise level and the corresponding RMSE and MCIW metrics to highlight performance degradation across several orders of magnitude.

Figure **??** illustrates the performance of the three **NF!** architectures under varying levels of additive Gaussian noise σ. At the low-noise level ($\sigma = 0.01$), all models exhibit a sharp decline in coverage probability, particularly the **FM!** and **NSF!** variants. This indicates a calibration failure where the condition network fails to fully distinguish observational stochasticity from parameter variance, leading to overly confident (narrow) posteriors. As

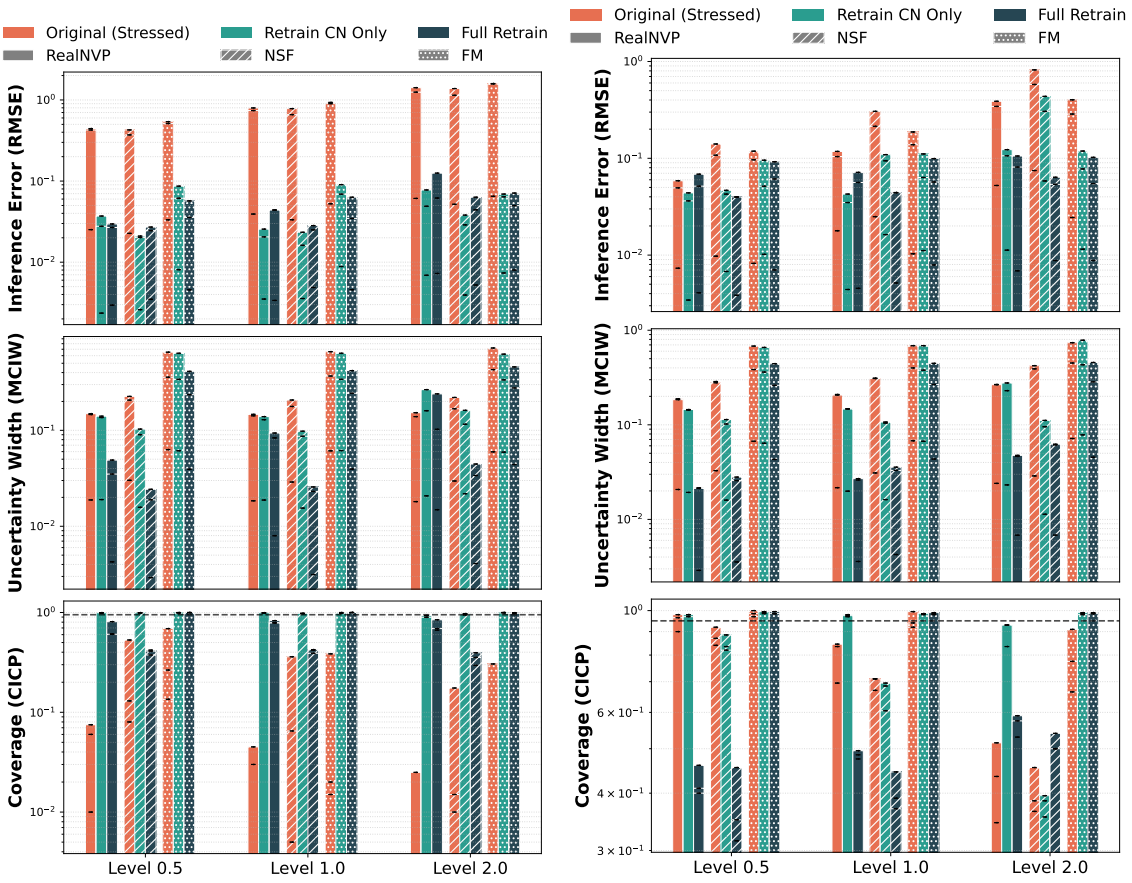
noise intensity increases beyond $\sigma = 0.05$, the models show a monotonic degradation in accuracy (RMSE) accompanied by a proportional expansion of uncertainty (MCIW). In the high-noise range ($\sigma \geq 0.05$), the models converge in behavior, successfully broadening their posterior estimates to maintain a stable, albeit lower, coverage floor of approximately 0.92.

In such cases, the condition network encounters a representational bottleneck; it fails to extract sufficient summary statistics, preventing the observational variance from being accurately mapped to the parameter sensitivity. To mitigate this, an **MCMC!** (**MCMC!**) algorithm can be employed to correct the estimates, using the amortized posterior as an informative proposal distribution. Alternatively, a more computationally efficient approach involves **SNF!** (**SNF!**), which integrates the simulator directly into the latent transformation as a log-posterior diagnostic, as detailed in Section **??**.

The structural misspecification and **OOD!** generalization problems can be solved by active learning, i.e., retraining the architectures with new data. Figure **??** shows the improvement through model adaptation of misspecification and **OOD!** generalization, respectively. In both cases, comparing different retraining methods (different colors), updating only the condition network effectively centers the distribution, significantly reducing RMSE and bringing CICP back toward the 0.95 target. While retraining the whole networks produces the most improvement in terms of minimizing error and tightening confidence intervals, it carries the highest risk of overfitting the posterior. Comparing different flow architectures (different patterns), RealNVP and NSF achieve the lowest RMSE and narrowest MCIW after retraining. However, they are highly sensitive to the misspecification or **OOD!** stress. In contrast, FM shows higher inference error and wider uncertainty than others but maintains higher coverage probability. It is less prone to overfitting and resilient against the stress.

Figure **??** is a micro view of the joint distributions between m and c in a test scenario. As the stress level increases, a clear divergence from the ground truth is observed in the baseline models. While FM exhibits a larger posterior variance compared to the other architectures, it demonstrates superior robustness. Retraining the condition network successfully shifts the posteriors back toward the ground truth. However, full-network retraining for RealNVP and NSF leads to overconfidence; while point accuracy increases, the distributions shrink excessively, causing a drop in coverage. In contrast, FM retraining further constrains the distribution without inducing this overfitting effect, maintaining better calibration than the condition-network adaptation.

Figure **??** represents a global residual analysis under OOD. By centering the ground truth of each scenario at $(0, 0)$, the subplots reveal the systematic bias and uncertainty induced by increasing **OOD!** shifts. As the shift increases from 0.5 to 2.0, the Original baseline distributions exhibit both a significant expansion in variance and a pronounced



(a) Misspecification **(b)** OOD

Figure IV.11: Evaluation of model performance under structural misspecification and **OOD!** stress tests. Rows from top to bottom display inference accuracy (RMSE), uncertainty estimation (MCIW), and coverage probability (CICP). The dashed line in CICP subfigure shows 0.95 coverage probability. Metric values are averaged across 200 test scenarios and three mechanical parameters (m, k, c). The grouped bar plots illustrate performance at varying stress levels $0.5, 1, 2$. For structural misspecification, the level corresponds to the magnitude of the nonlinear factor β introduced into the system dynamics: $\dot{v} = (F_t - kx + \beta x^3 - cv + \beta v|v|)/m$. For **OOD!** testing, the level α variates the shift Δ of the test data range relative to the original training set, where $\Delta = (+0.1\alpha, +\alpha, +0.5\alpha)$. At each level, colors indicate the training state: the original baseline under stress, and the improved results after retraining either the condition network or the entire network. Bar patterns represent the normalizing flow architecture (RealNVP, NSF, and FM).

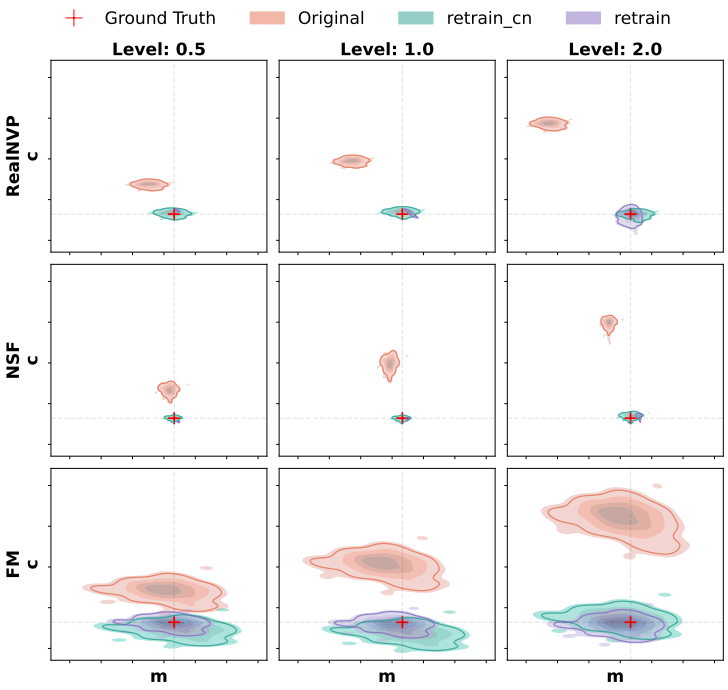


Figure IV.12: Micro View of posterior distributions under misspecification for a single test scenario. The 3×3 grid illustrates the joint posterior distributions of parameters m and c across three architectures (RealNVP, NSF, and FM) and three misspecification levels ($\beta = 0.5, 1, 2$). Colors represent the model state: the baseline under stress (Original), the improved result after retraining the condition network (retrain-cn), and the result after full-network retraining (retrain). The ground truth is marked by a red cross (+).

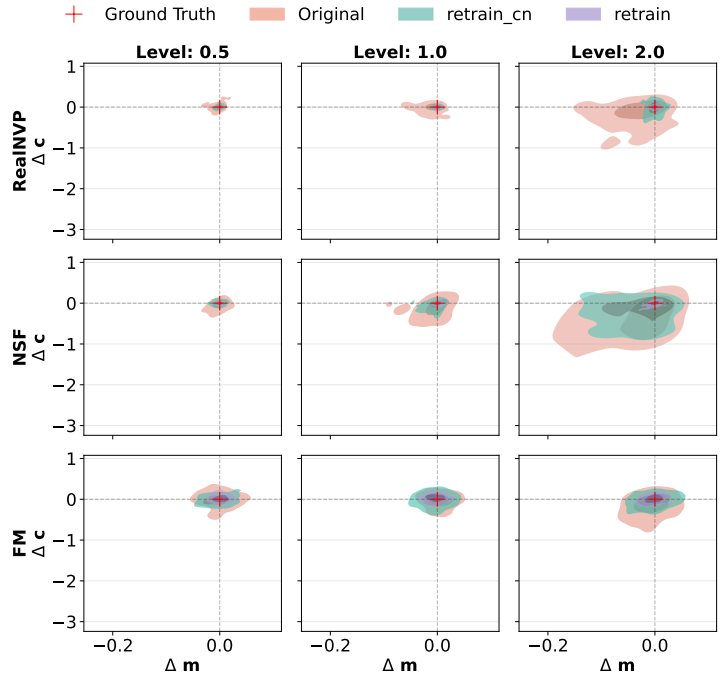


Figure IV.13: Global residual analysis under **OOD!** for 200 test scenarios. The 3×3 grid illustrates a global residual analysis of the joint distributions for mass $\Delta m = \hat{m} - m$ and damping $\Delta c = \hat{c} - c$ across three architectures (RealNVP, NSF, and FM) and three **OOD!** levels ($\alpha = 0.5, 1, 2$). Colors represent the model state: the baseline under stress (Original), the improved result after retraining the condition network (retrain-cn), and the result after full-network retraining (retrain). The perfect match (zero error) is marked by a red cross (+).

negative systematic bias, particularly in the NSF and RealNVP architectures. NSF displays the highest sensitivity to **OOD!**, showing the largest displacement from the origin and the most irregular posterior shapes. In contrast, FM demonstrates superior baseline robustness; even at Level 2.0, its original distribution remains more symmetric and better centered than the other architectures. Consequently, while retraining the conditional network and the full network significantly reduces error for RealNVP and NSF by recentering the distributions on the ground truth, FM shows a smaller marginal improvement.

IV.1.5 Stochastic Normalizing Flows

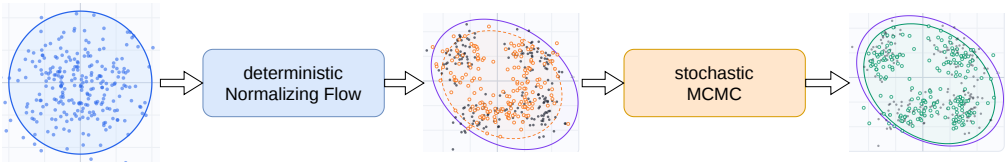


Figure IV.14: Combining Normalizing Flow with MCMC. The normalizing flow transforms the latent distribution to the target distribution. Then, MCMC further refines the proposed distribution by Normalizing Flow.

Normalizing flows, including flow matching, are deterministic by design, which enables precise and efficient density estimation. However, in practice this deterministic structure can limit their ability to fully capture complex, highly multimodal target distributions, especially when model capacity or data are limited. One way to alleviate this limitation is to combine normalizing flows with **MCMC!** methods. In such hybrid approaches, the normalizing flow serves as an expressive proposal or initialization mechanism, while **MCMC!** refines the samples by enforcing consistency with the true posterior through its accept–reject mechanism or transition kernel, as shown in Figure **??**. The stochastic exploration of **MCMC!** improves mode coverage and helps the overall system discover and accurately represent multiple modes that might be underrepresented by the deterministic flow alone. Moreover, by repeatedly evaluating the forward dynamics within the **MCMC!** procedure, the method can better discriminate between parameter configurations that produce similar noisy observations, thereby improving parameter identifiability under measurement noise.

IV.1.5.1 Theory of Stochastic Normalizing Flows

? (**?**) proposed to construct **SNF!**s by alternating a sequence of deterministic **INN!**s (**INN!**s) and stochastic **MCMC!** sampling layers (INN - MCMC - $\cdots$ - INN - MCMC). **?** (**?**) provided the theoretical foundations for stochastic normalizing flows and proved

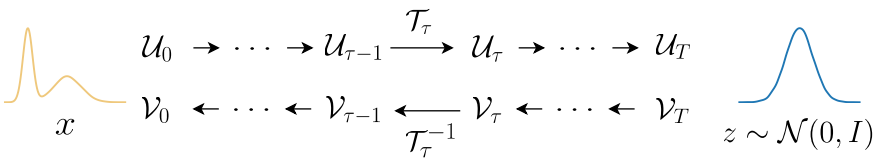


Figure IV.15: The architecture of an **SNF!** with Markov chains $((\mathcal{U}_0, \ldots, \mathcal{U}_T), (\mathcal{V}_T, \ldots, \mathcal{V}_0))$. T is the number of stochastic blocks. We denote the transformation in the forward process (from data to latent) by $\mathcal{T}_t = P_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}$ and that in the inverse process (from latent to data space) by $\mathcal{T}_t^{-1} = P_{\mathcal{V}_{\tau-1}|\mathcal{V}_\tau}$. To be consistent with **NF!** as shown in Figure **??**, the left side of the figure is the data distribution, while the right side is the latent distribution. Usually, the latent variable follows a standard normal distribution. The upper row illustrates the forward direction from data to latent space, whereas the bottom row shows the inverse direction from latent to data space.

their consistency with **CNF!**s (**CNF!**s) for solving inverse problems. An **SNF!** is a pair of Markov chains of random variables $((\mathcal{U}_0, \ldots, \mathcal{U}_T), (\mathcal{V}_T, \ldots, \mathcal{V}_0))$. The transformation between two adjacent variables is either a deterministic **INN!** block or a stochastic **MCMC!** sampling layer. The process $(\mathcal{U}_\tau)_{\tau=0}^T$ is denoted as the process from data to latent space, whereas the process $(\mathcal{V}_\tau)_{\tau=T}^0$ is the reverse process from latent to data space. $\mathcal{V}_T$ represents the latent variable, which follows a standard normal distribution and $\mathcal{U}_0$ represents the data distribution. Since both processes are Markov chains, the transformations can be written as conditional distributions, where each state depends only on its immediate predecessor. Figure **??** illustrates this architecture.

Unlike **CNF!**s, for which the objective function is minimizing the KL-divergence between the learned and target density, the objective function for training an **SNF!** is to minimize the difference of the joint distributions between the forward and inverse processes. We define the joint distributions as:

$$P_{\mathcal{U}_{0:T}} = P_{\mathcal{U}_0 \sim p_X} \prod_{t=1}^{T} P_{\mathcal{U}_\tau|\mathcal{U}_{\tau-1}} \qquad \text{(forward: data} \to \text{latent)}, \tag{IV.19a}$$

$$P_{\mathcal{V}_{0:T}} = P_{\mathcal{V}_T \sim p_Z} \prod_{t=1}^{T} P_{\mathcal{V}_{\tau-1}|\mathcal{V}_\tau} \qquad \text{(inverse: latent} \to \text{data)}. \tag{IV.19b}$$

Then, the objective taking the forward process as training variables is:

$$\mathcal{L}_{SNF}(\varphi) = D_{\mathrm{KL}}(P_{\mathcal{V}_{0:T}} || P_{\mathcal{U}_{0:T}}) = \mathbb{E}_{\mathcal{U}_{0:T} \sim P_{\mathcal{V}_{0:T}}} \log \frac{dP_{\mathcal{V}_{0:T}}}{dP_{\mathcal{U}_{0:T}}}. \tag{IV.20}$$

Similar to **CNF!**s, the objective function of an **SNF!** is trained from data to the latent

space. The derivative term can be expanded as:

$$\frac{dP_{\mathcal{V}_{0:T}}}{dP_{\mathcal{U}_{0:T}}} = \frac{p_{\mathcal{V}_0}(\mathcal{U}_0)}{p_{\mathcal{U}_0}(\mathcal{U}_0)} \prod_{t=1}^{T} \frac{p_{\mathcal{V}_\tau|\mathcal{V}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})}{p_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})} = \frac{p_{\mathcal{V}_T}(\mathcal{U}_T)}{p_{\mathcal{U}_T}(\mathcal{U}_T)} \prod_{t=1}^{T} \frac{p_{\mathcal{V}_{\tau-1}|\mathcal{V}_\tau}(\mathcal{U}_{\tau-1}|\mathcal{U}_t)}{p_{\mathcal{U}_{\tau-1}|\mathcal{U}_t}(\mathcal{U}_{\tau-1}|\mathcal{U}_t)}. \tag{IV.21}$$

Because $\mathcal{V}_T \equiv Z$ and $\mathcal{U}_0 \equiv X$ and the SNF is trained from $\mathcal{U}_0$ to $\mathcal{U}_T$, the term can be rewritten as:

$$\frac{dP_{\mathcal{V}_{0:T}}}{dP_{\mathcal{U}_{0:T}}} = \frac{p_{\mathcal{V}_T}(\mathcal{U}_T)}{p_{\mathcal{U}_0}(\mathcal{U}_0)} \prod_{t=1}^{T} f_t(\mathcal{U}_{\tau-1}, \mathcal{U}_t) \frac{p_{\mathcal{V}_{\tau-1}}(\mathcal{U}_{\tau-1})}{p_{\mathcal{V}_\tau}(\mathcal{U}_t)}, \quad f_t(\mathcal{U}_{\tau-1}, \mathcal{U}_t) = \frac{p_{\mathcal{V}_\tau|\mathcal{V}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})}{p_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})}. \tag{IV.22}$$

Then, the objective is:

$$\mathcal{L}_{SNF}(\varphi) = \mathbb{E}_{\mathcal{U}_{0:T}\sim P_{\mathcal{V}_{0:T}}} \left[\log p_{\mathcal{V}_T}(\mathcal{U}_T) - \log p_{\mathcal{U}_0}(\mathcal{U}_0) + \sum_{t=1}^{T} \log f_t(\mathcal{U}_{\tau-1}, \mathcal{U}_t) \frac{p_{\mathcal{V}_{\tau-1}}(\mathcal{U}_{\tau-1})}{p_{\mathcal{V}_\tau}(\mathcal{U}_t)} \right] \tag{IV.23}$$

The term $f_t(\mathcal{U}_{\tau-1}, \mathcal{U}_t) p_{\mathcal{V}_{\tau-1}}(\mathcal{U}_{\tau-1})/p_{\mathcal{V}_\tau}(\mathcal{U}_t) \overset{\text{def}}{=} r$ differs for deterministic and stochastic layers. For a **deterministic** transformation, $f_t(\mathcal{U}_{\tau-1}, \mathcal{U}_t) = 1$ and $p_{\mathcal{V}_{\tau-1}}(\mathcal{U}_{\tau-1})/p_{\mathcal{V}_\tau}(\mathcal{U}_t)$ equals the change of variable:

$$r = 1 \cdot \frac{p_{\mathcal{V}_{\tau-1}}(\mathcal{U}_{\tau-1})}{p_{\mathcal{V}_\tau}(\mathcal{U}_t)} = \det \left| J_{\mathcal{T}_t}(\mathcal{U}_{\tau-1}) \right|. \tag{IV.24}$$

For a **stochastic** transformation, r measures how well the backward kernel inverts the forward one; it can be reformulated as:

$$r = \frac{p_{\mathcal{V}_\tau|\mathcal{V}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})}{p_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})} \frac{p_{\mathcal{V}_{\tau-1}}(\mathcal{U}_{\tau-1})}{p_{\mathcal{V}_\tau}(\mathcal{U}_t)} = \frac{p_{\mathcal{V}_{\tau-1}|\mathcal{V}_\tau}(\mathcal{U}_{\tau-1}|\mathcal{U}_t)}{p_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})}. \tag{IV.25}$$

The stochastic layer must be invertible and fulfill the detailed balance condition for some density π_t:

$$\pi_t(\mathcal{U}_t) p_{\mathcal{V}_{\tau-1}|\mathcal{V}_\tau}(\mathcal{U}_{\tau-1}|\mathcal{U}_t) = \pi_t(\mathcal{U}_{\tau-1}) p_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1}). \tag{IV.26}$$

where π_t is the target density at step t, interpolating between p_Z and p_X: $\pi_t = c \cdot p_Z^{t/T} \cdot p_X^{(T-t)/T}$ with c a normalizing constant and $E_t(u) = -\log(\pi_t(u))$. Rearranging Equation **??**:

$$r = \frac{p_{\mathcal{V}_{\tau-1}|\mathcal{V}_\tau}(\mathcal{U}_{\tau-1}|\mathcal{U}_t)}{p_{\mathcal{U}_t|\mathcal{U}_{\tau-1}}(\mathcal{U}_t|\mathcal{U}_{\tau-1})} = \frac{\pi_t(\mathcal{U}_{\tau-1})}{\pi_t(\mathcal{U}_\tau)}. \tag{IV.27}$$

For **MCMC!** this is computed by evaluating π_t; for efficiency an **MALA!** (**MALA!**)

kernel (Section **??**) yields:

$$r = \frac{\pi_t(\mathcal{U}_{\tau-1})}{\pi_t(\mathcal{U}_\tau)} = \exp\left(\frac{1}{2}\left(\|\tilde{\eta}_t\|_2^2 - \|\eta_t\|_2^2\right)\right), \tag{IV.28}$$

where the Gaussian noise vectors are:

$$\eta_t := \frac{1}{\sqrt{\tau}}\left(\mathcal{U}_{\tau-1} - \mathcal{U}_t - \frac{\tau}{2}\nabla E_t(\mathcal{U}_{\tau-1})\right) \qquad \text{(forward: } \mathcal{U}_{\tau-1} \to \mathcal{U}_\tau\text{)}, \tag{IV.29}$$

$$\tilde{\eta}_t := \frac{1}{\sqrt{\tau}}\left(\mathcal{U}_{\tau-1} - \mathcal{U}_t + \frac{\tau}{2}\nabla E_t(\mathcal{U}_t)\right) \qquad \text{(backward: } \mathcal{U}_\tau \to \mathcal{U}_{\tau-1}\text{)}. \tag{IV.30}$$

IV.1.5.2 Experiments on Stochastic Normalizing Flows

We evaluate **SNF!**s on two fronts: multimodal posterior modeling and parameter identification under noisy observations. The following experiments demonstrate the practical benefits of combining normalizing flows with stochastic **MCMC!** layers.

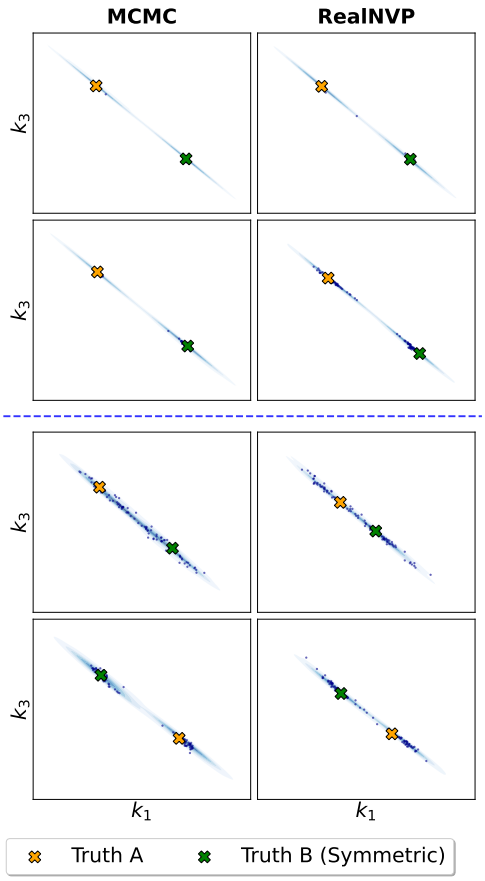


Figure IV.16: Comparison of MCMC-RealNVP (left) and RealNVP (right) for multimodal modeling. Experimental settings match Fig. **??**.

Table IV.4: Comparison of Posterior Estimation Metrics. For RMSE/MCIW, lower is better (↓); for CICP, ≈ 0.95 is ideal. Metrics are mode-aligned with centers identified via K-means clustering. The results except the MCMC are corresponding to Figure **??**. The MCMC stands for an SNF with MCMC and RealNVP.

Model	RMSE (↓)	MCIW (↓)	CICP
BNN	1.611	4.581	0.843
VAE	1.375	1.527	0.460
DDPM	1.415	2.211	0.560
SBM	0.540	0.838	0.553
FM	0.300	0.830	0.873
RealNVP	0.258	1.067	0.925
NSF	0.108	**0.412**	0.928
MCMC	**0.105**	0.508	**0.970**

In the previous multimodal experiments (Section **??**), **NSF!** performs the best in terms of accuracy of multimodal recovery, while **RealNVP!** exhibits some dislocation. To strengthen the multimodal capability of **RealNVP!**, we implement a hybrid **SNF!** architecture that interleaves four intermediate **MCMC!** refinement steps with the flow (step size 0.001). During the forward (training) pass, the **MCMC!** updates are applied to the latent encoding to improve likelihood estimation; during the inverse (generative) pass, they are applied to the output samples to improve alignment with the target data distribution. This stochastic refinement enables the flow to capture disconnected modes that a simpler deterministic **NF!** may miss. Table **??** shows that this hybrid approach achieves the best RMSE and near-ideal coverage, outperforming **NSF!** in both precision and uncertainty quantification.

The **MCMC!** layer can also be replaced by a **MALA!** kernel, which accounts for forward dynamics and can improve parameter identification under noisy observations. Except for NF architectures like **RealNVP!** and **NSF!**, the refinement is also applicable to **FM!**. We define the log-likelihood by measuring the residual between the observed trajectories and the signals generated by the inferred parameters via the governing dynamics. To accommodate varying parameter sensitivities, the **RMALA!** (**RMALA!**) refinement leverages a metric tensor derived from the local curvature of the log-posterior, typically approximated via the Fisher information matrix or Hessian. Whereas standard **MALA!** uses a constant step size in every direction, **RMALA!** adapts its step size to the local geometry, taking larger steps along directions of low sensitivity and smaller, more precise steps along directions of high sensitivity.

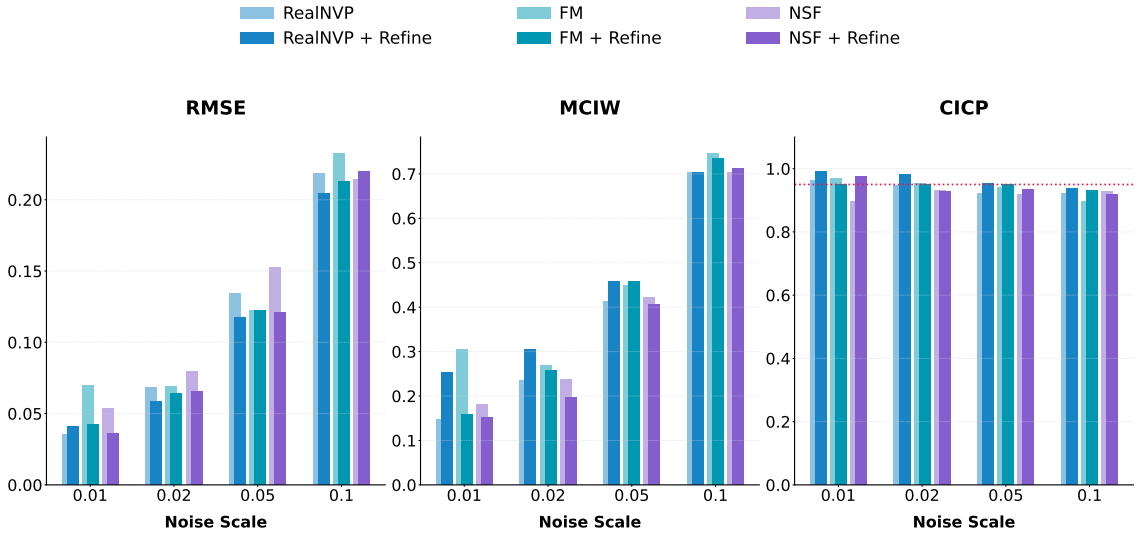


Figure IV.17: Improved parameter identification under noisy observations using **SNF!**s. Three **NF!** architectures—**RealNVP!**, **FM!**, and **NSF!**—are evaluated across noise scales of 0.01, 0.02, 0.05, and 0.1. Performance is measured in terms of accuracy (RMSE) and uncertainty quantification (MCIW and CICP). Grouped bars compare the baseline results with those obtained via **RMALA!**-based refinement. In the CICP subfigure, the dashed red line indicates the ideal coverage probability of 0.95.

Figure **??** compares parameter identification performance under noisy observations for **NF!** and **SNF!**. The RMSE panel shows error increasing with observational noise, but the stochastic refinement consistently outperforms the deterministic baseline models at all noise scales. At low noise levels (0.01 and 0.02), the refined models often exhibit larger MCIW values than the baseline models, indicating that the baseline intervals were overly narrow and overconfident. The **MALA!** layers broaden the posterior to better reflect the true uncertainty. At a noise scale of 0.1, the MCIW values converge, suggesting that in the high-noise regime the uncertainty magnitude is dominated by the data likelihood rather than the choice of flow architecture. The improvement in calibration is also evident in the CICP panel. In particular, at noise scale 0.01 the baseline **NSF!** model (light purple) falls well below the 0.95 target, while the refinement step (dark purple) restores coverage to the target. As noise increases, the deterministic models lose coverage (falling below 0.90), whereas the refined models maintain more stable coverage near the target across the entire noise range. This consistent improvement in CICP indicates that the stochastic layers move the approximate posterior closer to the true posterior.

Interestingly, **FM!** achieves exact 0.95 CICP at noise scales 0.01, 0.02, and 0.05. Because **FM!** learns a continuous vector field rather than a sequence of discrete couplings,

it provides a smoother initial density for the stochastic refinements. When **RMALA!** is applied to this smoother surface, the gradient-based updates can move samples to the true posterior boundaries with higher precision than in more rigid architectures such as **RealNVP!** and **NSF!**. This makes the **RMALA!**-refined **FM!** a particularly reliable approach for parameter identification in complex, noisy dynamical systems.

IV.2 State Estimation

Adapting normalizing flow methods from parameter identification to state estimation introduces several nontrivial challenges. Unlike parameter identification, which targets a fixed set of unknown variables, state estimation requires the recovery of an entire sequential trajectory from partial observations. In Bayesian terms, the task is to approximate the posterior distribution $p(x_{1:T}|y_{1:T})$, which may be high-dimensional and multimodal. Standard filtering approaches, such as the Kalman filter, typically rely on known linear Gaussian dynamics. The setting considered here imposes no such restrictions: both the state dynamics and the observation model may be nonlinear and stochastic, and the resulting posterior can have a complex geometry. A naive strategy that applies a parameter identification method independently at each time step fails to model temporal dependencies between successive states and allows errors to accumulate over time.

Classical approaches address this problem with sequential Monte Carlo methods, such as particle filters (**?**), which approximate the posterior by propagating a set of weighted particles through time. However, these methods are computationally expensive and suffer from degeneracy in high-dimensional state spaces. Variational approaches, including **VAE!**s and RNN-based inference networks, have also been proposed for amortized state estimation (see Section **??**), but they often struggle to represent complex temporal dependencies and multimodal posteriors.

To address these limitations, this section introduces four normalizing-flow-based methods for state estimation, ordered by increasing generality: Amortized Sequence Flow (AmortizedSeqFlow), Latent State Space Model Flow (LatentSSMFlow), Normalizing Flow SDE (FlowSDE), and Conditional Stochastic Interpolant (CondStochInterpolant). For AmortizedSeqFlow, we compare RealNVP and Flow Matching as the Bayesian inference backbones. For LatentSSMFlow, an exactly invertible flow is required as the decoder, so RealNVP is implemented. For FlowSDE, the RealNVP architecture is used to model the diffusion term of a NeuralSDE. For CondStochInterpolant, the whole process can be viewed as a refinement flow that transforms an observation-informed source distribution into the posterior state distribution.

All four methods are evaluated on the Lorenz-96 system (**?**), a canonical benchmark for nonlinear data assimilation. The state dimension is $N = 12$, and at each time step only $N/2 = 6$ components are observed, with additive Gaussian noise of varying magnitude. The governing equations are

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})\, x_{i-1} - x_i + F_l, \qquad i = 1, \ldots, N, \tag{IV.31}$$

with periodic boundary conditions $x_{-1} = x_{N-1}$, $x_0 = x_N$, $x_{N+1} = x_1$, and forcing constant

$F_l = 8$. This forcing regime produces sustained chaotic behavior.

IV.2.1 Amortized Sequence Flow

The most direct approach is to infer states across time by applying a normalizing flow independently to each observation segment. This reduces sequential state estimation to a collection of parameter inference problems. Its main advantage is that the parameter-identification methods introduced in the preceding section can be transferred with minimal modification. To introduce temporal structure into the latent variables, we use a Wiener

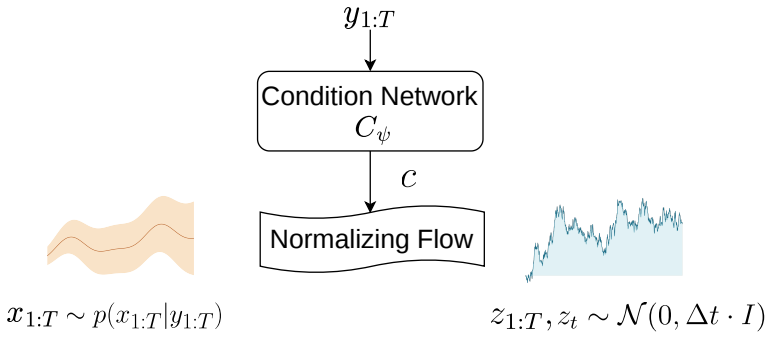


Figure IV.18: Architecture of amortized state estimation with normalizing flows.

process as the base distribution of the normalizing flow. This choice assigns monotonically increasing variance to successive latent variables while preserving a Gaussian distributional form, thereby providing an inductive bias toward smooth trajectories. The architecture is illustrated in Figure **??**.

This simplicity comes at a cost. Because each segment is processed as an independent set of observations, the method largely discards the temporal structure of the state trajectory. Long-range dependencies are not modeled, and estimation errors can propagate across segments without a corrective mechanism. AmortizedSeqFlow is therefore most appropriate when the underlying dynamics are relatively smooth and only weakly dependent over time. It is less suitable when the dynamics are strongly nonlinear or when the observations are substantially noisy.

Numerical Results

We first evaluate AmortizedSeqFlow under different segment lengths on the partially observed Lorenz-96 system. The dataset contains 2000 independent trajectories with horizon $[0s, 100s]$, time step $dt = 0.02s$, and 501 steps. Each initial state is sampled as a small Gaussian perturbation and then spun up before the training trajectory is

recorded. The observation matrix selects every second state coordinate and the others remain unobserved. The observation noise is Gaussian with standard deviation 0.5. The data are split into 1600 training trajectories, 200 validation trajectories, and 200 test trajectories.

Since this method applies the flow independently to each observation segment, it is not expected to scale well to long windows. The purpose of this experiment is therefore to assess to what extent this simple amortized construction can still be used for state estimation as the segment length increases. Figure **??** summarizes the quantitative performance, while Figure **??** shows representative reconstructed trajectories.

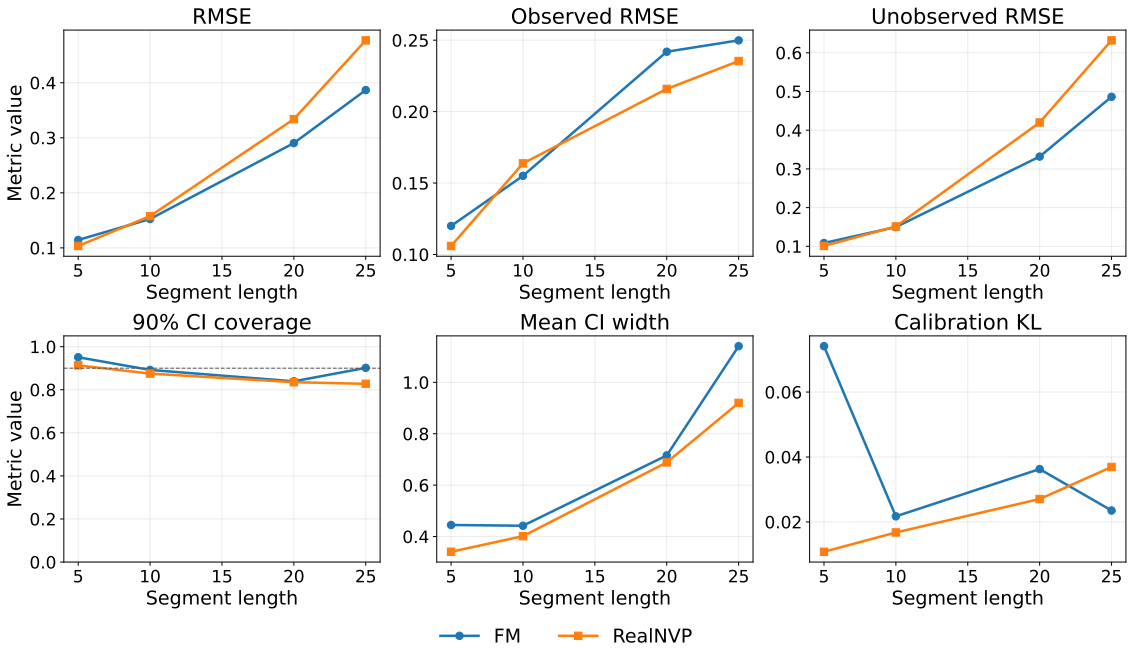


Figure IV.19: Segment-length comparison for AmortizedSeqFlow. The figure reports aggregate state-estimation metrics for different observation-window lengths $T_s = 5, 10, 20, 25$, showing how the independent segment-wise inference strategy changes in accuracy and uncertainty calibration as more temporal context is provided to each segment. The metrics include RMSE, 90% **CICP!**, **MCIW!**, and KL divergence between the estimated and true states.

The quantitative results in Figure **??** show that increasing the segment length decreases the amortized estimation precision. The total RMSE increases monotonically for both architectures. This degradation is mainly caused by the unobserved state components. Thus, the method can interpolate the directly observed coordinates reasonably well, but it becomes increasingly unreliable when reconstructing hidden coordinates over longer

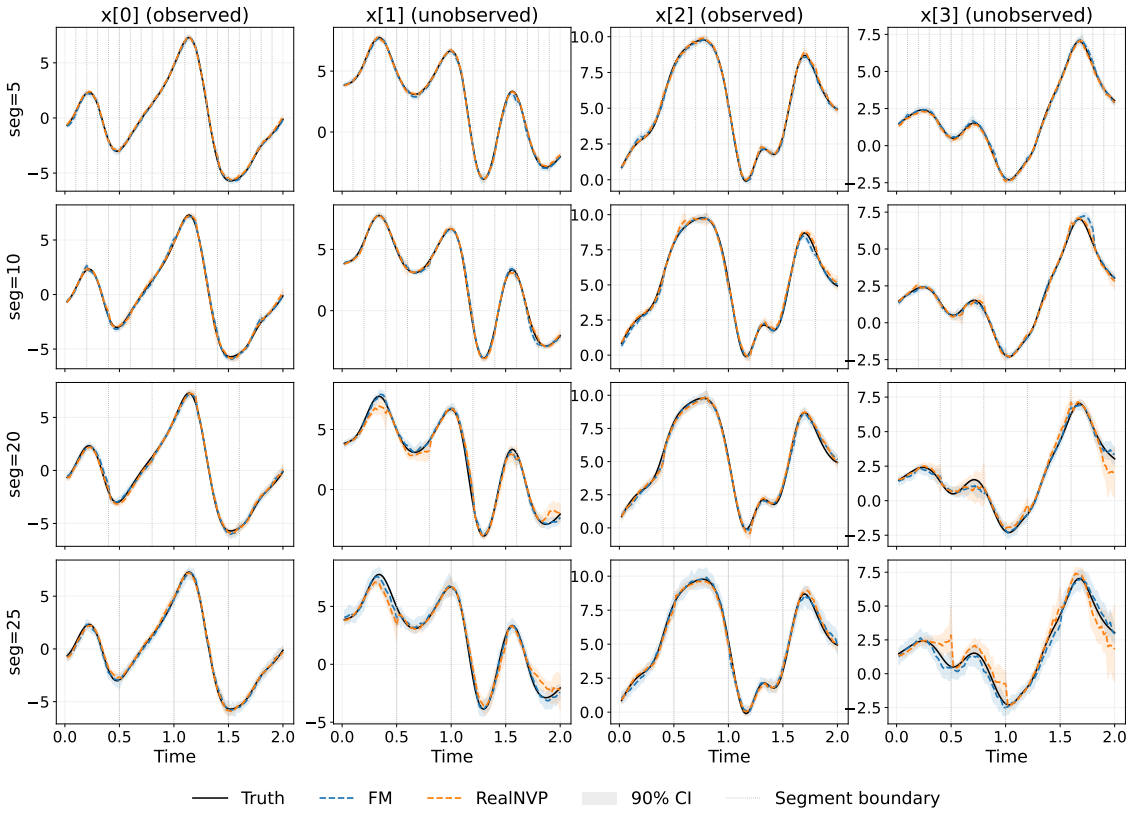


Figure IV.20: Representative trajectory reconstructions obtained with AmortizedSeqFlow for different segment lengths. The plots compare the estimated state trajectories with the reference dynamics and illustrate how segment-wise inference behaves across the physical time horizon. Observed (0, 2) and unobserved (1, 3) dimensions are shown separately.

windows. Comparing the two architectures, RealNVP has a lower RMSE for the shortest segment length, but its error increases more rapidly than Flow Matching as the segment length grows. The difference results from the characteristics of the two amortized estimators. RealNVP uses an explicitly invertible coupling architecture and is trained through likelihood maximization, which gives it a direct density-estimation objective for each segment. This often makes the posterior samples locally sharper and more stable when the observation window is short. Flow Matching, in contrast, learns a continuous transport vector field from the base process to the state trajectory distribution. Its estimates tend to vary more smoothly along the learned transport path and are more suitable for long-window estimation, but the method is more sensitive to vector-field approximation and numerical integration errors.

The uncertainty metrics lead to the same conclusion. The 90% **CICP!** is close to the

nominal level only for short windows; it drops below the nominal level at larger segment lengths. At the same time, the mean credible-interval width increases substantially, especially for Flow Matching. This indicates that the models respond to the harder long-window problem by broadening their posterior uncertainty, but the increased uncertainty is not sufficient to recover consistently calibrated intervals. The calibration KL comparison further suggests that RealNVP matches the uncertainty distribution of the observed components more closely than Flow Matching.

The trajectory reconstructions in Figure **??** explain this behavior visually. For segment lengths 5 and 10, both methods closely follow the reference trajectory for the shown state components, and the uncertainty bands remain narrow. At segment lengths 20 and 25, the estimates are still locally plausible, but deviations become visible around fast transitions and near segment boundaries. The effect is most evident in unobserved states $x[1]$ and $x[3]$, where the reconstructed trajectories begin to drift from the truth and the **CI!**s (**CI!**s) widen. RealNVP shows sharper local estimates at short segment lengths, but its long-window reconstructions develop stronger boundary artifacts and larger errors in the unobserved components. Flow Matching is smoother and slightly more robust at the longest segment length, but it also requires wider uncertainty bands. Overall, these results show that AmortizedSeqFlow is usable only for short local state-estimation windows; extending the segment length exposes the limitation of treating segments independently.

IV.2.2 Latent State Space Model Flow

To overcome the limitation of treating time steps independently, we impose a linear Gaussian **SSM!** (**SSM!**) directly in the latent space of the normalizing flow. This yields the architecture termed LatentSSMFlow, shown in Figure **??**. The model combines a structured temporal prior with a conditional normalizing flow decoder. A further motivation is to build a deep-learning-based Kalman filter or state-space model, as summarized in Sections **??** and **??**. In a classical Kalman filter, temporal propagation and uncertainty update are analytically tractable because the transition model is restricted to be linear and the observation model is Gaussian. Here, the linear Gaussian **SSM!** preserves this interpretable filtering structure in a latent space, while the flow decoder maps the latent Gaussian process to nonlinear and non-Gaussian physical state distributions. Thus, the model can be viewed as a flow-augmented state-space model that retains tractable temporal propagation and uncertainty updates while relaxing the model restrictions.

This idea is implemented by changing the base distribution of the flow to a structured dynamical prior. Instead of using an independent standard Gaussian base distribution at each time step, the latent trajectory $z_{1:T}$ follows a Markovian Gaussian process:

$$z_t = F z_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q). \tag{IV.32}$$

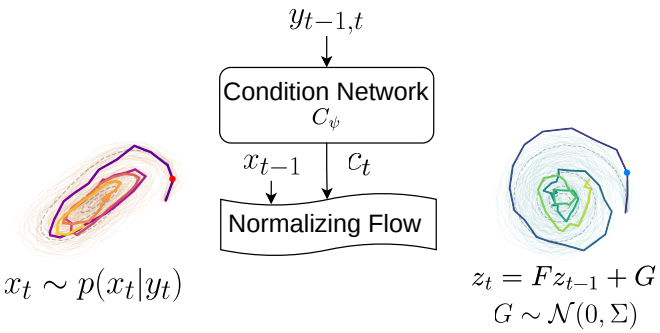


Figure IV.21: Architecture of Latent State Space Model Flow.

The transition matrix F is fixed before training so that the latent dynamics remain contractive or non-expansive. This is enforced by bounding the spectral radius:

$$\rho(F) = \max_i |\lambda_i| \leq \rho_{max} < 1,$$

where λ_i denotes an eigenvalue of F. The process noise covariance is written as $Q = LL^\top$, where L is a Cholesky factor. This parameterization guarantees that Q is a valid positive semidefinite covariance matrix. Since the transition matrix F is chosen to be stable, the latent linear Gaussian process has a stationary covariance P_∞ satisfying the discrete Lyapunov equation $P_\infty = FP_\infty F^\top + Q$. We sample the initial latent state from $\mathcal{N}(0, P_\infty)$, so that the latent trajectory starts from the stationary distribution of the base process.

The normalizing flow acts as a nonlinear decoder from the latent state space to the physical state space. At each step, it maps the latent variable z_t to the state x_t, conditioned on an observation embedding c_t and the previous state x_{t-1}. The conditioning vector c_t is obtained from an encoder applied to the observations. In a filtering setting, this encoder may use only local information such as y_t or $y_{t-1:t}$; in a smoothing setting, it may encode the full observation sequence $y_{1:T}$. Conditioning the flow on x_{t-1} introduces an explicit first-order autoregressive dependence in the decoded state trajectory, while the latent **SSM!** provides a global temporal prior.

The likelihood formulation requires an exact invertible decoder. For RealNVP, each state can be transformed into latent space during training because the architecture is bijective:

$$z_t = f_\phi^{-1}(x_t; c_t, x_{t-1}), \quad c_t = C_\psi(y_{t-1:t}). \tag{IV.33}$$

The model is then trained by **MLE!** (**MLE!**). Using the change-of-variable formula, the

log-likelihood of the state sequence is

$$\log p(x_{1:T}|y_{1:T}) = \log p_{SSM}(z_{1:T}) + \sum_{t=1}^{T} \log \left| \det \frac{\partial f_\phi(x_t; c_t, x_{t-1})}{\partial x_t} \right|, \tag{IV.34}$$

where $p_{SSM}(z_{1:T})$ is the likelihood of the latent trajectory under the linear Gaussian **SSM!**. Thus, the latent dynamics are modeled by the **SSM!**, while the nonlinear state transition is learned by RealNVP.

Flow Matching, however, is not suitable for an equivalent LatentSSMFlow decoder. Because it has no exact inverse map and no tractable log-determinant, the state trajectory cannot be mapped back to a latent trajectory whose probability is evaluated under $p_{SSM}(z_{1:T})$. Instead, the model is trained with a transport regression objective: samples from the latent **SSM!** prior are interpolated toward data states, and the network learns the corresponding velocity field. The latent **SSM!** therefore acts only as a source distribution for generation, not as a base measure coupled to the observed state trajectory through an exact change of variables. This distinction causes large training and evaluation mismatches. Thus, Flow Matching is omitted from the LatentSSMFlow comparison.

To improve training efficiency, we use a segmented multiple-shooting strategy. Each trajectory is divided into segments of length T_s. Within a segment, the model is unrolled autoregressively, so that prediction errors can influence later states in the same segment. Across segments, the initial state is anchored by the ground-truth state during training, and the segment computations can be parallelized. This strategy keeps the temporal training problem tractable while still exposing the model to multi-step state evolution.

Numerical Results

We evaluate the RealNVP-based LatentSSMFlow with the same Lorenz-96 dynamics, observation mask, trajectory length, and train-validation-test split as in the previous experiment. The main difference is the observation-noise model. To challenge the uncertainty modeling ability of LatentSSMFlow, the observation noise follows a Student-t distribution with $\nu = 5$ degrees of freedom and scale 0.5. Equivalently, the noise can be written as the Gaussian scale mixture

$$\tau \sim \text{Gamma}(\nu/2, \nu/2), \qquad e \sim \mathcal{N}(0, I), \qquad \eta_t = 0.5\, \frac{e}{\sqrt{\tau}}. \tag{IV.35}$$

The marginal noise variance is therefore $\nu(\nu-2)^{-1}0.5^2 = 5/12 \approx 0.4167$, corresponding to a marginal standard deviation of approximately 0.646. This heavier-tailed observation model makes the evaluation more challenging by allowing occasional large observation errors, similar to outlier-contaminated measurements in real systems.

During training, the previous state provided to the decoder is the ground-truth state. In the results shown here, we use models trained with segment lengths $T_s = 5, 10, 20, 25, 50$. Full-horizon training was not successful. Evaluation is then performed as a continuous autoregressive rollout over the full horizon. At the beginning of a test trajectory, the model is initialized from the given initial state. Thereafter, the previous state provided to the decoder is the model's own previous inferred state. Thus, the rollout tests whether LatentSSMFlow can maintain a coherent state estimate when its predictions are recursively fed back into the model.

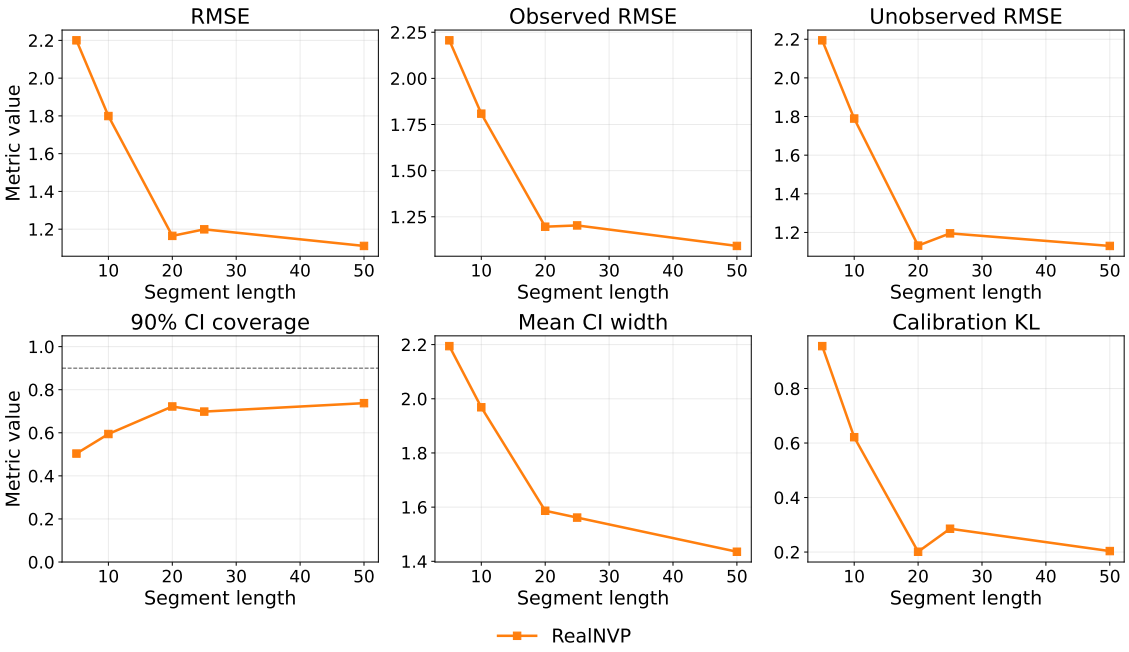


Figure IV.22: Full-horizon rollout performance obtained with the RealNVP-based LatentSSMFlow trained with segment length $T_s = 5, 10, 20, 25, 50$. The metrics are computed over the full 100-step horizon and include RMSE, 90% **CICP!**, **MCIW!**, and KL divergence between the estimated and true states.

The full-horizon rollout evaluation results show that the segment length used during training has a strong effect on long-term consistency. As shown in Figure **??**, increasing T_s consistently reduces the error over the 100-step autoregressive rollout. The total RMSE decreases from 2.2 for $T_s = 5$ to 1.1 for $T_s = 50$. The same trend is visible for both observed and unobserved state dimensions, indicating that longer training segments improve the learned latent dynamics rather than only the reconstruction of directly observed coordinates. This behavior contrasts with AmortizedSeqFlow in the previous subsection: here, longer segments expose the autoregressive decoder to longer multi-step dependencies during

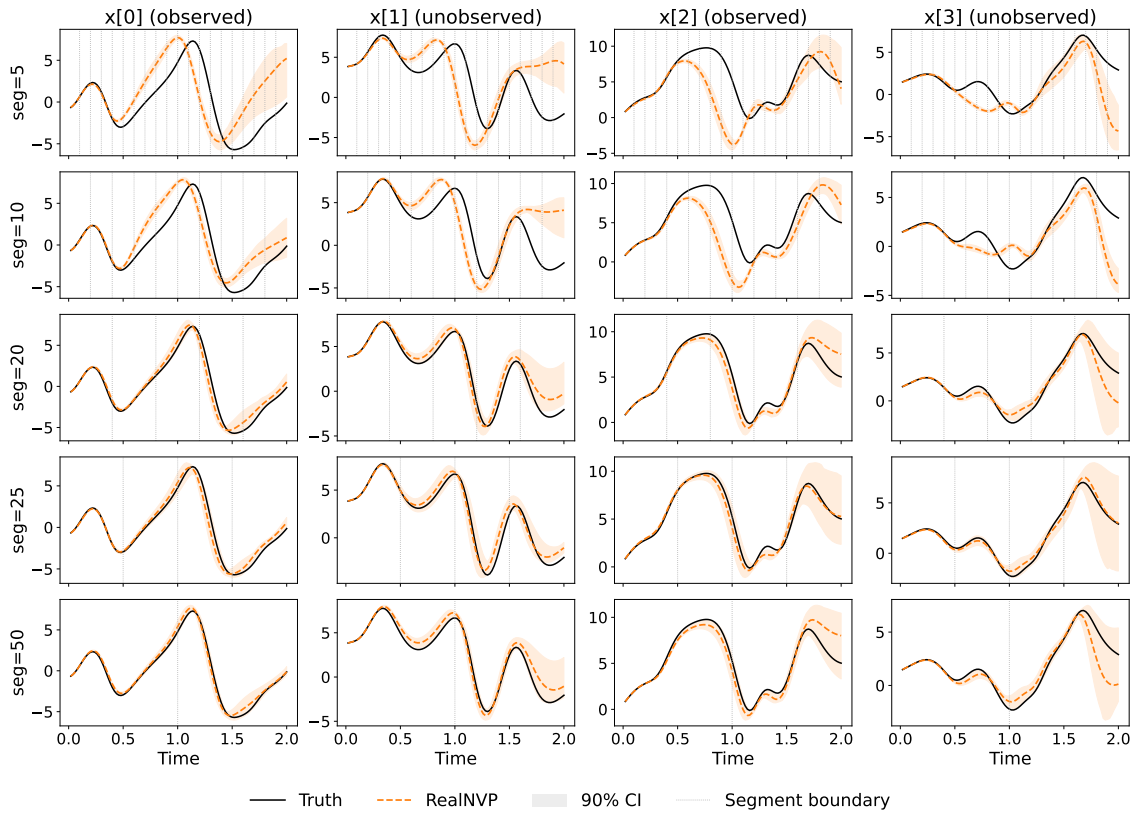


Figure IV.23: Full-horizon rollout trajectories obtained with the RealNVP-based LatentSSMFlow trained with segment length $T_s = 5, 10, 20, 25, 50$. The plots show observed and unobserved components over the full 100-step horizon.

training and therefore reduce the train–test mismatch in full-horizon rollout.

In terms of uncertainty quantification, the 90% **CICP!** scores remain below the nominal level for all training segment lengths. The **CICP!** increases as the segment length grows from $T_s = 5$ to $T_s = 20$, and then stays relatively stable for $T_s = 25$ and $T_s = 50$ at approximately 75%. For short segments, the model is underfit in the rollout sense: it has only learned local one-step or short-horizon dependencies, so the autoregressive prediction error grows quickly over the full horizon and the posterior intervals cannot cover the reference trajectory. Increasing T_s reduces this dynamic underfitting and improves the posterior mean, which explains the rise in **CICP!**. However, the coverage remains far below the nominal 90%, indicating that the posterior is still under-dispersed.

The simultaneous decrease in **MCIW!** suggests that longer-segment training makes the model more confident as the mean trajectory becomes more accurate. This is beneficial

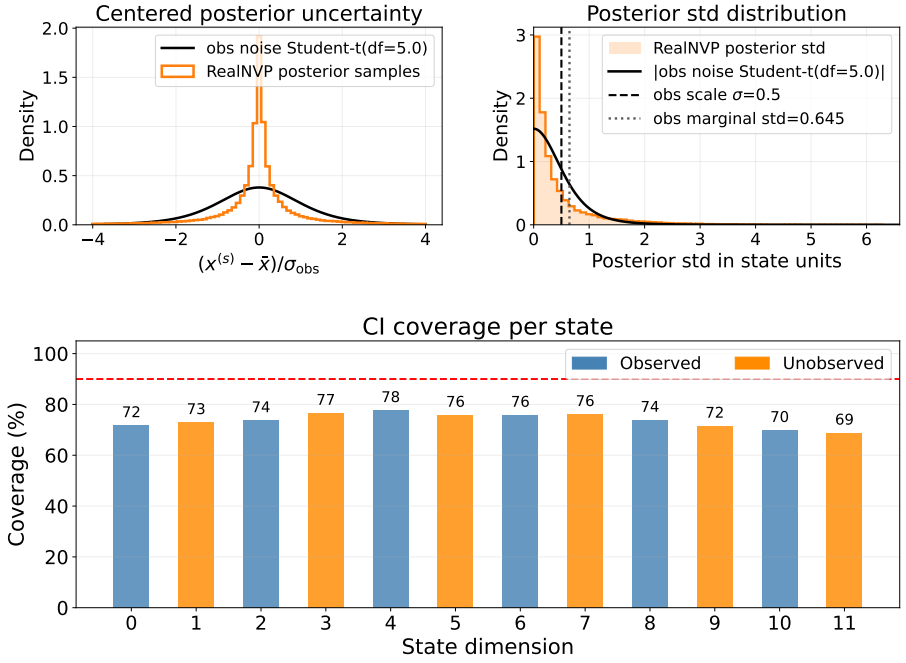


Figure IV.24: Uncertainty analysis for the RealNVP-based LatentSSMFlow trained with segment length $T_s = 50$ during full-horizon rollout. The left panel compares centered posterior samples with the Student-t observation-noise distribution after normalization by the observation scale. The middle panel compares the posterior standard-deviation distribution with the observation scale and marginal observation-noise standard deviation. The right panel reports the empirical 90% credible-interval coverage for each state dimension.

for sharpness, but it also means that the uncertainty does not expand enough to account for residual rollout error. The nearly unchanged KL divergence at $T_s > 20$ further supports this interpretation: although the mean prediction improves and the **CI!**s become narrower, the overall shape mismatch between the learned posterior and the reference uncertainty distribution is not substantially reduced. Thus, increasing the segment length mainly improves deterministic trajectory consistency, while the probabilistic calibration remains limited.

The representative trajectories in Figure **??** make the difference between models with different training segment lengths visually clear. Models trained with $T_s = 5$ and $T_s = 10$ can follow the trajectory locally, but their predictions accumulate errors when rolled out over the full horizon. The divergence is most apparent after major turning points and in the second half of the rollout. By contrast, the models trained with $T_s = 20, 25, 50$ maintain better alignment and reproduce the long-range trajectories more accurately. From the **CI!** perspective, longer-segment models cover the true trajectory more often because their

mean rollouts are more accurate, even though the intervals themselves become sharper as reflected by the decreasing **MCIW!**.

Figure **??** further examines the posterior uncertainty for the $T_s = 50$ model. The centered posterior samples are concentrated around their posterior mean, while retaining heavier tails than a narrow Gaussian-like distribution. This indicates that the model is confident for many time–state locations but still assigns probability to larger deviations when the rollout is harder to predict. The posterior standard deviations are mostly below or near the observation-noise scale 0.5, with only a limited right tail for high-uncertainty regions. The per-state coverage therefore remains below the nominal 90% level, consistent with the aggregate **CICP!** of approximately 75%.

Overall, these results reveal a structural limitation of LatentSSMFlow for long-horizon state estimation. Although the latent **SSM!** introduces temporal dependence, the decoder is trained mainly through local one-step transitions conditioned on the previous state. This is not sufficient to learn the full long-range dynamics of the partially observed chaotic system. As a result, the model can improve trajectory accuracy when longer training segments are used, but the learned posterior uncertainty remains inadequate: errors accumulated during rollout are not fully reflected in the predictive spread, leading to under-dispersed credible intervals for both observed and unobserved states.

IV.2.3 Normalizing Flow SDE

To further increase the capacity for modeling dynamics and stochasticity, we estimate the state trajectory by representing the underlying dynamics as a stochastic differential equation. We define the conditional state $x_t|y_t$ as a new system variable z_t. When the state evolves continuously in time, its dynamics can be written as

$$dz_t = f(z_t, t)\, dt + g(z_t, t)\, dW_t, \quad dW_t = \sqrt{dt}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, \boldsymbol{I}). \tag{IV.36}$$

By modeling f and g with neural networks, the model can represent nonlinear dynamics and state-dependent noise. However, training a NeuralSDE by directly maximizing the likelihood of the observed data can cause the stochastic component to collapse, since the networks may learn to ignore the diffusion term and fit the dynamics mainly through the drift.

One way to improve the expressiveness of the diffusion term is to use variational inference, as in LatentSDE (**?**). LatentSDE defines both a prior NeuralSDE and a posterior NeuralSDE, sharing the same diffusion function but using different drift functions. Because the diffusion term is shared, the KL divergence between the two stochastic processes is analytically tractable, allowing the model to be trained by maximizing the **ELBO!** (**ELBO!**). Another approach treats a NeuralSDE as an infinite-dimensional **GAN!** (**GAN!**) (**?**). In this

framework, the NeuralSDE acts as a generator that transforms Brownian motion into sample paths, while a Neural Controlled Differential Equation (NeuralCDE) acts as a discriminator on continuous-time trajectories. By optimizing a Wasserstein-type objective, this method can match the full distribution of generated paths to the data distribution in path space.

IV.2.3.1 Integration of Normalizing Flow in NeuralSDE

Both approaches, however, require substantial computational resources and careful tuning. To reduce this burden, we incorporate a normalizing flow directly into the NeuralSDE framework, avoiding the need for an additional differential equation to support training. For computational efficiency, we use the RealNVP architecture to model the diffusion term. Taking a standard normal random variable $\epsilon \sim \mathcal{N}(0, \boldsymbol{I})$ as the latent variable of the flow gives the following modified SDE:

$$dz_t = f_\varphi(z_t, t)\, dt + g_\phi(\epsilon; z_t, t)\, \sqrt{dt}, \quad \epsilon \sim \mathcal{N}(0, \boldsymbol{I}), \tag{IV.37}$$

where f_φ is the drift function and g_ϕ is the normalizing flow.

The factor $\sqrt{dt}$ is included so that the flow-generated random increment has the same time scaling as a standard **SDE!** (**SDE!**) diffusion term, where the variance of the stochastic step scales linearly with time ($\mathcal{O}(dt)$). Decoupling the normalizing flow's input ϵ from the time step dt avoids this noise collapse and preserves a genuine continuous-time diffusion process.

Unlike standard amortized Bayesian inference, the normalizing flow is not trained by minimizing a KL divergence against an available ground-truth posterior. Instead, the flow is used only in the forward stochastic direction and the latent variable ϵ is sampled from its standard normal distribution during training. Then, FlowSDE is trained by maximizing the likelihood of the observed data, which requires integrating the SDE (Equation **??**) through time. Several SDE solvers can be chosen, such as Euler–Maruyama, Heun, Midpoint, Milstein, or reversible Heun. The initial state is supplied to FlowSDE as part of the initial value problem and is then evolved by the SDE over all state coordinates.

The flow captures complex dependencies between the stochastic input and the current state, enabling the model to learn richer stochastic dynamics. An advantage of RealNVP is its simple Jacobian determinant, which simplifies likelihood computation and supports efficient training. However, RealNVP is non-volume-preserving. To prevent variance collapse, we apply hyperbolic tangent (tanh) clamping to the raw neural network scale output, parameterizing each diagonal scale factor as $\exp(\alpha \cdot \tanh(\cdot))$, where $\alpha > 0$ is a bounding hyperparameter. This restricts the scale factors to the open interval $(e^{-\alpha}, e^{\alpha})$. Thus, RealNVP preserves the capacity to represent expressive, varying noise levels across

the state space without suffering from stochastic collapse or gradient explosion.

IV.2.3.2 Training FlowSDE

The training objective is formulated probabilistically to prevent noise collapse. Rather than minimizing a deterministic trajectory-matching loss (such as **MSE!** (**MSE!**)), which can force the diffusion term to zero, we minimize the **NLL!** (**NLL!**). This objective combines the likelihood of the full state trajectory estimated by the SDE with the continuous-time transition likelihood (the normalizing flow loss) of the latent trajectory. Specifically, assuming a Gaussian state likelihood, the state loss is:

$$\mathcal{L}_{\text{state}} = \sum_t -\log \mathcal{N}\left(x_t; z_t, \sigma^2_{\text{state}} I\right), \tag{IV.38}$$

where x_t is the reference full state and σ^2_{state} controls the assumed state-likelihood variance.

To compute the transition loss $\mathcal{L}_{\text{trans}} = \sum_t -\log p(z_{t+dt}|z_t)$ under discretization in Equation **??**, we apply the change-of-variables formula of normalizing flows:

$$\mathcal{L}_{\text{trans}} = \sum_t \left(\frac{1}{2}\|\epsilon\|^2 + \log\left|\det \frac{\partial g_\phi(\epsilon; z_t, t)}{\partial \epsilon}\right| + \frac{D}{2}\log(2\pi dt)\right), \tag{IV.39}$$

where D is the dimension of the state space. The standard Gaussian negative log-likelihood of the latent variable ϵ is a constant. Thus, the transition loss simplifies to the Jacobian-determinant term. It is worth noting that there is a term related to dt in the transition loss, which reflects the time scaling of the diffusion term. For a fixed-step solver, this term is constant and does not affect the optimization. For an adaptive-step solver, since the solver's internal step does not directly affect the network parameters, the gradient of this term is still zero. Unless exact probability density is required, the actual dt used by the solver is not needed. Therefore, the dt-term can be treated as a constant during training.

Compared with the previous two methods, FlowSDE is more general: it can represent nonlinear long-term dynamics and state-dependent noise. This expressiveness comes with higher computational cost and more challenging training, since the **SDE!** must be integrated while the neural networks are optimized. The overall computational graph for FlowSDE is illustrated in Figure **??**.

We apply a multiple shooting framework to make long-horizon training computationally tractable and more stable. The full trajectory horizon T is divided into K shorter segments of length $T_s = T//K$. On each segment $k \in \{1, \ldots, K\}$, the SDE is integrated independently starting from a given initial state $z_0^{(k)}$. To address trajectory disjointedness, two strategies are integrated into this framework:

- **Continuity Constraints:** To force the disjointed segments to form a single contin-

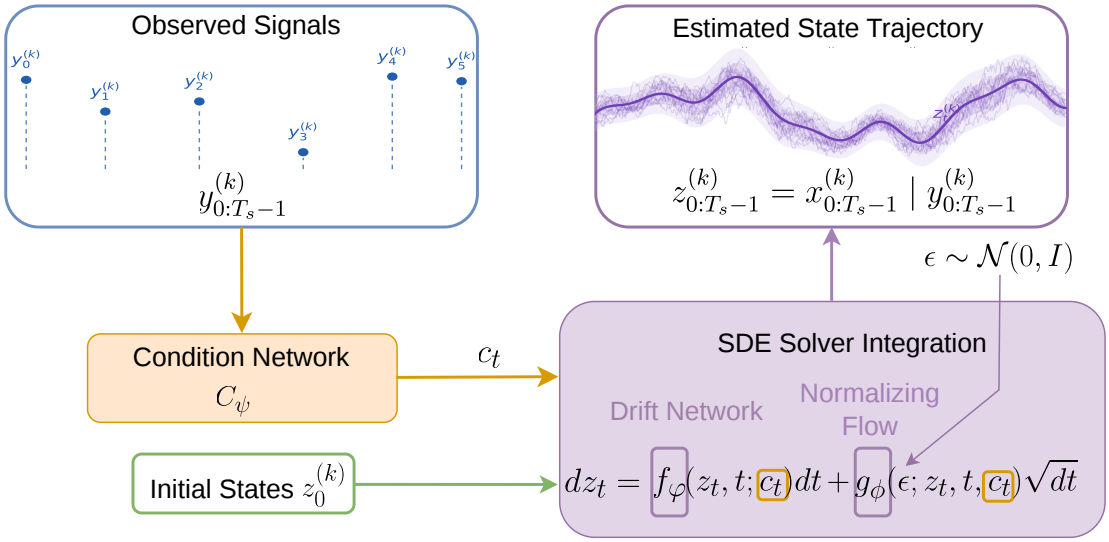


Figure IV.25: Computational graph of FlowSDE. For the k-th segment of length T_s, the supplied initial state $z_0^{(k)}$ initializes the SDE solver directly. The drift network f_φ and diffusion network g_ϕ model the deterministic and stochastic dynamics, respectively. The condition c_t is extracted from the segment observations by the condition network C_ψ. The stochastic solver integrates all state coordinates forward in time using FlowSDE.

uous trajectory in the continuous-time limit, we add a consistency penalty to the overall training loss:

$$\mathcal{L}_{\text{cont}} = \sum_{k=1}^{K-1} \|z_{\text{end}}^{(k)} - z_0^{(k+1)}\|^2 \tag{IV.40}$$

where $z_{\text{end}}^{(k)}$ is the terminal state predicted at the end of the k-th segment.

- **Longer Horizon Check:** To improve model performance and consistency over longer horizons, we implement a rollout loss within the multiple-shooting framework. For the sake of computational efficiency, the rollout loss is only added to the training loss every 5 optimizer steps. The rollout loss compares the generated trajectories to the true states over the whole trajectory using an energy-score-type objective:

$$\mathcal{L}_{\text{rollout}} = \sqrt{\mathbb{E}[(z-x)^2]} + \lambda_u \sqrt{\mathbb{E}[(z-z')^2]}. \tag{IV.41}$$

It consists of two terms: the first term measures the distance between the generated and true states, while the second term penalizes the pairwise spread of the generated trajectories. The hyperparameter λ_u controls the uncertainty weight.

- When $\lambda_u = 0.5$, the loss corresponds to the standard energy-score scaling and is a strictly proper scoring rule.
- Larger values, such as $\lambda_u = 0.8$, place more emphasis on preventing under-dispersed ensembles; this encourages wider trajectories, larger predictive intervals, and higher empirical coverage.
- Smaller values, such as $\lambda_u = 0.3$, prioritize mean trajectory accuracy and produce narrower intervals.
- In the limiting case $\lambda_u = 0$, the pairwise spread is ignored, so the rollout loss becomes a deterministic reconstruction objective and the learned stochastic variance can collapse.

In summary, the overall training objective for FlowSDE combines the state and transition losses with a weighted continuity penalty and a rollout loss.

$$\mathcal{L} = \mathcal{L}_{\text{state}} + \mathcal{L}_{\text{trans}} + \lambda_{\text{cont}}\mathcal{L}_{\text{cont}} + \lambda_{\text{rollout}}\mathcal{L}_{\text{rollout}}, \tag{IV.42}$$

where λ_* controls the strength of the corresponding term.

IV.2.3.3 Optional Theoretical Note: Weak Convergence of FlowSDE

This subsection provides the theoretical justification for the continuous-time limit of FlowSDE.

We first introduce two definitions for Markov-chain convergence (**??**).

Definition 3 (Weak Convergence)**.** *A sequence of discrete-time Markov chains converges weakly to a diffusion process if the laws of the interpolated processes converge in distribution to the solution of the corresponding stochastic differential equation as* $\Delta t \to 0$.

Definition 4 (Infinitesimal Generator)**.** *The infinitesimal generator* $\mathcal{A}$ *of a Markov chain is an operator that characterizes the expected instantaneous rate of change of a test function along the trajectories of the process. For a discrete-time Markov chain with transition kernel* $P_\Delta(z, dz')$*, the generator is defined as*

$$\mathcal{A}h(z) = \lim_{\Delta t \to 0} \frac{\int h(z')P_\Delta(z, dz') - h(z)}{\Delta t} \tag{IV.43}$$

for suitable test functions h.

Theorem 1 (Weak Convergence of FlowSDE)**.** *Let* $z_t \in \mathbb{R}^d$ *be a state variable whose discrete-time temporal transition over a step of size* $\Delta t > 0$ *is governed by the update rule:*

$$z_{t+\Delta t} = z_t + f_\varphi(z_t, t)\,\Delta t + g_\phi(\epsilon; z_t, t)\,\sqrt{\Delta t} \tag{IV.44}$$

where $\epsilon \sim \mathcal{N}(0, \boldsymbol{I}_d)$ is a standard Gaussian random vector. Assume that:

- *The drift function f_φ and the normalizing flow g_ϕ are continuously differentiable and satisfy global Lipschitz and linear growth conditions.*
- *The normalizing flow is centered conditioned on the state, such that $\mathbb{E}_\epsilon[g_\phi(\epsilon; z_t, t)] = 0$ for all z_t, t.*
- *The conditional covariance of the flow is defined as:*

$$\Sigma_\phi(z_t, t) = \mathbb{E}_\epsilon \left[g_\phi(\epsilon; z_t, t) g_\phi(\epsilon; z_t, t)^T \right] \tag{IV.45}$$

and has finite second- and higher-order moments.

*Then, as the step size $\Delta t \to 0$, the discrete-time sequence defined by (**??**) weakly converges to the unique continuous-time Itô stochastic differential equation:*

$$dz_t = f_\varphi(z_t, t)\, dt + \Sigma_\phi(z_t, t)^{1/2}\, dW_t \tag{IV.46}$$

where W_t is a standard Brownian motion.

Proof. The proof follows the standard diffusion-approximation argument for locally consistent Markov chains and Euler–Maruyama weak convergence (**???**). Let

$$\Delta z_t = z_{t+\Delta t} - z_t = f_\varphi(z_t, t)\Delta t + g_\phi(\epsilon; z_t, t)\sqrt{\Delta t}. \tag{IV.47}$$

For a test function $h \in \mathcal{C}_c^2(\mathbb{R}^d)$, the infinitesimal generator of the limiting process is identified from

$$\mathcal{A}h(z) = \lim_{\Delta t \to 0} \frac{\mathbb{E}\left[h(z + \Delta z_t) - h(z) | z_t = z\right]}{\Delta t}. \tag{IV.48}$$

Step 1: Taylor expansion. A second-order multivariate Taylor expansion around z gives

$$h(z + \Delta z_t) - h(z) = \nabla h(z)^T \Delta z_t + \frac{1}{2} \Delta z_t^T \nabla^2 h(z) \Delta z_t + R(\Delta z_t), \tag{IV.49}$$

where $\nabla h(z)$ is the gradient and $\nabla^2 h(z)$ is the Hessian of h at z. $R(\Delta z_t)$ is the remainder term, which satisfies:

$$|R(\Delta z_t)| \leq C \cdot |\Delta z_t|^3, \tag{IV.50}$$

where $C > 0$ is a constant related to the maximum value of the third derivative of h.

Step 2: First moment. Conditioning on $z_t = z$ and using the centering assumption on the flow $\mathbb{E}_\epsilon[g_\phi(\epsilon; z, t)] = 0$, the first moment of the increment is

$$\mathbb{E}[\Delta z_t | z_t = z] = \mathbb{E}_\epsilon \left[f_\varphi(z, t)\Delta t + g_\phi(\epsilon; z, t)\sqrt{\Delta t} \right] = f_\varphi(z, t)\Delta t. \tag{IV.51}$$

Thus the infinitesimal drift is $f_\varphi(z,t)$.

Step 3: Second moment. Using $x^T Ax = \mathrm{Tr}(Axx^T)$, the quadratic term in (**??**) is determined by $\mathbb{E}[\Delta z_t \Delta z_t^T | z_t = z]$. Expanding the update gives

$$\begin{aligned}\mathbb{E}[\Delta z_t \Delta z_t^T | z_t = z] = f_\varphi(z,t) f_\varphi(z,t)^T \Delta t^2 \\ + \mathbb{E}_\epsilon \left[f_\varphi(z,t) g_\phi(\epsilon; z, t)^T + g_\phi(\epsilon; z, t) f_\varphi(z,t)^T \right] \Delta t^{3/2} \\ + \mathbb{E}_\epsilon \left[g_\phi(\epsilon; z, t) g_\phi(\epsilon; z, t)^T \right] \Delta t. \end{aligned} \tag{IV.52}$$

The cross terms vanish because $\mathbb{E}_\epsilon[g_\phi(\epsilon; z, t)] = 0$, and therefore

$$\mathbb{E}[\Delta z_t \Delta z_t^T | z_t = z] = \Sigma_\phi(z,t)\Delta t + \mathcal{O}(\Delta t^2). \tag{IV.53}$$

Step 4: Remainder and small jumps. The finite higher-order moments of g_ϕ imply, for any $p > 2$,

$$\mathbb{E}\left[\|\Delta z_t\|^p | z_t = z \right] \leq C_p (\Delta t)^{p/2}. \tag{IV.54}$$

In particular, the Taylor remainder satisfies

$$\mathbb{E}\left[|R(\Delta z_t)| | z_t = z \right] = \mathcal{O}(\Delta t^{3/2}), \tag{IV.55}$$

and hence $\mathbb{E}\left[|R(\Delta z_t)| | z_t = z \right] / \Delta t \to 0$. This is the Lindeberg-type small-jump condition that excludes a discontinuous jump component in the weak limit.

Step 5: Limiting the infinitesimal generator. Substituting the Taylor expansion (**??**) into the generator definition (**??**) gives

$$\begin{aligned}\mathcal{A}h(z) = \lim_{\Delta t \to 0} \frac{1}{\Delta t} \Bigg[\nabla h(z)^T \mathbb{E}[\Delta z_t | z_t = z] \\ + \frac{1}{2} \mathrm{Tr} \left(\nabla^2 h(z) \mathbb{E}[\Delta z_t \Delta z_t^T | z_t = z] \right) + \mathbb{E}[R(\Delta z_t) | z_t = z] \Bigg]. \end{aligned} \tag{IV.56}$$

Using the first moment (**??**), the second moment (**??**), and the remainder bound (**??**), this

becomes

$$\begin{aligned}\mathcal{A}h(z) &= \lim_{\Delta t\to 0}\frac{1}{\Delta t}\Bigg[\nabla h(z)^T\left(f_\varphi(z,t)\Delta t\right)\\ &\qquad + \frac{1}{2}\,\mathrm{Tr}\left(\nabla^2 h(z)\left(\Sigma_\phi(z,t)\Delta t + \mathcal{O}(\Delta t^2)\right)\right) + \mathcal{O}(\Delta t^{3/2})\Bigg]\\ &= \lim_{\Delta t\to 0}\left[f_\varphi(z,t)^T\nabla h(z) + \frac{1}{2}\,\mathrm{Tr}\left(\nabla^2 h(z)\Sigma_\phi(z,t)\right) + \mathcal{O}(\Delta t) + \mathcal{O}(\Delta t^{1/2})\right]\\ &= f_\varphi(z,t)^T\nabla h(z) + \frac{1}{2}\,\mathrm{Tr}\left(\Sigma_\phi(z,t)\nabla^2 h(z)\right). \qquad \text{(IV.57)}\end{aligned}$$

To connect this operator with a continuous-time SDE, recall that an Itô diffusion of the form

$$dZ_t = b(Z_t,t)\,dt + \sigma(Z_t,t)\,dW_t \qquad \text{(IV.58)}$$

has infinitesimal generator

$$\mathcal{L}h(z) = b(z,t)^T\nabla h(z) + \frac{1}{2}\,\mathrm{Tr}\left(\sigma(z,t)\sigma(z,t)^T\nabla^2 h(z)\right). \qquad \text{(IV.59)}$$

Comparing this standard form with (**??**) shows that

$$b(z,t) = f_\varphi(z,t), \qquad \sigma(z,t)\sigma(z,t)^T = \Sigma_\phi(z,t). \qquad \text{(IV.60)}$$

Therefore, choosing $\sigma(z,t) = \Sigma_\phi(z,t)^{1/2}$ gives the limiting Itô SDE

$$dz_t = f_\varphi(z_t,t)\,dt + \Sigma_\phi(z_t,t)^{1/2}\,dW_t. \qquad \text{(IV.61)}$$

Thus, the discrete update has, in the limit $\Delta t \to 0$, the same infinitesimal generator as this Itô diffusion. By the standard generator convergence theorem for Markov processes, the Markov-chain approximation converges weakly to the stated continuous-time SDE. □

IV.2.3.4 Numerical Results

To further challenge the FlowSDE, the observation noise is modeled as a Gaussian mixture. Each observed component receives independent noise from

$$n_t \sim 0.70\,\mathcal{N}(0,\,0.25^2) + 0.15\,\mathcal{N}(1.25,\,0.75^2) + 0.15\,\mathcal{N}(-1.25,\,0.75^2). \qquad \text{(IV.62)}$$

This corresponds to a base observation scale of 0.5, with mixture offsets $(0, 2.5, -2.5)$ and component standard deviations $(0.50, 1.50, 1.50)$ expressed in units of this scale. The mixture is symmetric and therefore has zero mean overall, but it is multimodal and heavy-

tailed: most observations contain small noise around zero, while 30% of the samples are drawn from broader side components centered at ± 1.25. This noise model is intended to test whether the stochastic dynamics can represent non-Gaussian observation uncertainty and occasional large measurement errors.

Some important hyperparameters are as follows. The results reported below use Heun as the stochastic solver, with the segment size for multiple shooting $T_s = 5$, a rollout loss applied every 5 optimization steps with weight $\lambda_{\text{rollout}} = 1.0$, and an uncertainty weight of 0.6 in the energy score.

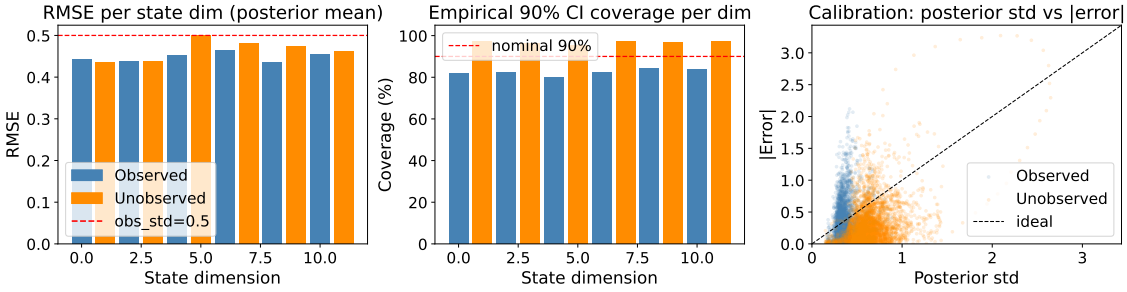


Figure IV.26: Estimation accuracy and calibration diagnostics of FlowSDE. The left panel shows the **RMSE!** of the posterior mean trajectory against the reference state trajectory. The center panel shows the confidence interval coverage probability of the posterior predictive distribution. The nominal coverage probability is 90%. In the right panel, the calibration scatter plot compares posterior standard deviations with realized absolute estimation errors.

The results in Figure **??** indicate that FlowSDE is able to recover the latent state trajectory with a favorable balance between accuracy and uncertainty calibration. In the RMSE comparison (left), the posterior mean remains below the observation-noise level for both observed and unobserved coordinates, showing that the method does not simply reproduce the noisy measurements but instead extracts a denoised estimate of the underlying state. The empirical coverage (middle) is 89.6%, which is very close to the nominal 90% level, and the calibration scatter (right) shows that larger posterior standard deviations tend to coincide with larger realized errors, suggesting that the stochastic component is being used meaningfully rather than collapsing to an overly confident deterministic fit. This shows that FlowSDE represents not only the mean dynamics but also the uncertainty induced by partial observations and chaotic evolution.

The multimodal results (Figure **??**) further demonstrate that FlowSDE captures the non-Gaussian structure of the posterior rather than collapsing onto a single narrow mode. The particle-based histograms (left) show that the posterior samples reproduce the skewed and multimodal behavior of the latent state distribution, while the predictive distribution

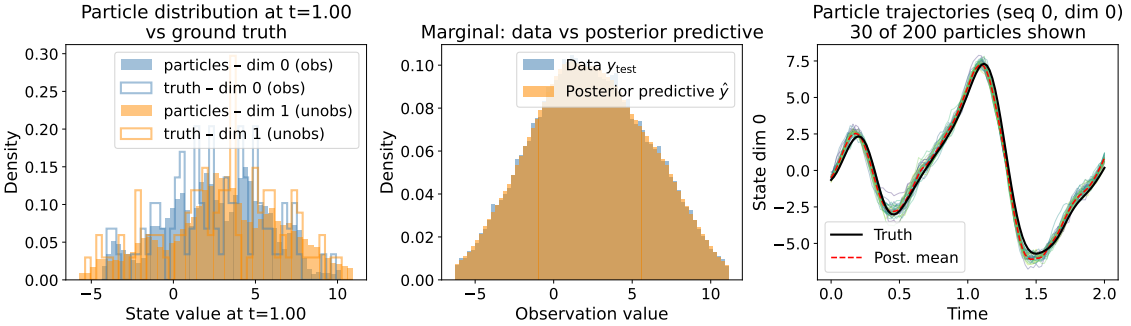


Figure IV.27: Posterior multimodality and predictive consistency of FlowSDE. Panel A shows histograms of posterior particle states at a fixed time, comparing observed and unobserved dimensions with the true state distribution to assess marginal shape, spread, and multimodality. Panel B compares real test observations with posterior predictive observations generated from sampled state trajectories and the same observation-noise model. Panel C shows posterior particle trajectories for one test sequence, where the black curve is the true state, the red dashed curve is the posterior mean, and the thin colored curves are sampled trajectories, illustrating uncertainty over time.

of observations (middle) remains consistent with the empirical test data. The particle trajectories (right) also remain dispersed around the true state, indicating that the method preserves diversity in the posterior while staying anchored to the underlying physical trajectory.

In the trajectory reconstructions (Figure **??**), the posterior mean follows the oscillatory Lorenz-96 dynamics closely over time, and the uncertainty bands widen in regions of rapid transition or higher uncertainty. Taken together, the RealNVPFlowSDE model demonstrates reliable tracking capabilities on the Lorenz-96 system under Gaussian mixture noise. It keeps the reconstruction RMSE below the observation noise level for both observed and unobserved dimensions, while maintaining an overall empirical coverage close to the nominal 90% target. Some minor calibration differences exist between observed and unobserved dimensions, but the model effectively captures the multimodal characteristics of the state space.

IV.2.4 Conditional Stochastic Interpolant

The preceding methods apply normalizing flow architectures to model state trajectories. Stochastic Interpolants (**?**) provide a different perspective, as introduced in Section **??**. Instead of inferring states from observations, we formulate state estimation as signal-to-signal refinement: an observed trajectory $y_{1:T}$ is gradually transformed into samples that follow the posterior trajectory distribution $p(x_{1:T}|y_{1:T})$. The gradual refinement is

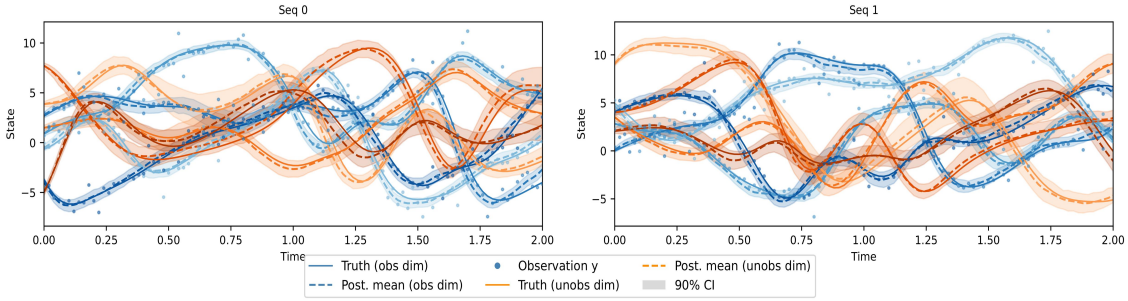


Figure IV.28: Trajectory reconstruction and uncertainty quantification of FlowSDE. Six representative test sequences are illustrated, where the curves are the posterior mean and the bands are 90% **CI!**s. The blue dots are the observed data points. The blue curves represent the observed trajectories whereas the red curves show the predicted trajectories.

parameterized by an artificial flow from time $\tau = 0$ to $\tau = 1$, and this transformation is modeled as a stochastic process.

IV.2.4.1 Conditional Stochastic Interpolant for State Estimation

In the classical generative setting, an interpolant connects a simple source distribution, such as a Gaussian prior, to the data distribution. For state estimation, the observations should constrain the generative process as conditioning information. We therefore define a conditional stochastic interpolant by

$$\mathcal{U}_0 \sim p_0(\cdot|y_{1:T}), \qquad \mathcal{U}_1 \sim p(x_{1:T}|y_{1:T}), \tag{IV.63}$$

where $p_0(\cdot|y_{1:T})$ is an observation-informed source distribution. The source distribution is obtained by an inference network q_η:

$$\left[\mu_\eta, \log \sigma_\eta^2\right] = q_\eta(y_{1:T}). \tag{IV.64}$$

The parameters μ_η and σ_η are reparameterized to build the source distribution, which guarantees differentiability and allows gradients to propagate back to the inference network:

$$\mathcal{U}_0 = \mu_\eta + \sigma_\eta \odot \epsilon_0, \qquad \epsilon_0 \sim \mathcal{N}(0, \boldsymbol{I}). \tag{IV.65}$$

This inference network provides an adaptive initialization for the interpolant, which can be trained end-to-end with the interpolant dynamics. The source distribution is not fixed as a simple prior; instead, it is learned to capture the information in the observations and to provide a good starting point for the refinement process.

To better extract information from observations, a conditioning network C_ψ is applied:

$$c = C_\psi(y_{1:T}). \tag{IV.66}$$

The conditioning information c is fed into the interpolant dynamics, which allows the refinement process to be directly informed by the observations at every step along the flow time τ.

Target Interpolant

The target interpolant can be defined using either a simple linear path or a smoother cosine path. For the default linear interpolant, we use

$$\mathcal{U}_\tau = \alpha(\tau)\,\mathcal{U}_0 + \beta(\tau)\,\mathcal{U}_1 + \sigma_\tau \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{IV.67}$$

with

$$\alpha(\tau) = 1 - \tau, \qquad \beta(\tau) = \tau, \qquad \sigma_\tau = \sigma_0 \tau(1 - \tau). \tag{IV.68}$$

The corresponding derivatives are

$$\alpha'(\tau) = -1, \qquad \beta'(\tau) = 1, \qquad \dot{\sigma}(\tau) = \sigma_0(1 - 2\tau), \tag{IV.69}$$

which define the supervised target velocity

$$v^* = \alpha'(\tau)\,\mathcal{U}_0 + \beta'(\tau)\,\mathcal{U}_1 + \dot{\sigma}(\tau)\epsilon. \tag{IV.70}$$

For the linear path, this target is the instantaneous direction of the noisy bridge from the source trajectory $\mathcal{U}_0$ to the target state trajectory $\mathcal{U}_1$.

Learned Interpolant Dynamics

The learned interpolant dynamics are described by a refinement **SDE!** with a deterministic transport term and a score-based correction:

$$d\mathcal{U}_\tau = \Big[v_\varphi(\mathcal{U}_\tau, \tau; \mu_\eta, c) + g_0^2 S_\phi(\mathcal{U}_\tau, \tau; \mu_\eta, c) \Big]\, d\tau + g_0 \sqrt{2\sigma_\tau}\, dW_\tau, \tag{IV.71}$$

where both neural fields, the transport velocity v_φ and the score network S_ϕ, are conditioned on the context c and on the encoder mean μ_η. The encoder mean μ_η provides a state-space estimate of the trajectory and gives the SDE a physically meaningful state-space reference.

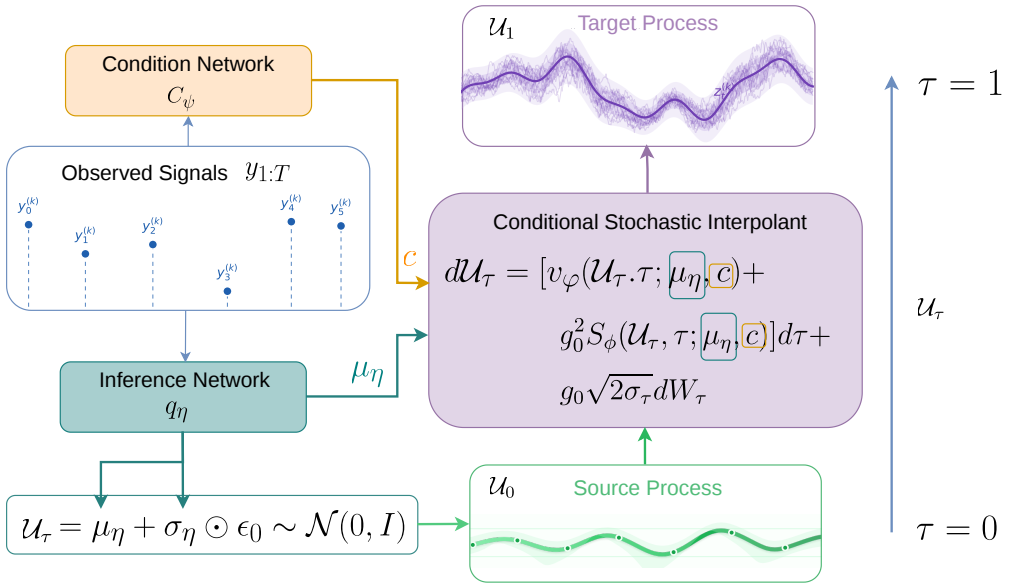


Figure IV.29: Computational graph of Conditional Stochastic Interpolant. The inference network q_η produces the source distribution parameters (μ_η, σ_η) from the observations $y_{1:T}$. The conditioning network C_ψ extracts conditioning information c from the observations. Both the mean parameter μ_η and the conditioning information c are used as conditions for the transport velocity network v_φ and the score network S_ϕ, which model the deterministic and stochastic components of the interpolant dynamics, respectively. The stochastic solver integrates the interpolant **SDE!** forward in flow time τ, starting from the source trajectory $\mathcal{U}_0$, to produce terminal trajectories that approximate samples from the posterior distribution over state trajectories.

The model is trained end-to-end with a joint encoder-interpolant objective:

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{flow}} + \mathcal{L}_{\text{score}} + \lambda \mathcal{L}_{\text{nll}}, \tag{IV.72}$$

where

$$\mathcal{L}_{\text{flow}} = \left\| v_\varphi(\mathcal{U}_\tau, \tau; \mu_\eta, c) - v^* \right\|^2, \tag{IV.73}$$

$$\mathcal{L}_{\text{score}} = \left\| S_\phi(\mathcal{U}_\tau, \tau; \mu_\eta, c) + \epsilon \right\|^2, \tag{IV.74}$$

$$\mathcal{L}_{\text{nll}} = -\log \mathcal{N}\Big(x_{1:T}; \mu_\eta, \text{diag}(\sigma_\eta^2)\Big). \tag{IV.75}$$

The flow loss learns the deterministic transport field, while the score loss trains the score network in a sigma-scaled form. The negative log-likelihood term trains the inference network to assign high probability to the reference state trajectory $x_{1:T}$ under the Gaussian

source distribution $\mathcal{N}(\mu_\eta, \mathrm{diag}(\sigma_\eta^2))$.

Since $\mathcal{U}_\tau$ and v^* depend on the sampled source $\mathcal{U}_0$ (see Equations **??** and **??**), the flow and score losses are differentiable with respect to $\mathcal{U}_0$. Consequently, gradients from the losses propagate through the interpolant path into the inference network q_η.

For a new observation trajectory, posterior estimation is performed by evaluating the inference network once, drawing N_p source particles from Equation **??** using independent samples of ϵ_0, solving Equation **??** with an Euler–Maruyama discretization, and collecting the terminal particles $\{u_1^{(i)}\}_{i=1}^{N_p}$. These terminal trajectories represent samples from the approximate posterior $p(x_{1:T}|y_{1:T})$.

CondStochInterpolant therefore acts as a Bayesian filter/smoother. Based on the observed data, the stochastic inference network produces an adaptive initial particle distribution, and the refinement **SDE!** transports those particles into a calibrated and potentially multimodal posterior over full state trajectories.

IV.2.4.2 Numerical Results

The observation noise is the same as in the FlowSDE experiments. The full-horizon results are shown in Figures **??**, **??**, and **??**.

Figure **??** summarizes the estimation accuracy and calibration behavior. The posterior mean reconstructs observed coordinates more accurately than unobserved coordinates, as expected from the partial-observation setting. The **RMSE!** is approximately 0.4 on observed states and approximately 0.6 on unobserved states, which is close to the observation-noise scale of 0.5. This indicates that the estimator's error is comparable to the intrinsic uncertainty in the measurements. Because CondStochInterpolant is a conditional generative model, its goal is not to drive the error of every generated trajectory to zero, but to learn a calibrated posterior distribution over plausible state trajectories. Under noisy and partially observed measurements, the posterior retains intrinsic uncertainty, so nonzero sample-level error is expected and should not be interpreted as a failure of the method. In addition, the refinement process successfully transfers information from observed coordinates to hidden ones, but the unobserved components remain intrinsically harder because they must be inferred through the learned dynamics and cross-coordinate correlations.

The empirical coverage is close to the nominal 90% level, suggesting that the posterior carries meaningful uncertainty. The calibration scatter further supports this interpretation: larger posterior standard deviations tend to coincide with larger realized errors, so the model assigns higher uncertainty to cases where its estimates are less reliable.

Figure **??** illustrates the distributional behavior of the posterior particles. The particle distribution is concentrated around the reference state values while retaining a visible

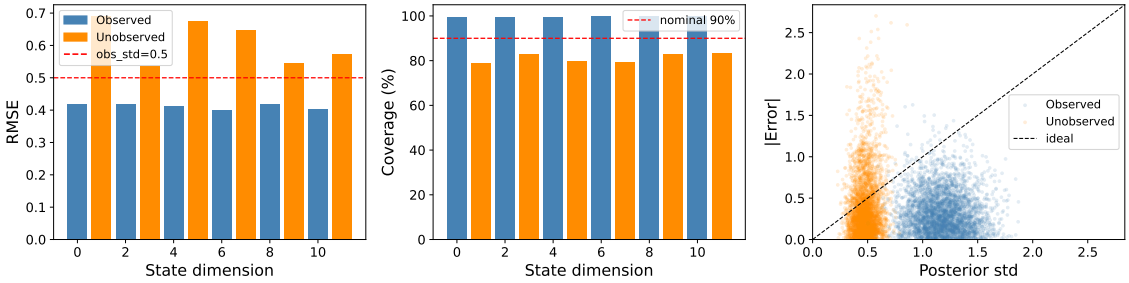


Figure IV.30: Estimation accuracy and calibration diagnostics of CondStochInterpolant. The left panel shows the **RMSE!** of the posterior mean trajectory against the reference state trajectory. The center panel shows the confidence interval coverage probability of the posterior predictive distribution. The nominal coverage probability is 90%. In the right panel, the calibration scatter plot compares posterior standard deviations with realized absolute estimation errors.

spread, which shows that CondStochInterpolant does not collapse to a single deterministic trajectory. The comparison between the empirical observation distribution and the posterior predictive distribution suggests that the model captures the broad shape of the noisy data distribution, including the heavy-tailed and multimodal effects introduced by the mixture noise. The representative particle trajectories also show that uncertainty is propagated coherently through time rather than appearing as independent pointwise noise. This is important for state estimation, since plausible posterior samples should remain dynamically consistent over the full horizon.

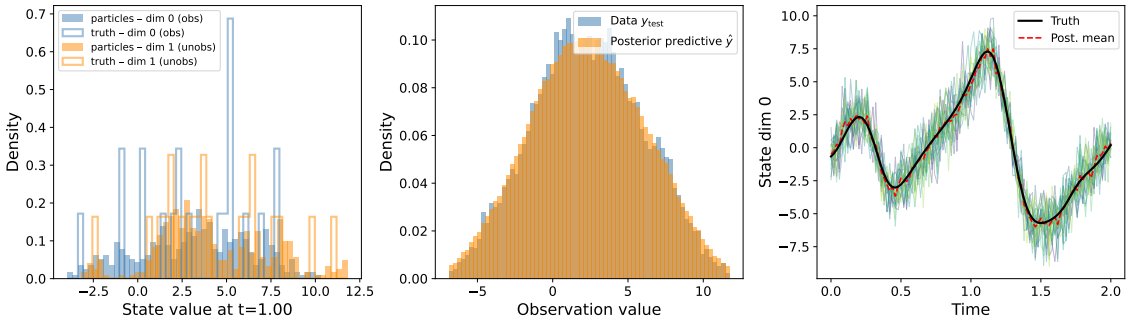


Figure IV.31: Posterior multimodality and predictive consistency of CondStochInterpolant. The left panel compares the particle distribution against the ground-truth state values. The center panel compares the empirical distribution of the test observations with the posterior predictive distribution. The right panel shows posterior particle trajectories for a representative state dimension.

The trajectory reconstructions in Figure **??** provide a more local view of this behavior. For the displayed test sequences, the posterior mean follows the reference trajectory closely over most of the horizon, while the 90% confidence bands widen around more uncertain or rapidly changing regions. The observed data points are noisy and occasionally deviate from the underlying state, but the inferred trajectories remain smoother than the raw observations. This indicates that the model uses the refinement **SDE!** as a denoising and data-generation mechanism rather than directly interpolating noisy measurements.

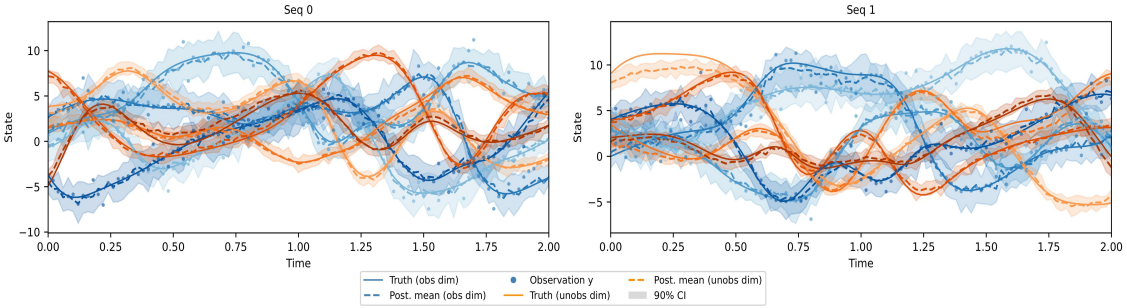


Figure IV.32: Trajectory reconstruction and uncertainty quantification of CondStochInterpolant. Six representative test sequences are illustrated, where the curves are the posterior mean and the bands are 90% **CI!**s. The blue dots are the observed data points. The blue curves represent the observed trajectories whereas the red curves show the predicted trajectories.

Overall, the results show that CondStochInterpolant as a state estimator achieves a favorable balance between reconstruction accuracy, uncertainty calibration, and posterior diversity under multimodal observation noise.

IV.2.5 Summary of State Estimation Methods

Across the four methods, a clear trade-off emerges between reconstruction accuracy, uncertainty quantification, and the ability to capture long-range temporal structure. AmortizedSeqFlow is the simplest and most computationally efficient approach, and it performs reasonably accurately for short local windows. However, because it treats each segment independently, it is less effective when the horizon becomes longer or when the dynamics are strongly nonlinear. LatentSSMFlow improves upon this by introducing a structured latent temporal prior, which helps preserve temporal consistency and leads to better long-horizon rollouts than AmortizedSeqFlow. Nevertheless, its ability to model system dynamics is limited, and the uncertainty estimates remain under-dispersed.

FlowSDE provides a more expressive framework for modeling nonlinear dynamics and stochasticity. It achieves strong trajectory reconstruction and more meaningful uncertainty

calibration because the flow-based diffusion term allows the model to represent both dynamical uncertainty and multimodal posterior structure. CondStochInterpolant treats state estimation as a refinement process from an observation-informed source distribution to the posterior trajectory distribution; although it cannot yield the most precise point estimates because of its stochastic procedure, it provides a flexible and expressive posterior estimator that captures multimodal uncertainty and preserves temporal coherence under partially observed, non-Gaussian noise.

IV.3 Optimal Control

From a general perspective, the state-estimation methods discussed in the previous section transform one temporal sequence into another. These two sequences are related through the underlying dynamical system. The same perspective also applies to the control framework: in an **MPC!** (**MPC!**) setting, the simulation model maps the current state and control input to the next state; a control policy finds the next control input from the target and observations; a path planner generates a sequence of future control inputs from the current state and the relevant history. In this section, we will discuss how to integrate the normalizing-flow-based methods into control frameworks.

The four methods are applied differently in a control framework due to their different characteristics.

1. **AmortizedSeqFlow** is based on training a desired mapping offline and then directly using the trained model online. Thus, the optimization process of finding the optimal control input is performed implicitly offline by training the posterior of control inputs conditioned on the observed and reference signals. Once trained, the control input can be inferred directly.

2. **LatentSSMFlow** applies a linear state space model in the latent space, which is natural to adapt it to capturing the underlying dynamics of the system. The system states are then modeled stochastically by taking the previous states and inputs as conditions, i.e., $x_t \leftarrow (x_{t-1}, u_{t-1})$. Then, the control input is optimized in a stochastic **MPC!** framework.

3. **FlowSDE**, likewise, can also model the system dynamics and be integrated into a stochastic **MPC!** framework.

4. **CondStochInterpolant** is a conditional generative model, which can be used as a control policy or path planner to generate sequential control inputs conditioned on the current system states and observation histories.

IV.3.1 AmortizedSeqFlow for Control

Recall the Section **??**, a standard **MPC!** requires solving a constrained trajectory optimization problem at every single environment time step. This is highly compute-intensive and can cause execution latencies. At every time step for updating the system, a sequence of future control inputs are supposed to be optimized based on the observed and referred signals. This mapping from observations and references to control inputs can be learned by the AmortizedSeqFlow. It bypasses this iterative optimization loop entirely and effectively "amortizes" the optimization cost.

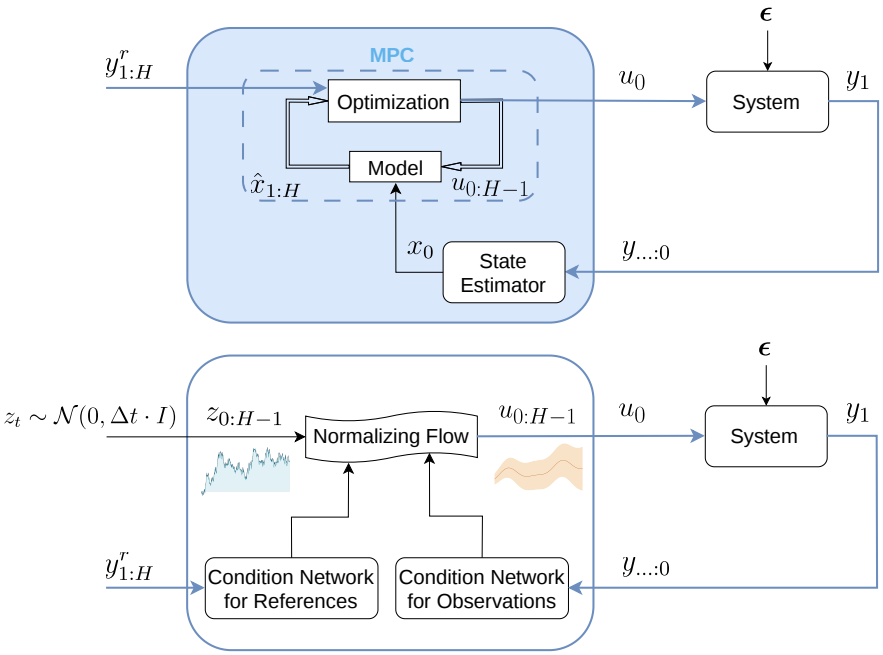


Figure IV.33: Applying AmortizedSeqFlow as an **MPC!**. The latent variable of the normalizing flow $z_{0:H-1}$ is sampled from a Wiener process. On the other side of the normalizing flow, the target is the control input sequence $u_{0:H-1}$. Two condition networks are applied for next-step references $y^r_{1:H}$ and history observations $y_{...:0}$, respectively.

As shown in Figure **??**, the whole **MPC!** block including the optimization and model, together with the state estimation are replaced by a two-head conditional normalizing flow. One head extracts information from the reference signals, while the other head is a similar condition network for the history observations. The normalizing flow transforms a latent sequence to the control input sequence under the conditions from both heads.

Once the action chunk $u_{0:H-1}$ is generated, the first control action u_0 is extracted and applied to the system (the physical robot or plant). The system interacts with environmental disturbances/noise ϵ and advances one step, producing a new observation y_1. The history buffer is then updated ($y_{...:0} \rightarrow [y_{...:0}, y_1]$) and ready for the next control cycle.

Comparing with the classical **MPC!**, AmortizedSeqFlow offers several advantages. First of all, by replacing the iterative optimization loop with a single feedforward pass through the normalizing flow, the controller has a constant execution time, which helps prevent control loop delays. Secondly, AmortizedSeqFlow integrates a state estimator with the controller, which utilizes information from history observations as much as possible. Thirdly, the normalizing flow implicitly learns the system's dynamic constraints from offline training data, ensuring that the generated control input sequences are dynamically feasible

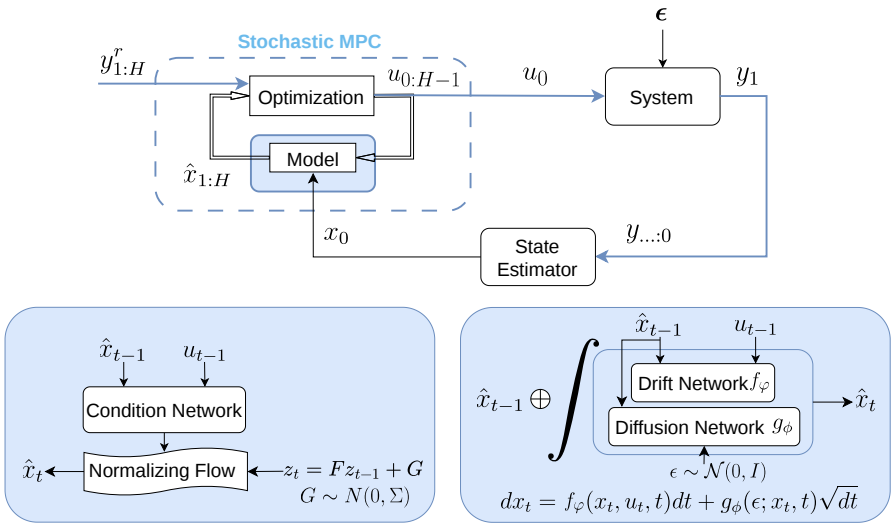


Figure IV.34: Application of LatentSSM or FlowSDE for the Surrogate Model in Stochastic **MPC!**. In LatentSSMFlow, the latent state z_t propagates through linear transition equations $z_t = Fz_{t-1} + G$, where $G \sim \mathcal{N}(0, \Sigma)$ represents system noise. A Normalizing Flow maps the latent state z_t back to the physical state representation $\hat{x}_t$ under the previous state $\hat{x}_{t-1}$ and input u_{t-1} as conditions. In FlowSDE, the state $\hat{x}_t$ is updated by integrating the transition dynamics. The dynamics are modeled by an **SDE!**, where the drift network f_φ learns the deterministic trend and the flow-integrated diffusion network g_ϕ learns the state-dependent uncertainty.

for the system to execute.

IV.3.2 LatentSSMFlow and FlowSDE for Control

A stochastic **MPC!** relies heavily on a surrogate dynamics model to project future system behaviors over a prediction horizon H. When the underlying physical system is highly nonlinear or exhibits multi-modal stochasticity, standard deterministic or Gaussian dynamics models fail. This method resolves this by modeling transitions using either a LatentSSMFlow or a FlowSDE. As shown in Figure **??**, the stochastic **MPC!** iteratively chooses candidate inputs $u_{0:H-1}$ through the dynamics model to predict next-step states $\hat{x}_{1:H}$ and select the optimal sequence that tracks a reference trajectory $y^r_{1:H}$.

IV.3.3 CondStochInterpolant for Control

The CondStochInterpolant control policy generates action chunk based on history observations and current state, as shown in Figure **??**. At the beginning of environment step

$t = 0$, the agent retrieves the sliding window of historical observations $O_{\dots:0}$. A condition network C_ψ embeds these temporal observations into a low-dimensional embedding vector c, which provides context information for the generative process of the stochastic interpolant. Besides, a inference network q_η estimates the statistical parameters (μ_η, σ_η) of a prior distribution. Then, the prior distribution is reparameterized with these parameters and has the same dimension of the target action chunk, $\mathcal{U}_0 \in \mathbb{R}^{D_a \times H}$. The coarse trajectory $\mathcal{U}_0$ is transformed into a high-fidelity action chunk $\mathcal{U}_1$ by numerically integrating the **SDE!** from $\tau = 0$ to $\tau = 1$:

$$d\mathcal{U}_\tau = \Big[v_\varphi(\mathcal{U}_\tau, \tau; \mu_\eta, c) + g_0^2 S_\phi(\mathcal{U}_\tau, \tau; \mu_\eta, c)\Big]\, d\tau + g_0\sqrt{2\sigma_\tau} dW_\tau. \tag{IV.76}$$

The agent generates an action chunk $a_{0:H-1}$ in closed loop. Taking the actions, the environment updates the new observations $O_{1:H}$. To update the networks $(\varphi, \phi, \eta, \psi)$ online using **RL!** (**RL!**), the generated action chunks are evaluated against a reward function r, which also considers the history and updated observations.

Compared with other diffusion policies, the CondStochInterpolant control policy provides several advantages:

- Standard diffusion models typically initiate their denoising process from pure, isotropic Gaussian noise, requiring a lot of flow steps to produce coherent results. By contrast, the CondStochInterpolant begins from the "warm start" action chunk $\mathcal{U}_0$, which reduces the flow steps significantly.
- The stochastic interpolant formulation unifies probability flow **ODE!**s and **SDE!**s within a single mathematical framework, which allows configuring the solver dynamically at test-time without retraining.
- Predicting cohesive action chunks $a_{1:H}$ rather than step-by-step actions helps mitigate compounding tracking errors, smoothens the trajectory, and generates a long sequence fast in parallel. The horizon length H can be adjusted at test-time without retraining the model by varying the dimension of $\mathcal{U}_0$.

IV.3.4 Numerical Results

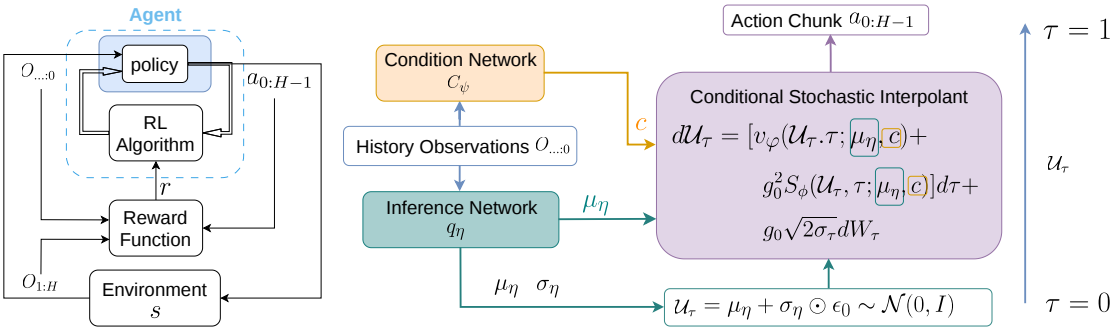


Figure IV.35: Closed-loop reinforcement learning agent utilizing a Conditional Stochastic Interpolant policy for action chunk generation. At timestep $t = 1$, the policy processes past observation history $O_{...:0}$ and the condition network C_ψ maps these conditions to a context embedding c. Simultaneously, the inference network q_η estimates the statistical parameters (μ_η, σ_η) of a prior distribution. These parameters are used to sample an initial noisy chunk $\mathcal{U}_0$ at flow time $\tau = 0$, which has the shape as the to be generated action chunks. The policy integrates an **SDE!**, which is driven by a velocity network v_φ and a score network S_ϕ. Both networks are conditioned on the mean parameter μ_η and the context embedding c. At flow time $\tau = 1$, the integrated trajectory yields the final, smooth action chunk $a_{1:H}$, which is then executed in the environment.

Chapter V

Experiments and Results

V.1 Overview

V.2 Parameter Identification

von der pol single track car quadcopter

V.3 State Estimation

V.4 Optimal Control

Chapter VI

Conclusion and Future Work

Abstract

Zusammenfassung

Own Contributions

This chapter gives an overview of my contributions in distinction to team efforts.

Own share in data acquisition and data analysis

Own share in Data Acquisition.

Own share in Data Analysis.

Own Publications

In this section, all publications that I was a part of and contributed to during my Ph.D. work are listed. It is subdivided into *First Authorships* and *Co-Authorships*.

First Authorships - Peer Reviewed International Journal Publications

Y. Zhang, L. Mikelsons. **Advanced Modeling and Simulation in Engineering and Science**. *Sensitivity-guided iterative parameter identification and data generation with BayesFlow and PELS-VAE for model calibration.* doi:10.1186/s40323-023-00246-y.

First Authorships - Peer Reviewed International Conference Publications

Y. Zhang, L. Mikelsons. **2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)**. *Solving Stochastic Inverse Problems with Stochastic BayesFlow.* doi:10.1109/AIM46323.2023.10196190.

Y. Zhang, L. Mikelsons. **2024 IEEE 63rd Conference on Decision and Control(CDC)**. *Inverse Modeling with Conditional Normalizing Flows for Data-Driven Model Predictive Control*. doi:10.1109/CDC56724.2024.10886862.

Co-Authorships

M.E. Heringhaus, Y. Zhang, A. Zimmermann, L. Mikelsons. **Sensors 2022**. *Towards Reliable Parameter Extraction in MEMS Final Module Testing Using Bayesian Inference*. doi:10.3390/s22145408.

Appendix

Acknowledgement

Eidesstattliche Versicherung

Statutory Declaration

1. Bei der eingereichten Dissertation zu dem Thema ***Deep Bayesian Inference for Control: Normalizing Flow–Based Approaches to System Identification, State Estimation, and Optimal Control*** handelt es sich um meine eigenständig erbrachte Leistung.
   I herewith formally declare that I have written the submitted dissertation ***Deep Bayesian Inference for Control: Normalizing Flow–Based Approaches to System Identification, State Estimation, and Optimal Control*** *independently.*
2. Ich habe nur die angegebenen Quellen und Hilfsmittel benutzt und mich keiner unzulässigen Hilfe Dritter bedient. Insbesondere habe ich wörtlich oder sinngemäß aus anderen Werken übernommene Inhalte als solche kenntlich gemacht.
   I did not use any third party support except for the quoted literature and other sources mentioned in the text. Content from other work, either literally or in content, has been declared as such.
3. Die Arbeit oder Teile davon habe ich bislang nicht an einer Hochschule des In- oder Auslands als Bestandteil einer Prüfungs- oder Qualifikationsleistung vorgelegt.
   The thesis has not been submitted to any examination body in this, or similar, form.
4. Die Richtigkeit der vorstehenden Erklärungen bestätige ich.
   I confirm the correctness of the aforementioned declarations.
5. Die Bedeutung der eidesstattlichen Versicherung und die strafrechtlichen Folgen einer unrichtigen oder unvollständigen eidesstattlichen Versicherung sind mir bekannt. Ich versichere an Eides statt, dass ich nach bestem Wissen die reine Wahrheit erklärt und nichts verschwiegen habe.
   I am aware of the legal consequences of this declaration. To the best of my knowledge I have told the pure truth and not concealed anything.

Augsburg, 08.07.2026 _____

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel

Information on the use of AI-based tools

Zur Dokumentation der verwendeten Hilfsmittel ist der schriftlichen Ausarbeitung ein besonderer Anhang hinzugefügt, der eine Liste und Beschreibung aller verwendeter KI-basierter Hilfsmittel enthält. Der besondere Anhang zur Dokumentation der verwendeten Hilfsmittel erfüllt folgende Kriterien:

To document the tools used, a special appendix has been added to the written report, which contains a list and description of all AI-based tools used. The special appendix to document the tools used meets the following criteria:

1. Auflistung der Ziele, für die die KI-basierten Hilfsmittel in der vorliegenden Arbeit eingesetzt wurden.
   List of the goals for which the AI-based tools were used in this work.

2. Dokumentation der Verwendungsweise der KI-basierten Hilfsmittel.
   Documentation of how AI-based tools are used.

3. Nennung der Kapitel und Abschnitte der vorliegenden Arbeit, in denen die KI-basierten Hilfsmittel eingesetzt wurden, um Inhalte zu erzeugen.
   Identification of the chapters and sections of this work in which AI-based tools were used to generate content.

Der Gebrauch dieser Hilfsmittel inklusive Art, Ziel und Umfang des Gebrauchs wurde mit meinem offiziellen Betreuer **Prof.Dr.Ing. Lars Mikelsons** abgesprochen.

*The use of these aids, including the type, purpose and extent of use, has been agreed with my official supervisor **Prof.Dr.Eng. Lars Mikelsons**.*

Mir ist bewusst, dass insbesondere der Versuch einer nicht dokumentierten Nutzung KI- basierter Hilfsmittel als Täuschungsversuch zu werten ist:

I am aware that in particular the attempt to use AI-based tools without documentation is to be regarded as an attempt at deception:

Gem. § 16 Abs. 2 der Promotionsordnung „Dr. med. / dent.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Kandidat/die Kandidatin bei einer Promotionsleistung getäuscht hat, so können einzelne oder alle Promotionsleistungen für ungültig erklärt werden. In schweren Fällen kann die Zulassung zum Promotionsverfahren zurückgenommen werden.“

According to § 16 paragraph 2 of the doctoral regulations for "Dr. med. / dent.": "If it emerges before the doctoral certificate is issued that the candidate has cheated in a doctoral achievement, individual or all doctoral achievements can be declared invalid. In serious cases, admission to the doctoral process can be withdrawn.

Und § 16 Abs. 2 der Promotionsordnung „Dr. sc. hum.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Doktorand / Doktorandin bei einer Promotionsleistung getäuscht hat, so kann der Promotionsausschuss diese Promotionsleistung oder alle bisher erbrachten Promotionsleistungen für ungültig erklären. In besonders schweren Fällen kann der Promotionsausschuss die Annahme als Doktorand / Doktorandin endgültig widerrufen.“

And § 16 paragraph 2 of the doctoral regulations “Dr. sc. hum.”: “If it emerges before the doctoral certificate is issued that the doctoral candidate has cheated in a doctoral achievement, the doctoral committee can declare this doctoral achievement or all doctoral achievements to date invalid. In particularly serious cases, the doctoral committee can permanently revoke the acceptance as a doctoral candidate.”

Augsburg, 08.07.2026 _____